

AIRCRAFT DESIGN OPTIMIZATION USING SIGNOMIAL PROGRAMMING

A THESIS

Presented to the Department of Mechanical and Aerospace Engineering
California State University, Long Beach

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Aerospace Engineering

Committee Members:

Cody J. Karcher, Ph.D. (Chair)
Ehsan Madadi, Ph.D.
Jingyi Zeng, Ph.D.

College Designee:

Kerop Janoyan, Ph.D.

By David Benjamin Avila

B.S., 2023, California State University, Long Beach

December 2025

ABSTRACT

Multidisciplinary design optimization (MDO) in aircraft design presents a range of interconnected challenges across aerodynamics, propulsion, structures, and performance. One of the primary difficulties lies in coordinating the interaction between subsystems while maintaining computational efficiency. Traditional sequential design methods often converge to suboptimal or locally optimal solutions, limiting their utility in increasingly demanding aerospace applications. Recent advancements in mathematical programming, along with improved solver capabilities, offer opportunities for more integrated and globally efficient design strategies. This paper extends previous research on optimization-centered approaches to aircraft sizing by developing a signomial programming framework for the conceptual design of commercial transport aircraft. A Python-based model is implemented to integrate aerodynamic loading, structural constraints, weight estimation, and mission performance within a unified optimization environment. The objective is to minimize the maximum takeoff weight (MTOW) while satisfying all governing constraints. By expressing the problem in a signomial form, the formulation effectively captures nonlinear and non-convex system interactions while remaining computationally tractable. The resulting configurations demonstrate that signomial programming can produce high-fidelity, feasible designs that align with industry expectations. Additionally, the model architecture enables easier scaling to higher fidelity or more detailed subsystem analyses. This research reinforces the applicability of convexification strategies in early stage aircraft design and highlights the potential for extending such methods toward more complex multidisciplinary environments.

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my mentor and advisor, Dr. Cody Karcher, whose guidance and insight have been invaluable throughout my graduate research. His dedication, expertise, and patience have profoundly influenced my growth, not only as a student but also as a person.

I also wish to thank my thesis committee members, Dr. Ehsan Madadi and Dr. Jingyi Zeng, for their time, thoughtful feedback, and continuous support in reviewing my work.

My sincere appreciation goes to everyone at the FVD Lab for providing constructive feedback during our weekly meetings. A special thanks to Michael Shoda and Brian "Bobo" Contreras for generously sharing their time and knowledge to help with this project, I am truly indebted to you both.

I am deeply thankful to my family—my mother, father, and brother—as well as my extended family for their unwavering support and patience throughout this academic journey. Their love and encouragement have been the foundation that allowed me to pursue my ambitions with confidence.

Finally, I want to thank all my friends, including those with whom I may have lost touch over time. Each of you has played a part in shaping the person I am today, and for that, I am sincerely grateful.

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGMENTS	iii
LIST OF TABLES	v
LIST OF FIGURES	vi
1. INTRODUCTION	1
2. METHODOLOGY	18
3. RESULTS	47
4. CONCLUSIONS AND OUTLOOK	63
APPENDICES	65
A. FULL MODEL	66
B. PYTHON SCRIPT	80
C. VARIABLES AND CONSTANTS	89
D. RESULTS	96
REFERENCES	100

LIST OF TABLES

3.1	Constant Parameters and Reference Values used in Aircraft Model	48
3.2	Optimized Mission Variables	49
3.3	Constants set in the Wing Model	50
3.4	Optimized Wing Geometry Variables	51
3.5	Optimized Wing Lift Variables	51
3.6	Optimized Wing Drag Variables	51
3.7	Optimized Wing Weight Variables	52
3.8	Optimized Vertical Tail Geometric Variables	53
3.9	Optimized Vertical Tail Weight Variables	54
3.10	Optimized Vertical Tail Aerodynamic Variables	54
3.11	Optimized Horizontal Tail Geometric Variables	56
3.12	Optimized Horizontal Tail Aerodynamic Variables	56
3.13	Optimized Horizontal Tail Weight Variables	56
3.14	Optimized Fuselage Weight Variables	57
3.15	Optimized Fuselage Geometric Variables	58
3.16	Optimized Fuselage Structural and Drag Variables	59
3.17	Performance Variable Comparison	60
3.18	Wing Variable Comparison	60
3.19	Vertical Tail Variable Comparison	61
3.20	Horizontal Tail Variable Comparison	61
3.21	Fuselage Variable Comparison	62

LIST OF FIGURES

1.1	Idealized design process	3
1.2	Block Gauss-Seidel MDA process to solve a three-discipline coupled system	11
1.3	Structure for the AAO problem	12
1.4	Diagram for the SAND architecture	14
2.1	Coupling of the Subsystems Model	19
2.2	Geometry of the Wing Model	26
2.3	Vertical Tail Geometry Model	31
2.4	Horizontal Tail Geometry Model	35
2.5	Fuselage Geometry Model	40

CHAPTER 1

INTRODUCTION

1.1 Background on Aircraft Design

Aircraft design is a complex design problem due to the requirement to balance aerodynamic performance, structural integrity, propulsion efficiency, and regulatory constraints. The process even extends beyond the technical performance and involves considerations that influence manufacturability and life-cycle costs [1]. Traditional design methods rely on iterative trial-and-error analysis through empirical relationships and handbook-based guidelines that lead to designs that may not be fully optimal. In many cases, not all disciplines have well-defined models that integrate seamlessly with conventional engineering frameworks [1]. Despite successful designs, it is often seen that there is a difficulty in utilizing the entire design space efficiently. Due to recent advancements in computational power and mathematical techniques, aircraft designs have shifted to frameworks that explore larger design spaces while meeting various constraints. These optimization techniques have emerged as a powerful alternative. By considering a wide range of constraints and design spaces, more efficient and innovative design solutions can be achieved. Mathematical programming provides a structured approach to optimizing aircraft configurations by defining objectives and implementing constraints. At its core, the aircraft design process can still be represented by a sequential flow, progressing from requirement definition through conceptual, preliminary, and detailed design stages that culminate in manufacturing and testing [1]. This framework provides structure while integrating methods aim to reduce the linearity of the process by incorporating feedback between stages [1].

Aircraft design has evolved remarkably, transitioning from the pursuit of powered flight to the development of aircraft optimized for specific mission objectives. Over time, changes to the design process have been driven by demands such as increasing range, speed, payload, or fuel efficiency. Traditional aircraft were designed using empirical and iterative trial and error methods. Engineers would make incremental changes to previous designs and observe the outcomes to revise or approve the new configurations. Based on the results, the design team would

determine whether to proceed with further development or not. Although this approach has produced many successful aircraft and remains commonly used, it demands considerable time and resources while restricting the exploration of innovative configurations.

Traditional aerospace engineering education emphasizes the use of empirical formulation and simplified assumptions to evaluate whether a design meets performance and feasibility criteria. By leveraging handbook methods, engineers can quickly assess the viability of a configuration and make informed modification based on observed trends from historical designs. This approach allows for rapid iteration, but it is inherently manual and limited.

Modern design integrates aerodynamics, structures, propulsion, control, and system engineering disciplines. These subsystems interact in complex ways where slight changes could have a massive impact on the overall design. Each design revision typically requires returning to earlier stages, applying incremental changes, and re-evaluating the outcome. This process can become tedious and time-consuming when the desired results cannot be easily achieved. With the advancement of computational tools and numerical methods, many of these iterative steps can now be automated and executed efficiently through optimization frameworks. This addition marks a fundamental shift in the design process, enabling the use of numerical analysis and optimization techniques to explore broader design spaces and systematically converge on high performance solutions.

1.2 Numerical Methods in Aerospace Design

Numerical optimization enables rapid, performance-driven modifications to an initial aircraft design, facilitating convergence toward an optimal design. However, the success of optimization frameworks depend on the underlying computational architecture, the fidelity of the performance models, and proper formulation of design constraints. Different designers tailor optimization approaches based on the specific performance metrics or subsystems of interest. As a result, many existing designs are shaped by historical assumptions or constrained by isolated subsystem objectives, which may directly affect system-level optimality [2]. Historically, limited computational resources hindered the adoption of advanced optimization techniques. Many

promising methods were simplified or underutilized due to the lack of sufficient computational power. The design space in aircraft optimization often involves dozens to hundreds of design variables and constraints, leading to high dimensional and nonlinear territory. This substantially increases computational costs and demands efficient algorithms and problem formulations to achieve convergence within a reasonable timeframe. With recent advancements in high performance computing and numerical solvers, many of these limitations have been mitigated. Modern optimization techniques are capable of handling complex, multidisciplinary aircraft design problems with greater accuracy and efficiency.

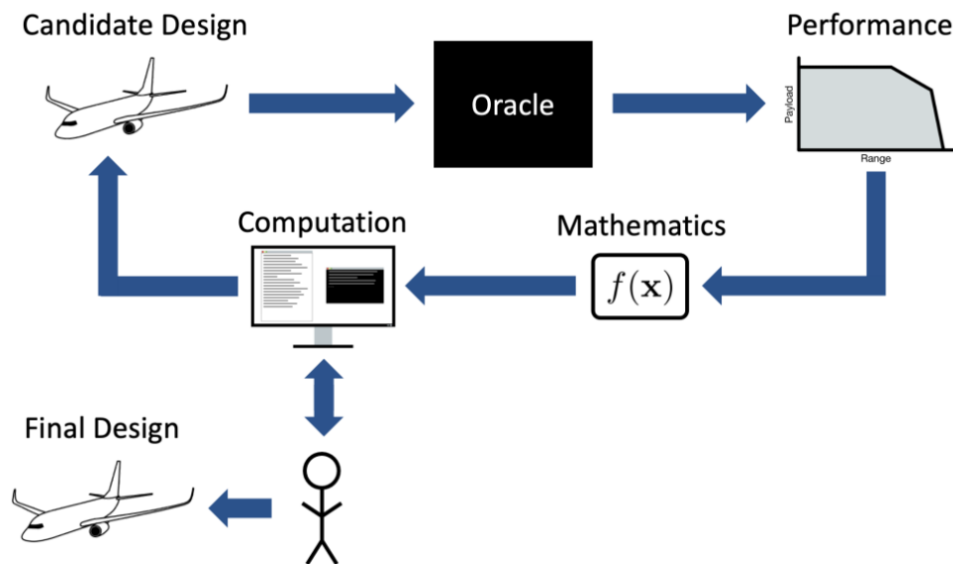


FIGURE 1.1. Idealized design process [2].

At its core, numerical optimization is the process of minimizing or maximizing an objective function subject to a set of constraints [2]. In the context of aircraft design, the objective functions reflect performance metrics, like total aircraft weight, fuel consumption, or range. Constraints serve to define the feasible design space and prevent the solution from diverging toward unrealistic or unbounded values. These constraints are typically categorized as equality or inequality constraints. The formulation of constraints is based on the available design variables, which can include wing area, aspect ratio, engine thrust, fuel mass, fuselage geometry, and material properties. For example, a typical aircraft design problem may involve minimizing fuel

consumption while ensuring that the wing generates adequate lift, the fuselage meets payload and structural requirements, and the propulsion system provides sufficient thrust for the intended mission profile. Although the true feasible region can be highly nonlinear and complex, optimization typically begins with initial estimates for these variables to guide the solver towards a viable solution. Proper problem formulation with realistic constraints and initial values is essential to achieving a successfully optimized solution.

Integrating numerical optimization into aircraft design introduces several significant challenges, starting with the complexity of high fidelity simulation modeling. As the demand for higher design accuracy increases, the dimensionality and complexity of the design space also expand accordingly. This requires more carefully defined constraints and simplifications of assumptions to manage the connection between subsystems. The integration of multidisciplinary constraints is particularly challenging due to the varying modeling approaches and the relationships among aircraft systems, such as aerodynamics, structures, propulsion, and control systems. Properly capturing these interactions is essential but leads to complex, nonlinear, and non-convex optimization problems. A common challenge in non-convex design spaces is that optimization algorithms often converge to a local optimum instead of the global one [2]. This behavior is highly sensitive to initial conditions, which affects both the convergence path and the computational cost. Therefore, careful algorithm selection, formulation, and strategies are necessary to reduce the risk of obtaining suboptimal solutions and excessive computational costs.

Different optimization approaches offer distinct advantages and limitations, depending on the specific application and constraints involved. Mathematical programming has emerged as a powerful tool for addressing challenges of aircraft design optimization. This approach allows designers to model variables, constants, objectives, and constraints within a structured optimization architecture problem.

A significant subset of mathematical programming, known as Geometric Programming (GP), has been effectively utilized to optimize a wide range of aircraft design problems. The strength of GP lies in its ability to represent objective and constrain functions as monomials and

posynomials, specific types of mathematical functions that enable efficient solution through convex optimization techniques. However, many aircraft design relationships do not naturally conform to the strict functional forms required by GP, which requires reformulation or approximations of these functions. Furthermore, GP imposes limitations that prevent it from capturing essential nonlinearities and negative terms present in typical design constraints. To overcome these limitations, Signomial Programming (SP) has been introduced as a generalization of GP, allowing for a broader class of functions and enabling more accurate modeling of complex aircraft design problems. SP extends this framework by relaxing some convexity constraints, allowing the use of signomials but at the cost of losing guaranteed global optimality. Because SP problems are generally non-convex, they require iterative solution methods that approximate a sequence of convex subproblems.

1.3 Geometric and Signomial Programming

The primary goal of this research is to develop a Python-based Signomial Programming framework for optimizing commercial aircraft design. Building upon the work of Kirschen et al. [3], this thesis demonstrates the capability of Signomial Programming to enhance design efficiency in the conceptual and preliminary design phase. The key optimization objective is to minimizing the aircraft's maximum takeoff weight (MTOW), thereby indirectly reducing fuel consumption, while satisfying a set of performance and operational constraints. To accomplish this, the framework models five key aircraft components: the wing, fuselage, horizontal tail, vertical tail, and landing gear. A convergent and feasible solution requires that these components models be closely coupled through shared variables and interdependent constraints. The formulation integrates aerodynamic performance, structural weight estimation, and mission requirements to ensure a physically consistent and practical design outcome. The coupling between components plays a vital role in the framework's success. For instance, the wing geometry directly influences aerodynamic performance and structural weight, which in turn affects fuel consumption and mission range.

Although this research focuses on the conceptual design stage and does not include high-

fidelity simulations, the optimization framework employs physics-based constraints derived from validated sources such as the TASOPT documentation [4], ensuring realism within the study's intended scope.

Future extensions of this work may incorporate higher-fidelity simulations, cost and environmental impact metrics, and models that account for uncertainty. Validation using existing aircraft data or alternative design tools will further establish the framework's credibility.

To understand the advantages and limitations of Signomial Programming, it is helpful to first explore its foundation: Geometric Programming. GP is a specialized form of mathematical optimization that imposes strict structural requirements on both, objective and constraint functions. Specifically, GP problems must be formulated using a combination of monomials and posynomials, which dictate the permissible mathematical operations and function forms. In GP, a monomial is defined as a function consisting of a positive coefficient multiplied by a product of decision variables raised to real number exponents. These exponents may be fractional or negative, but the overall coefficient must remain strictly positive.

$$m(x) = c \prod_{i=1}^N x_i^{a_i} \quad (1.1)$$

A posynomial is then formed by summing multiple monomial terms and are closed under addition, multiplication, and positive scaling.

$$p(x) = \sum_{k=1}^K c_k \sum_{i=1}^N x_i^{a_{ik}} \quad (1.2)$$

The standard structure of a geometric program includes a posynomial objective to be minimized, inequality constraints expressed as posynomials less than or equal to one, and equality constraints restricted to monomials set equal to one.

$$\text{minimize } p_o(x) \quad (1.3)$$

$$\text{subject to } m_i(x) = 1, i = 1, \dots, N \quad (1.4)$$

$$p_j(x) \leq 1, j = 1, \dots, M \quad (1.5)$$

These requirements ensure that the problem can be transformed into a convex form, enabling efficient and reliable solutions through convex optimization methods. While this structure contributes to the computational tractability of GP, it also introduces limitations. Many models do not conform to these forms, requiring reformulation or approximation in order to be used within a GP framework [5].

To solve a Geometric Program efficiently, the original formulation must first be converted into a convex optimization problem. This transformation enables the use of reliable convex solvers that guarantee convergence to a global optimum. The conversion process typically involves applying a logarithmic change of variables and taking the logarithm of the objective and constraint functions. Consequently, monomial equality constraints are rendered linear in the logarithmic space, and posynomial inequality constraints assume a convex form. After the transformation, the standard convex optimization form is achieved: a convex objective function, convex inequality constraints, and linear equality constraints. This reformulation can then be passed to a solver that handles convex programs. The transformation is essential because convex problems are not only easier to solve but also ensure that any local optimum is the global optimum. The solution process generally involves two stages: determining feasibility, whether there exists any point that satisfies all constraints, and then locating the optimal solution within the feasible region. If no feasible solution exists, the optimization problem is considered infeasible, and no valid optimum can be identified. Once a feasible and optimal solution is found, designers often perform trade-off analysis to examine how changes in constraints or objectives influence the outcome. This analysis typically involves relaxing or tightening constraints to evaluate the impact on performance metrics such as weight, drag, or fuel consumption. Closely related is sensitivity analysis which focuses on how small changes in parameters affect the solution. These analyses are particularly valuable near constraint boundaries, such as when a constraint becomes active at the optimum. In such cases, minor adjustments might not affect the solution significantly, while more substantial

changes can lead to entirely different optima. Moreover, when a problem is initially infeasible, sensitivity analysis can help guide adjustments to constraint values to restore feasibility [5].

An important extension of Geometric Programming is Signomial Programming, which expands the class of admissible functions by allowing negative coefficients within otherwise posynomial expressions. While a posynomial is defined as a sum of monomials with strictly positive coefficients, a signomial relaxes this condition and permits both, positive and negative coefficients, allowing more flexible modeling of real-world engineering relationships [5].

$$s(x) = \sum_{k=1}^K c_k \prod_{i=1}^N x_i^{a_{ik}} - \sum_{p=1}^P d_p \prod_{i=1}^N x_i^{g_{ip}} \quad (1.6)$$

Finding the global optimum of a SP is challenging due to the absence of convexity presence in geometric programs. However, underlying structure can still be exploited as a signomial program is simply the difference between two geometric programs [5]. Thus, SP problems are often solved using an iterative approach known as the Convex Concave Procedure often employed to solve Difference of Convex (DOC) programs. In this method, signomial constraints are approximated locally as posynomials by applying monomial approximations to non-convex terms and iterated to some relative convergence criteria.

$$\text{minimize} \quad \frac{p_o(x)}{n_o(x)} \quad (1.7)$$

$$\text{subject to} \quad s_i(x) \leq 0, i = 1, \dots, S \quad (1.8)$$

$$p_j(x) \leq 1, j = 1, \dots, P \quad (1.9)$$

$$m_k(x) = 1, k = 1, \dots, M \quad (1.10)$$

At each iteration, the signomial constraints are linearized around a current solution estimate, resulting in a efficient GP solution. The solution from one iteration is then used to update the approximation, gradually improving the estimate of the optimal point. This iterative refinement allows for the application of convex solvers to non-convex SP problems, provided that care

is taken in choosing initialization points and handling convergence criteria. While the final solution may only represent a local optimum, the method enables tractable exploration of complex design spaces that would otherwise be inaccessible using GP alone [5].

In modern aerospace engineering, optimization often relies on computational models treated as black-box simulations. While powerful, these models are frequently time consuming to evaluate and slow to converge to an optimal solution. In the context of aircraft design, where trade-offs between mission performance, structural integrity, weight, and fuel efficiency are central, the ability to explore the design space efficiently is essential. General purpose optimization methods often struggle with the complexity and coupling of aerospace subsystems. However, for more specialized and structured problems, techniques like GP offer a promising alternative. Despite its advantages, GP is not universally applicable. Many aircraft design relationships do not conform to GP's required functional forms, especially with discrete decisions or certain non-linear behaviors. The TASOPT model [4], which Hoburg and Abbeel's framework [6] partially builds upon, provides simplified yet physics-based relationships for estimating weight, drag, and performance metrics. These approximations can be structured as posynomials, making them GP-compatible while still grounded in modeling. This balance between fidelity and tractability is a key reason why GP has proven effective for conceptual and preliminary aircraft sizing. In some cases, engineers approximate these relationships using integer or binary variables to simulate behavior that cannot be directly modeled within the GP framework. While this workaround enables progress, it typically reduces the mathematical tractability and may prevent the solver from finding a truly optimal solution. One of GP's main strengths lies in its ability to unify multiple design disciplines, such as aerodynamics, structures, propulsion, and mission analysis, into a single coherent optimization problem. Furthermore, GP's structure naturally supports architecture-level decisions, such as determining wing aspect ratio, fuselage length, tail sizing, and cruise conditions. Because these parameters affect multiple subsystems simultaneously, traditional optimization techniques often struggle with system-level coupling. Hoburg and Abbeel [6] demonstrate that GP enables scalable multidisciplinary design optimization by linking these variables

through consistent physical and performance based constraints across the entire vehicle model. By leveraging its disciplined structure, GP avoids many of the organizational and communication bottlenecks often associated with multidisciplinary design. Moreover, adding or modifying components within the optimization allows engineers to explore new trade-offs with minimal reformation effort. One of the key insights from Hoburg and Abbeel’s work [6] is that the disciplined structure of GP allows for optimization runs that complete in seconds, rather than hours or days, due to its ability to generate globally optimal solutions for convex formulations. The fast turnaround is particularly advantageous in the initial stages of aircraft design, where engineers need to evaluate numerous configurations before selecting a feasible architecture. The efficiency, transparency, and robustness of GP-based solvers make them well-suited for the conceptual design phase, which requires quick iteration over high-level architectural decisions. Although its limitations require careful formulation and approximation, where the formulation remains a promising avenue for solving complex multidisciplinary problems in aerospace design, especially with rapid iteration and architectural exploration is required [6].

1.4 Multidisciplinary Design Optimization (MDO)

Multidisciplinary Design Optimization (MDO) is a framework for solving engineering design problems that span multiple interacting subsystems or disciplines, such as aerodynamics, structures, propulsion, and mission analysis. Rather than optimizing each subsystem independently, MDO seeks to account for the coupling and interdependencies between them in a unified optimization framework. A key challenge in MDO lies in capturing these interactions through mathematical models that can be embedded within a larger optimization structure [7]. This typically involves formulating constraints and objectives across subsystems, along with incorporating appropriate approximations to reduce computational expense. By structuring the MDO problem effectively and leveraging advanced numerical methods, engineers can achieve higher performance solutions while reducing overall development time and cost. This is made possible through the MDO architecture, which defines how models, approximations, and optimization routines are organized and coordinated within the design process.

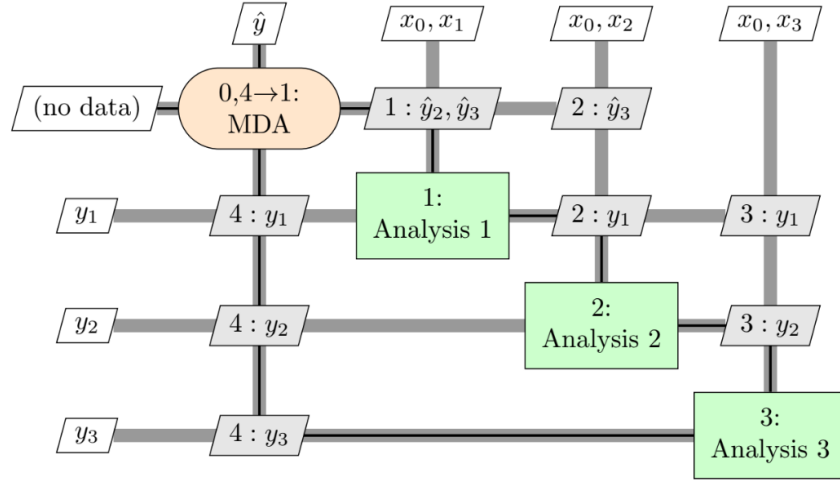


FIGURE 1.2. Block Gauss-Seidel MDA process to solve a three-discipline coupled system [7].

This architecture governs the communication between disciplinary analysis tools and the decomposition and solution of the optimization problem. Two common classifications are monolithic and distributed architectures. In a monolithic architecture, all subsystem models are combined into a single, large optimization problem that is solved simultaneously. This approach can provide more globally consistent results but may become computationally intractable for large scale or highly nonlinear problems. In contrast, distributed architecture partition the overall design into smaller, more manageable subproblems, each associated with a particular discipline or design function [7]. These subproblems can then be solved iteratively or in parallel, often improving modularity and scalability, especially for complex systems.

The selection of an appropriate MDO architecture plays a critical role in determining, both, the computational efficiency and the quality of the final design. Choosing the right approach depends on the structure of the design problem, the nature of the subsystem interactions, the available optimization algorithms, and computational resources. In their comprehensive review, Martins and Lambe [7] emphasize that many practical MDO problems can be formulated as single-objective problems with continuous design variables. In such formulations, the optimality of a solution is often defined in terms of the KKT conditions, which specify necessary conditions for local optimality in constrained nonlinear programs. Satisfying these conditions requires ac-

cess to gradient information for all constraint and objective functions. Therefore, architectures that support efficient gradient computation are generally preferred for solving large-scale MDO problems, either analytically or by automatic differentiation. A key challenge in MDO is managing the coupling between subsystems. When systems are strongly interdependent, it becomes difficult to isolate and solve them independently. Monolithic architectures address this by formulating a single optimization problem that encapsulates all coupling constraints and solves for multidisciplinary feasibility in one step. While this approach can produce consistent and potentially higher-quality solutions, it comes at the cost of increased problem size and complexity. Alternative strategies for achieving multidisciplinary feasibility involve distributed architectures and coordinated methods, which decompose the overall problem and iteratively enforce consistency between subsystems [7]. These methods allow more flexibility in solver choice and subsystem model reuse, but may converge slower or may require additional coordination algorithms to ensure convergence to a feasible design.

The All-at-Once (AAO) formulation represents one of the most general MDO architectures, where all design, state, and coupling variables are included directly in a single optimization problem [7].

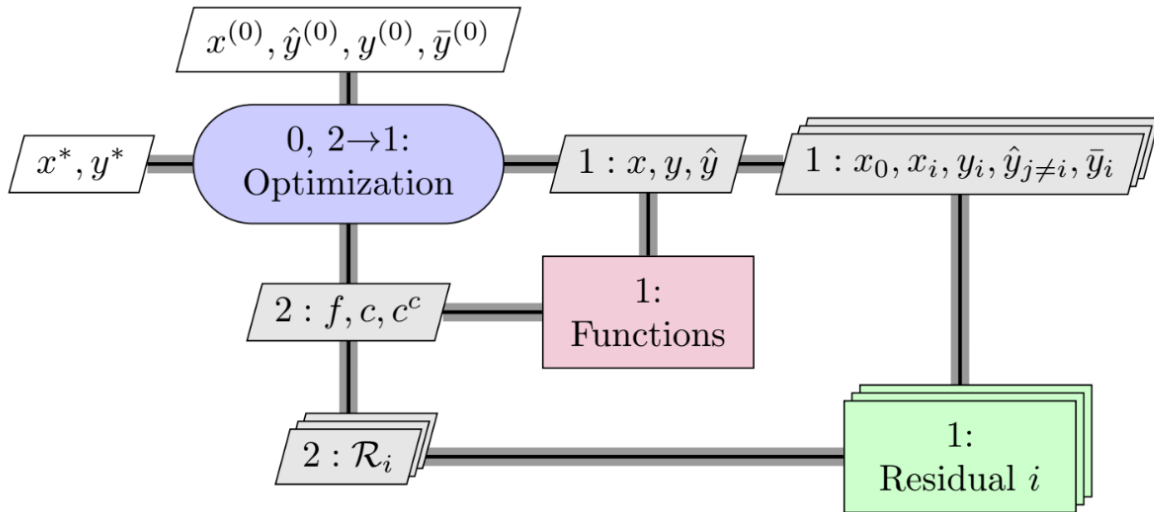


FIGURE 1.3. Structure for the AAO problem [7].

Although conceptually simple, this approach results in a large-scale problem that simulta-

neously enforces all disciplinary models and design constraints. The AAO framework provides a foundational basis from which other architectures can be derived by selectively relaxing or modifying constraints. Within the AAO family, the Simultaneous Analysis and Design (SAND) architecture is of particular relevance to this work. SAND modifies the standard AAO problem by introducing explicit coupling variables that serve to decouple individual discipline analyses. These variables are organized into target-response pairs, which allow the optimizer to manage consistency between subsystems without requiring discipline models to be solved at every iteration.

As a result, SAND allows the optimizer to explore both, feasible and infeasible regions of the design space, improving the ability to converge on a globally consistent solution [7]. In contrast to architectures that rely on external solvers or black-box evaluations of each discipline, the SAND formulation embeds the analysis models directly within the optimization problem. This approach eliminates the need for repeated discipline solutions at each step, making it well suited for problems where model gradients are available and can be efficiently evaluated. For present aircraft optimization framework, a SAND architecture is adopted to enable full integration of aerodynamic, structural, and mission constraints using an embedded signomial programming formulation. This structure supports both flexibility in constraint formulation and improved convergence behavior, as the optimizer is able to navigate complex, coupled design spaces without the overhead of repeatedly solving subsystems models.

It is also important to note that SAND architectures are not limited to multidisciplinary systems alone, they can also be applied to single-discipline optimization problems where embedding the analysis model directly into the optimization process is advantageous [7]. However, there are several challenges associated with this approach. First, SAND formulations require explicit inclusion of all state variables and discipline analysis equations within the optimization problem. This can significantly increase the problem size and lead to numerical instability or convergence difficulties, especially in large-scale systems. In some cases, this can result in premature termination or convergence to infeasible designs. Another challenge arises from the fact that discipline analysis models are treated as equality constraints in the optimization. This requires that

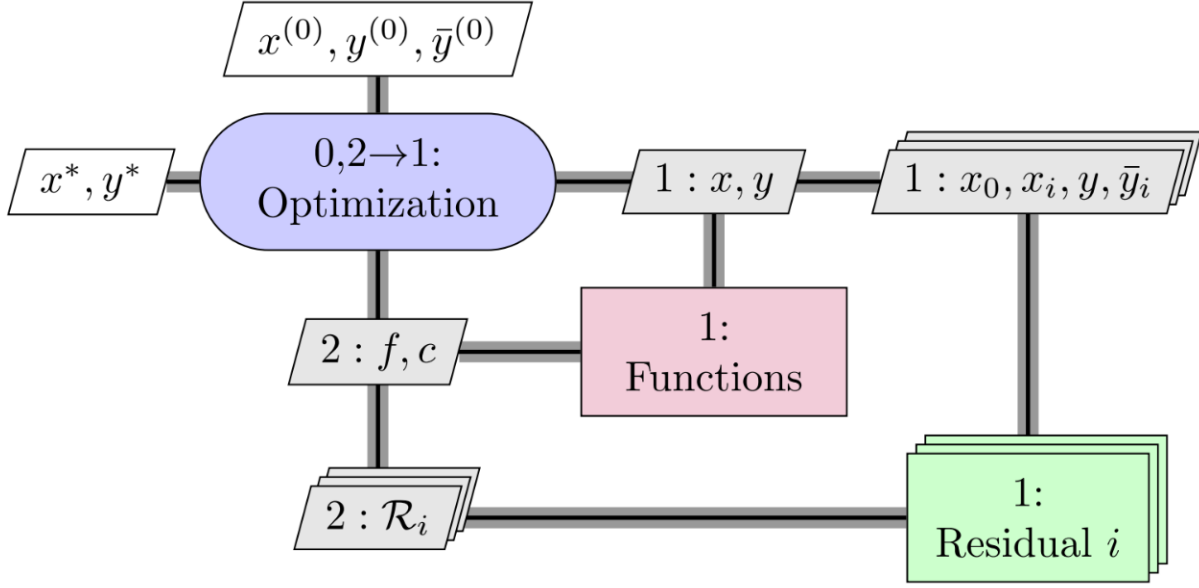


FIGURE 1.4. Diagram for the SAND architecture [7].

the residuals of the analysis equations be computable and passed directly to the optimizer, since it represents the difference between the predicted and actual system behavior. Instead of solving for coupling and state variables externally, the optimizer must rely on these residuals to enforce consistency between subsystems. While other MDO architectures may be more appropriate in different contexts, the SAND formulation aligns well with the structure of the aircraft design problem addressed in this thesis. Its ability to integrate subsystem models and constraints into the optimization enables a unified and flexible design environment that avoids reliance on black-box evaluations.

1.5 Pyomo-Based Implementation

While many optimization solvers exist across a range of platforms, a common challenge is integrating these solvers into problem-specific applications, especially when they are written in lower-level programming languages such as C or Fortran. These implementations, though efficient in terms of performance, often present difficulties when used within broader modeling workflows due to stricter syntax, lack of abstraction, or the need for external compilers. Pyomo is providing a high-level, object-oriented modeling language embedded in Python that addresses

many of these challenges [8, 9]. It enables users to define complex optimization problems in an intuitive and flexible syntax, while leveraging powerful solvers written in lower-level languages. This two-language architecture allows developers to combine ease of modeling with high computational performance, where model definition occurs in Python and solution routines execute in compiled solvers. A key advantage of Pyomo is its compatibility with a wide range of open-source and commercial solvers, and its ability to handle both, linear and nonlinear programming problems, including mixed integer formulations. Pyomo’s class structure supports modular model development, allowing for reusable components across subsystems or optimization scenarios. In this research, Pyomo is used as the core modeling framework for implementing the Signomial Programming formulation of the aircraft design problem. Its object-oriented structure and solver flexibility make it particularly well-suited for embedding coupled aerodynamic, structural, and performance constraints within a single optimization environment [8, 9].

Pyomo was developed at Sandia National Laboratories as part of a broader effort to explore and promote the adoption of algebraic modeling languages in real-world optimization applications. Rather than aiming to outperform existing commercial tools, the developers focused on creating an open-source modeling framework that emphasizes transparency, customizability, and extensibility. Pyomo’s open-source nature allows users to prototype new capabilities, modify internal functionality, and address bugs or limitations without relying on vendor timelines, which is a significant advantage over closed-source commercial platforms [8].

A key design goal of Pyomo was to leverage a modern, widely supported programming language. Python was chosen due to its large developer community, active online support, and extensive ecosystem of scientific libraries. One of Pyomo’s core design principles is the separation between model definition and solver execution. This abstraction allows users to modify formulations or switch between solvers without rewriting the model code, which is particularly valuable in research environments where rapid iteration and testing of optimization strategies are needed. Compared to legacy languages such as AMPL, or GAMS, Python offers increased accessibility for new users, more flexible licensing, and integration with contemporary numerical tools. Py-

omo is designed to be platform-independent, supporting operation on both Windows and Linux, and it allows users to define both concrete and abstract models. Pyomo operates under Python’s open-source license and is backed by a rich ecosystem of documentation, community-contributed packages, and numerical backends. It also enables users to define optimization problems in an abstract, symbolic form, which can then be translated into specific instances for solution by a variety of solvers. Pyomo benefits from tight integration with broader Python scientific computing stack, including libraries such as NumPy, SciPy, and Matplotlib, which enables efficient data manipulation, visualization, and post-processing of optimization results. In this research, Pyomo’s syntax enables clear representation of mathematical expressions and facilitates user interaction with optimization models, making it an ideal platform for building flexible and extensible design tools. While this thesis concentrates on deterministic, convex formulations for conceptual aircraft design, Pyomo additionally supports nonlinear, stochastic, and mixed-integer programming, offering a means to extend the optimization framework to more complex mission scenarios.

1.6 Research Contributions and Novelty

This thesis extends prior research in aircraft design optimization by combining established subsystem models with a new application of signomial programming [3]. The aerodynamic and structural formulations draw primarily from TASOPT documentation, but the optimization framework has been restructured to improve flexibility and numerical convergence.

While the overall methodology is informed by the framework described in Kirschen’s work [3], this work introduces several modifications, including adjustments to solver settings, variable coupling, and constraint management, that strengthen the robustness of signomial formulation. The initial implementation was carried out in GPkit, an open source toolkit for geometric programming. However, limitations in solver compatibility and extensibility motivated a transition to Pyomo, which provided greater control over the optimization process. This shift enabled the incorporation of nonlinear constraints and custom solver routines, both of which are essential for advanced signomial modeling. A key novelty of this research lies in this transition: by moving from GPkit to Pyomo, the framework evolves from a fixed geometric programming environment

into a more flexible and extensible platform, capable of handling nonlinear formulations, subsystem coupling, and solver customization not previously achievable in GPkit.

Within this environment, each aircraft subsystem is represented by its own set of performance relations and design constraints, all of which are integrated into a unified optimization framework in Python. Rather than proposing entirely new aerodynamic or structural models, the emphasis of this research is on demonstrating the efficiency, scalability, and practicality of a signomial-based optimization architecture for conceptual aircraft design. The central objective is to replicate the iterative design process, evaluate solver performance, and validate the framework against representative aircraft data and established benchmarks. In doing so, this thesis establishes a methodological foundation that not only advances optimization-based aircraft design but also sets the stage for the contributions detailed in the following chapters.

CHAPTER 2

METHODOLOGY

2.1 Performance Model

The aircraft design problem is decomposed into key subsystems representative of a conventional commercial aircraft configuration. Similar to the approach outlined by Kirschen et al. [3], the central objective is to minimize the maximum takeoff weight of the aircraft. In similar optimization frameworks, fuel weight has been used as a proxy for aircraft performance since it reflects aerodynamic quality, propulsion efficiency, and structural design trade-offs. By directly minimizing the maximum takeoff weight of the aircraft, the optimization captures these interactions more comprehensively across all subsystems. To achieve this, system-level performance constraints are applied to enforce physical feasibility and enable interdependence between subsystems. Each subsystem is governed by its own set of assumptions and constraints, such as structural loading for the wing, volumetric requirements for the fuselage, and thrust-to-weight limitations for the propulsion system. These variables and constraints can be found in the appendix of this paper. The overall system model incorporates an adaptation of the Breguet Range equation, which captures the coupled influence of aerodynamic performance, propulsion efficiency, and structural mass. This coupling ensures that changes in one subsystem propagate through the model and influence the global design outcome, enabling sensitivity analysis and performance driven trade-offs. Allowing the optimizer to evaluate feasible designs in a coupled, multidisciplinary design space.

Flight performance modeling begins with the segmentation of the mission profile to enable a more accurate representation of aircraft behavior across different phases of flight. Rather than applying the Breguet Range equation to the entire mission as a single block, the total range is divided into multiple segments. This segmentation allows for the tracking fuel burn, weight reduction, and performance changes more precisely over the course of the mission. Given that the primary objective is to minimize the maximum takeoff weight, constraints are applied on drag, lift, and fuel weight at each segment. These constraints capture how aircraft weight evolves as

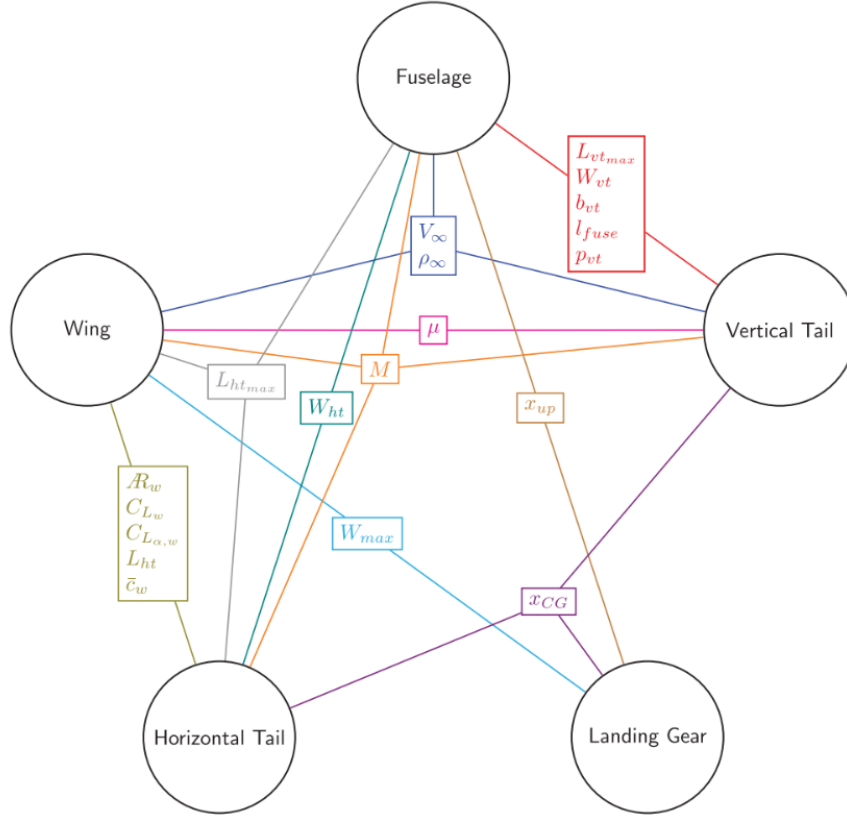


FIGURE 2.1. Coupling of the Subsystems Model [3].

fuel is consumed and how this affects aerodynamic efficiency and required thrust. To reflect the dynamic nature of fuel usage, proportional fuel fractions are assigned to each segment, and the distribution of weight throughout the airframe is carefully modeled. Additionally, the aircraft's moment of inertia is modeled as the sum of contributions from its major components. This constraint is critical for maintaining longitudinal and lateral stability and ensures that the design remains physically viable across the mission profile. To implement the segmented mission model into the code, these key equations are used to define fuel consumption, weight change, and aerodynamic relationships.

$$R = \frac{\eta}{\text{SFC}} \frac{L}{D} \left(\frac{W_{\text{initial}}}{W_{\text{final}}} \right) \quad (2.1)$$

The Breguet range relation is incorporated into the model by reformulating it as a set of constraints compatible with geometric programming. The cumulative range covered across all

cruise segments must meet the overall mission range requirement.

$$\sum_{n=1}^N R_n \geq R_{\text{req}} \quad (2.2)$$

Because geometric programming can make logarithmic functions challenging to handle in maintaining convexity, a direct solution is often impractical. In this work, a series expansion is applied to simplify the formulation and obtain a feasible solution [3]. Fuel weight is expressed as a series expansion of the exponential form derived from the Breguet Range equation. The rate of fuel consumption tends to have differential equations that are variable to thrust and drag. Rather than having a complex fuel model, we concluded that this first-order approximation works well for steady, level cruise segments of flight. The dimensionless variable z_{bre} relates to the key flight parameters that capture the effect of aerodynamic and propulsion performance on fuel usage during the segment. Time is a direct relationship of the ratio between Range and freestream velocity, providing a direct link between mission geometry and flight duration. The Lift-to-Drag ratio is computed using the average aircraft weight over drag during the given segment. This ratio is a key indicator of aerodynamic efficiency and directly influences fuel burn. The thrust constraint ensures that the total drag at each segment does not exceed the total available thrust from all operating engines. The final equation in this block models the variation of available thrust with altitude through a density ratio where the thrust is scaled relative to the takeoff thrust by the ambient density to the sea-level takeoff density. This simplification is chosen to capture the dominant effect of altitude on thrust without introducing the complexity of engine-specific performance curves. By assuming a linear relationship with air density, we maintain analytical tractability while still reflecting the key physical trend that thrust decreases with altitude.

$$W_{\text{fuel}_n} \geq \left(z_{\text{bre}_n} + \frac{z_{\text{bre}_n}^2}{2} + \frac{z_{\text{bre}_n}^3}{6} \right) W_{\text{end}_n} \quad (2.3)$$

$$z_{\text{bre}_n} \geq c_T t_n \frac{D_n}{W_{\text{avg}_n}} \quad (2.4)$$

$$t_n = \frac{R_n}{V_{\infty_n}} \quad (2.5)$$

$$\left(\frac{L}{D}\right)_n = \frac{W_{\text{avg}_n}}{D_n} \quad (2.6)$$

$$D_n \leq n_{\text{eng}} T_n \quad (2.7)$$

$$T_n = T_{\text{TO}} \left(\frac{\rho_{\infty}}{\rho_{\text{TO}}} \right) \quad (2.8)$$

Since the mission is assumed to begin at the start of the cruise segment, the aircraft's initial gross weight corresponds to the weight at the onset of cruise flight. Fuel consumption throughout the mission is modeled as a reduction in aircraft weight across successive segments, with each segment's weight difference representing the mass of fuel burned.

Average weight during each flight segment is a geometric mean of the segment's start and end weights, with adjustments for any buoyancy effects. This captures the gradual reduction in fuel mass while maintaining physical consistency within the optimization framework. The following equations describe the recursive weight relationships between segments. The aircraft begins the mission at its maximum takeoff weight, and for each segment, the starting weight equals the previous segment's ending weight. The total fuel consumed during a segment is modeled as the difference between starting and ending weights. The terminal weight constraint ensures that the final aircraft weight accounts for dry weight, payload weight, and total fuel loads that include primary and reserve fuel quantities. The final sum constraint of $W_{\text{fuel}_{\text{prim}}}$ establishes the relationship between segment level fuel burn and the total fuel carried.

$$W_{\text{avg}_n} \geq \sqrt{W_{\text{start}_n} W_{\text{end}_n}} + W_{\text{buoy}_n} \quad (2.9)$$

$$W_{\text{start}_0} = W_{\text{max}} \quad (2.10)$$

$$W_{\text{start}_n} \geq W_{\text{end}_n} + W_{\text{fuel}_n} \quad (2.11)$$

$$W_{\text{start}_{n+1}} = W_{\text{end}_n} \quad (2.12)$$

$$W_{\text{end}_n} \geq W_{\text{dry}} + W_{\text{pay}} + f_{\text{fuel}_{\text{res}}} W_{\text{fuel}_{\text{prim}}} \quad (2.13)$$

$$W_{\text{max}} \geq W_{\text{dry}} + W_{\text{pay}} + W_{\text{fuel}_{\text{prim}}} (1 + f_{\text{fuel}_{\text{res}}}) \quad (2.14)$$

$$W_{\text{fuel}_{\text{prim}}} \geq \sum_{n=1}^N W_{\text{fuel}_n} \quad (2.15)$$

The next block introduces the breakdown of the aircraft's dry weight and total drag into their respective component level contributions.

The total dry weight is the sum of individual component weights that include the wing, fuselage, vertical tail, horizontal tail, landing gear, and engine group. The miscellaneous term is included to account for secondary systems that contribute to the overall mass but are not captured by the aerodynamic components.

$$W_{\text{dry}} \geq W_{\text{wing}} + W_{\text{fuse}} + W_{\text{vt}} + W_{\text{ht}} + W_{\text{lg}} + n_{\text{eng}} W_{\text{eng}} + W_{\text{misc}} \quad (2.16)$$

The total drag during each mission segment is the sum of the drag contributions from the main aerodynamic components like the wing, fuselage, vertical tail, and horizontal tail. This formulation allows the optimization framework to capture how design modifications to individual components affect both overall drag and aircraft performance.

$$D_n \geq D_{\text{wing}_n} + D_{\text{fuse}_n} + D_{\text{vt}_n} + D_{\text{ht}_n} \quad (2.17)$$

This section defines the relationship between the Mach number and the freestream conditions during cruise. The Mach number is expressed as the ratio of the aircraft's freestream velocity to the local speed of sound. To ensure consistency with typical cruise conditions for modern narrowbody commercial aircraft, the Mach number is fixed at 0.78. By constraining the Mach number, the optimization can more accurately capture aerodynamic and propulsion characteristics

corresponding to realistic cruise conditions.

$$M = \frac{V_\infty}{a} \quad (2.18)$$

$$M = 0.78 \quad (2.19)$$

Finally, the takeoff model is implemented to ensure that the aircraft satisfies stall speed requirements during departure. In parallel, the atmospheric model imposes constraints on ambient pressure, density, temperature, and the speed of sound to ensure consistency with standard atmospheric conditions throughout the analysis.

$$x_{\text{TO}} \leq l_r \quad (2.20)$$

$$1 + y \leq 2 \frac{g x_{\text{TO}} T_{\text{TO}}}{V_{\text{TO}}^2 W_{\text{max}}} \quad (2.21)$$

$$1 \geq 0.0464 \frac{\zeta^{2.7}}{y^{2.9}} + 1.044 \frac{\zeta^{0.3}}{y^{0.049}} \quad (2.22)$$

$$\zeta \geq \frac{1}{2} \frac{\rho_{\text{TO}} V_{\text{TO}}^2 S_w C_D}{T_{\text{TO}}} \quad (2.23)$$

$$V_{\text{TO}} = 1.2 \sqrt{\frac{2W_{\text{max}}}{C_{L_{w,\text{max}}} S_w \rho_{\text{TO}}}} \quad (2.24)$$

The next component of the Performance model addresses the System-Level Properties. In this section, we ensure that the center of gravity equations are formulated to remain compatible with geometric programming. To simplify the analysis, it is assumed that both the fuselage and payload weights are uniformly distributed along the fuselage length, while the wing weight acts at its centroid location.

$$\begin{aligned} W_{\text{end}} x_{\text{CG}_n} \geq & W_{\text{wing}} (x_{\text{wing}} + \Delta x_{\text{ac}_w}) + W_{\text{fuel}_{\text{prim}}} (f_{\text{fuel}_n} + f_{\text{fuel}_{\text{res}}}) (x_{\text{wing}} + \Delta x_{\text{ac}_w} f_{\text{fuel}_n}) \\ & + \frac{1}{2} (W_{\text{fuse}} + W_{\text{pay}}) l_{\text{fuse}} + W_{\text{ht}} x_{\text{CG}_{\text{ht}}} + W_{\text{vt}} x_{\text{CG}_{\text{vt}}} + n_{\text{eng}} W_{\text{eng}} x_b + W_{\text{lg}} x_{\text{CG}_{\text{lg}}} + W_{\text{misc}} x_{\text{CG}_{\text{misc}}} \end{aligned} \quad (2.25)$$

The total moment of inertia of the aircraft is determined by summing the contributions from its primary components: the wing, fuselage and tail assemblies. The wing contribution includes the effects of the fuel system and engines, assuming a uniform mass distribution along the span. The fuselage component is estimated by distributing the fuselage and payload weights evenly along its length. Finally, the tail contribution is modeled as a point mass located at its respective moment arm from the aircraft's center of gravity.

$$f_{\text{fuel}_n} \geq \frac{\sum_{n=1}^n W_{\text{fuel}_n}}{W_{\text{fuel}_{\text{prim}}}} \quad (2.26)$$

$$I_z = I_{z_{\text{wing}}} + I_{z_{\text{fuse}}} + I_{z_{\text{tail}}} \quad (2.27)$$

$$I_{z_{\text{wing}}} = \frac{n_{\text{eng}} W_{\text{eng}} y_{\text{eng}}^2}{g} + \left(\frac{W_{\text{fuel}_{\text{wing}}} + W_{\text{wing}}}{g} \right) \frac{b_w^3 c_{\text{root}_w}}{16 S_w} \left(\lambda_w + \frac{1}{3} \right) \quad (2.28)$$

$$I_{z_{\text{fuse}}} = \left(\frac{W_{\text{fuse}} + W_{\text{pay}}}{g} \right) \left(\frac{x_{\text{wing}}^3 + l_{\text{vt}}^3}{3 l_{\text{fuse}}} \right) \quad (2.29)$$

$$I_{z_{\text{tail}}} = \left(\frac{W_{\text{apu}} + W_{\text{vt}} + W_{\text{ht}}}{g} \right) l_{\text{vt}}^2 \quad (2.30)$$

The landing gear model described in the original Kirschen formulation [3] was omitted in this formulation and replaced with a fixed weight fraction representative of typical narrow-body configurations. Instead of a fully parameterized model, a constant value was implemented based on empirical weight fraction data from existing literature to account for its contribution to the total takeoff weight.

This simplified approach preserves the overall mass balance of the aircraft while minimizing model complexity and reducing computational cost. The fixed parameters used in this work are defined as follows:

$$W_{\text{lg}} = 14696.92 \quad \text{N} \quad (2.31)$$

$$W_{\text{misc}} = 102340.356 \quad \text{N} \quad (2.32)$$

$$x_{\text{CG}_{\text{lg}}} = 19.3 \quad \text{m} \quad (2.33)$$

$$x_{CG_{misc}} = 10.0 \quad \text{m} \quad (2.34)$$

The landing gear supports the total aircraft weight during ground operations and enables steering and braking during taxi, takeoff, and landing phases. It is subject to some of the highest structural loads within the mission profile, particularly during touchdown and rotation. In addition to load bearing considerations, the landing gear system must accommodate geometric constraints related to wing placement, engine clearance, and retraction space, ensuring sufficient ground clearance throughout all operating conditions.

2.2 Wing Model

The first subsystem integrated into the optimization framework is the wing model. The goal of the wing is generate enough lift that allows for the aircraft to complete its mission of takeoff, cruise, and landing. The wing must also satisfy the structural capability of supporting the engines, and some configurations also allow for fuel to be stored within the wings. Due to this, weight and drag must be taken into consideration when designing. Similar to the Kirschen paper [3], the model assumes a continuous-taper, low-wing configuration with a modern transonic airfoil. This model begins with a set of geometric constraints that define the wing. Once the geometry is established, performance parameters such as lift, drag, and structural weight are constrained to ensure aerodynamic feasibility across the mission profile. An additional constraint is applied to estimate the aerodynamic center of the wing, which is critical for stability and control considerations. Consistent with the TASOPT-optimized 737-800 configuration [4] and the methodology presented by Kirschen [3], the model assumes the wing functions as a primary fuel storage component. Therefore, the final step in the wing modeling process involves constraining the maximum allowable fuel weight that can be stored within the wing structure, ensuring it meets both volume and loading limits.

The geometric relationships governing the wing model include expressions that link the aspect ratio, wing area, span, and mean geometric chord. To aid in converting these expressions into monomial or posynomial forms compatible with geometric programming, dummy variables

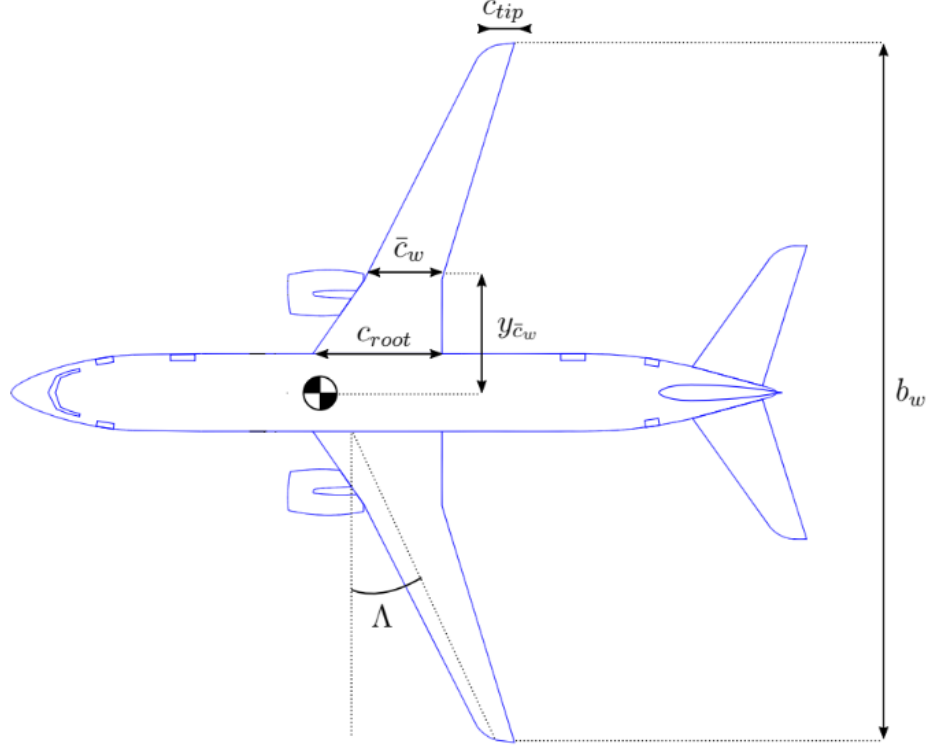


FIGURE 2.2. Geometry of the Wing Model [3].

derived from the structural model are introduced. The wing taper ratio is incorporated and given a lower bound to prevent excessively low Reynolds numbers at the wing tip, which would negatively affect aerodynamic performance. Finally, a maximum wingspan constraint is imposed to reflect airport gate size limitations.

The Oswald efficiency and function of taper ratio is also constrained here with the aspect ratio to achieve proper sizing requirements.

$$S_w = b_w \left(\frac{c_{root_w} + c_{tip_w}}{2} \right) \quad (2.35)$$

$$p_w = 1 + 2\lambda_w \quad (2.36)$$

$$q_w = 1 + \lambda_w \quad (2.37)$$

$$\bar{c}_w \geq \frac{2}{3} \left(\frac{1 + \lambda_w + \lambda_w^2}{q_w} \right) c_{root_w} \quad (2.38)$$

$$y\bar{c}_w = \frac{b_w q_w}{3p_w} \quad (2.39)$$

$$\lambda_w = \frac{c_{\text{tip}_w}}{c_{\text{root}_w}} \quad (2.40)$$

$$\lambda_w = \lambda_{w\min} \quad (2.41)$$

$$b_w \leq b_{w\max} \quad (2.42)$$

$$AR_w = \frac{b_w^2}{S_w} \quad (2.43)$$

$$e(1 + AR_w f(\lambda_w)) \leq 1 \quad (2.44)$$

$$f(\lambda_w) \geq 0.0524\lambda_w^4 - 0.15\lambda_w^3 + 0.1659\lambda_w^2 - 0.0706\lambda_w + 0.0119 \quad (2.45)$$

The total lift generated by the aircraft must exceed its weight to maintain flight. The lift produced by the wing is expressed through the standard lift equation, which defines the relationship between lift coefficient, dynamic pressure, and wing area. While aerodynamic losses, such as those caused by the fuselage and wingtip vortices exist in reality, they are not included in this formulation. For the purposes of our simulation, the standard lift equation provides an adequate representation of the wing's lifting performance. In this formulation, the lift coefficient is assumed rather than determined as a function of angle of attack, simplifying the analysis while maintaining reasonable accuracy for conceptual design.

$$L_{\text{total}} \geq W_{\text{avg}} + L_{\text{ht}} \quad (2.46)$$

$$L_{\text{total}} = f(L_{\text{total}_w})L_w \quad (2.47)$$

$$L_{\text{total}} = \frac{1}{2}\rho_{\infty}S_w C_{L_w} V_{\infty}^2 \quad (2.48)$$

The wing weight is comprised of various fractional components, including control surfaces such as flaps, ailerons, and ribs. The structural weight is modeled using a combination of

monomial and posynomial constraints, adapted from the framework introduced by Hoburg and Abbeel [6]. To estimate the root bending moments and loads, a simplified assumption is made, the lift distribution is considered uniform across the wing. This leads to a conservative estimate of structural loading and wing weight.

$$W_{\text{wing}} \geq W_{\text{struct}_w} (1 + f_{\text{flap}} + f_{\text{slat}} + f_{\text{aileron}} + f_{\text{lete}} + f_{\text{ribs}} + f_{\text{spoiler}} + f_{\text{watt}}) \quad (2.49)$$

$$W_{\text{struct}_w} \geq W_{\text{cap}} + W_{\text{web}} \quad (2.50)$$

$$W_{\text{cap}} \geq \frac{8\rho_{\text{cap}} g r_{w/c} t_{\text{cap}} S_w^{1.5} \nu}{3AR_w^{0.5}} \quad (2.51)$$

$$W_{\text{web}} \geq \frac{8\rho_{\text{web}} g r_h \tau_w t_{\text{web}} S_w^{1.5} \nu}{3AR_w^{0.5}} \quad (2.52)$$

$$\nu^{3.94} \geq 0.14 p_w^{0.56} + \frac{0.86}{p_w^{2.4}} \quad (2.53)$$

$$0.4232 \tau_w^2 t_{\text{cap}} r_{w/c} \geq 0.92 \tau_w t_{\text{cap}}^2 r_{w/c} + I_{\text{cap}} \quad (2.54)$$

$$\frac{AR_w M_r N_{\text{lift}} \tau_w q_w^2}{I_{\text{cap}} S_w \sigma_{\text{max}}} \leq 8 \quad (2.55)$$

$$\frac{AR_w L_{w_{\text{max}}} N_{\text{lift}} q_w^2}{S_w \sigma_{\text{max}_{\text{shear}}} \tau_w t_{\text{web}}} \leq 12 \quad (2.56)$$

$$M_r \geq \frac{AR_w L_{w_{\text{max}}} p_w}{24} \quad (2.57)$$

The wing drag model incorporates simulation data obtained from TASOPT [4] for C-Series airfoils and XFOIL analyses. The total wing drag is computed using the standard drag equation, while the overall drag coefficient is represented through a combination of component-based models that capture different sources of aerodynamic drag.

$$D_w = \frac{1}{2} \rho_{\infty} V_{\infty}^2 S_w C_{D_w} \quad (2.58)$$

$$C_{D_w} \geq C_{D_{p_w}} + C_{D_{t_w}} \quad (2.59)$$

$$\begin{aligned}
C_{D_{pw}}^{1.65} \geq & 1.61 \left(\frac{Re_w}{1000} \right)^{-0.55} \tau_w^{1.29} (M \cos(\Lambda))^{3.04} C_{L_w}^{1.78} \\
& + 0.0466 \left(\frac{Re_w}{1000} \right)^{-0.389} \tau_w^{0.784} (M \cos(\Lambda))^{-0.340} C_{L_w}^{0.951} \\
& + 191 \left(\frac{Re_w}{1000} \right)^{-0.219} \tau_w^{3.95} (M \cos(\Lambda))^{19.3} C_{L_w}^{1.15} \\
& + 2.82e - 12 \left(\frac{Re_w}{1000} \right)^{1.18} \tau_w^{-1.76} (M \cos(\Lambda))^{0.105} C_{L_w}^{-1.44} \quad (2.60)
\end{aligned}$$

To enhance the fidelity of the drag prediction, the inclusion of wave drag coefficient calculations would typically be necessary. However, due to the added complexity and time requirements associated with modeling transonic effects, the influence of wave drag is considered negligible for the purposes of this study.

$$Re_w = \frac{\rho_\infty V_\infty \bar{c}_w}{\mu} \quad (2.61)$$

$$C_{D_{tw}} \geq f_{tip} \frac{C_{L_w}^2}{\pi e A R_w} \quad (2.62)$$

The aerodynamic center is shifted as a result of wing sweep. Assuming a constant lift distribution per unit area across the wing, the location of the aerodynamic center can be computed analytically. This is achieved by expanding the relevant integrals, relaxing the exact equality to maintain GP compatibility, and applying appropriate substitutions. The resulting expression provides a tractable approximation of the aerodynamic center position within the optimization framework.

$$\Delta x_{ac_w} \geq \frac{1}{2} \tan \Lambda A R_w c_{root_w} \left(\frac{1}{3} + \frac{2}{3} \lambda_w \right) \quad (2.63)$$

The TASOPT optimized aircraft [4] indicates that the wing structure can accommodate internal fuel tanks. Given the previously established constraints on mean aerodynamic chord, airfoil thickness, and wingspan, the available fuel volume within the wing can be determined. These constraints are incorporated into the wing model to define the maximum feasible fuel storage ca-

capacity.

$$V_{\text{fuel}_{\text{max}}} \leq 0.303 \bar{c}_w^2 b_w \tau_w \quad (2.64)$$

$$W_{\text{fuel}_w} \leq \rho_{\text{fuel}} V_{\text{fuel}_{\text{max}}} g \quad (2.65)$$

$$W_{\text{fuel}_w} \geq \frac{f_{\text{fuel}_w} W_{\text{fuel}_{\text{total}}}}{f_{\text{fuel}_{\text{use}}}} \quad (2.66)$$

2.3 Vertical Tail Model

The vertical tail model plays a crucial role providing directional stability, particularly under critical flight conditions. In this framework, the size of the vertical tail is coupled with fuselage and engine parameters to ensure adequate side force generation during asymmetric thrust scenarios. The assumptions for this model are that the horizontal tail is mounted on the fuselage, so it is not required to design a reinforced vertical tail structure [3]. The modeling process begins with geometric and structural constraints, where certain formulations from the wing model are reused to maintain consistency across aerodynamic surfaces. From a performance standpoint, the vertical tail is sized to satisfy two critical flight conditions: an engine out condition and a crosswind scenario. The engine out condition is where yaw stability must be maintained in the presence of asymmetric thrust. The crosswind landing scenario is where the vertical tail must counter lateral wind effects on touchdown. Finally, the aerodynamic drag produced by the vertical tail is modeled and included as part of the total drag budget for the aircraft.

The geometry of the vertical tail begins by determining the moment arm, defined as the distance between the aircraft's center of gravity and the aerodynamic center of the vertical tail. This arm is constrained by the location of the tail's leading edge, the height of the mean aerodynamic chord above the fuselage, and the tail's sweep angle. The positions of the leading and trailing edges are linked through the chord length, with an additional constraint that prevents the tail root from extending forward of the fuselage. Using these relationships, the location of the vertical tail's center of gravity can be approximated. Structural sizing of the tail is based on the maximum

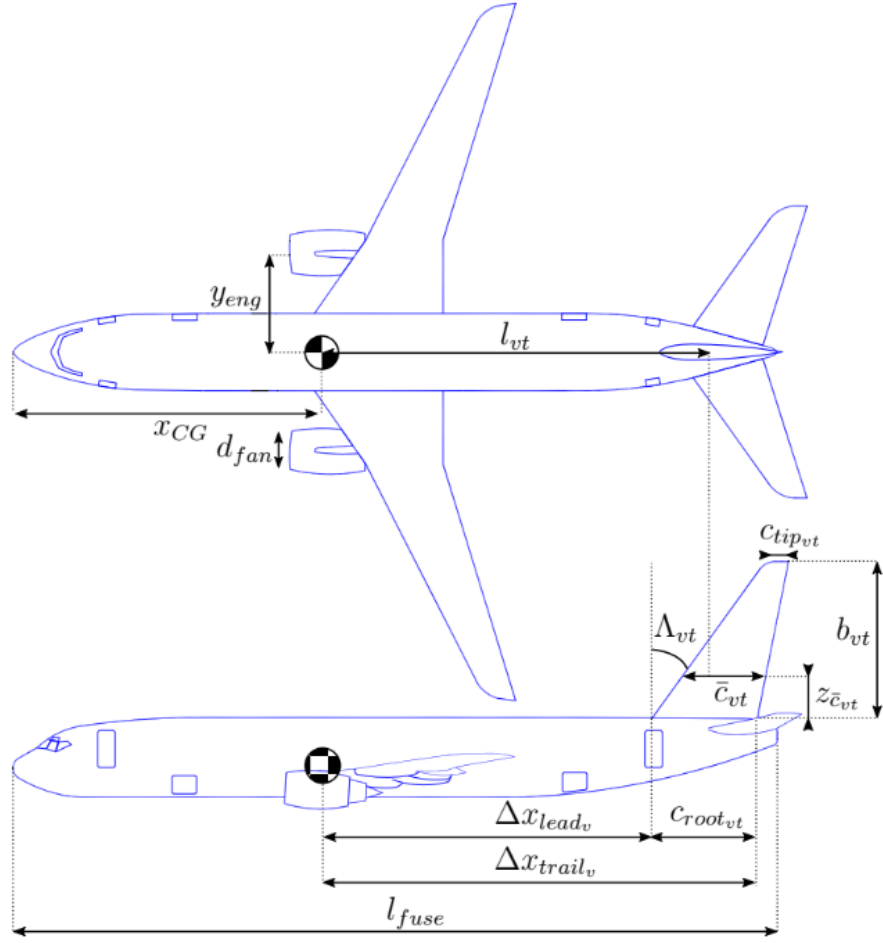


FIGURE 2.3. Vertical Tail Geometry Model [3].

lift coefficient and the never-exceed velocity.

$$l_{vt} \leq \Delta x_{lead_{vt}} + z_{\bar{c}_{vt}} \tan(\Lambda_{vt}) + 0.25 \bar{c}_{vt} \quad (2.67)$$

$$\Delta x_{trail_{vt}} \geq \Delta x_{lead_{vt}} + c_{root_{vt}} \quad (2.68)$$

$$l_{fuse} \geq x_{CG} + \Delta x_{trail_{vt}} \quad (2.69)$$

$$x_{CG_{vt}} \geq x_{CG} + \frac{1}{2}(\Delta x_{lead_{vt}} + \Delta x_{trail_{vt}}) \quad (2.70)$$

$$L_{vt_{max}} = \frac{1}{2} \rho_{TO} V_{ne}^2 S_{vt} C_{L_{vt_{max}}} \quad (2.71)$$

The geometric constraints applied to the tail are largely consistent with those used in the

wing model. The primary distinction lies in the inclusion of the tail volume coefficient relation, which defines the vertical tail area as a function of the wing geometry to satisfy stability and control considerations. The tail volume coefficient itself is selected based on reference aircraft with comparable geometric configurations [10, 11].

$$S_{vt} = \frac{V_{vt} S_w b_w}{l_{vt}} \quad (2.72)$$

$$S_{vt} \geq b_{vt} \left(\frac{c_{root_{vt}} + c_{tip_{vt}}}{2} \right) \quad (2.73)$$

$$\bar{c}_{vt} = \frac{2}{3} \left(\frac{1 + \lambda_{vt} + \lambda_{vt}^2}{q_{vt}} \right) c_{root_{vt}} \quad (2.74)$$

$$q_{vt} = 1 + \lambda_{vt} \quad (2.75)$$

$$p_{vt} = 1 + 2\lambda_{vt} \quad (2.76)$$

$$y \bar{c}_{vt} = \frac{b_{vt} q_{vt}}{3 p_{vt}} \quad (2.77)$$

$$\lambda_{vt} = \frac{c_{tip_{vt}}}{c_{root_{vt}}} \quad (2.78)$$

$$\lambda_{vt} \geq \lambda_{vt_{min}} \quad (2.79)$$

$$AR_{vt} = \frac{b_{vt}^2}{S_{vt}} \quad (2.80)$$

The vertical tail weight model is derived using the same general structure as the wing weight formulation, with variables modified to represent vertical tail geometry and loading conditions [6] [3]. Additional terms are included to account for component weight fractions specific to the vertical tail assembly, such as those associated with flaps, ribs, spoilers, etc.

$$W_{vt} \geq W_{struct_{vt}} (1 + f_{flap} + f_{lete} + f_{ribs} + f_{spoiler} + f_{watt}) \quad (2.81)$$

$$W_{struct_{vt}} \geq W_{cap_{vt}} + W_{web_{vt}} \quad (2.82)$$

$$W_{\text{cap}_{\text{vt}}} \geq \frac{8\rho_{\text{cap}} g r_{\text{vt}_{w/c}} t_{\text{cap}_{\text{vt}}} v_{\text{vt}} S_{\text{vt}}^{1.5}}{3AR_{\text{vt}}^{0.5}} \quad (2.83)$$

$$W_{\text{web}_{\text{vt}}} \geq \frac{8\rho_{\text{web}} g r_{\text{vt}_h} \tau_{\text{vt}} t_{\text{web}_{\text{vt}}} v_{\text{vt}} S_{\text{vt}}^{1.5}}{3AR_{\text{vt}}^{0.5}} \quad (2.84)$$

$$v_{\text{vt}}^{3.94} \geq 0.14 p_{\text{vt}}^{0.54} + \frac{0.84}{p_{\text{vt}}^{2.4}} \quad (2.85)$$

$$\frac{AR_{\text{vt}} M_{r_{\text{vt}}} N_{\text{lift}} \tau_{\text{vt}} q_{\text{vt}}^2}{I_{\text{cap}_{\text{vt}}} S_{\text{vt}} \sigma_{\text{max}}} \leq 8 \quad (2.86)$$

$$\frac{AR_{\text{vt}} L_{\text{vt}_{\text{max}}} N_{\text{lift}} q_{\text{vt}}^2}{S_{\text{vt}} \sigma_{\text{max}_{\text{shear}}} \tau_{\text{vt}} t_{\text{web}_{\text{vt}}}} \leq 12 \quad (2.87)$$

$$M_{r_{\text{vt}}} \geq \frac{AR_{\text{vt}} L_{\text{vt}_{\text{max}}} p_{\text{vt}}}{24} \quad (2.88)$$

The engine-out condition is critical for determining the maximum moment that the vertical tail must counteract when one engine becomes inoperative. In this scenario, the tail must generate enough force, at post-takeoff speeds, to produce a moment that balances or exceeds the moment induced by the windmilling drag of the failed engine. The operating speed is assumed to be the minimum safe speed following a critical engine failure. To model this, the wing's lift coefficient is constrained using finite wing theory, linking it to the sectional lift coefficient of the airfoil. The windmill drag is then estimated using a drag coefficient and the engine's reference area.

$$L_{\text{vt}_{\text{EO}}} l_{\text{vt}} \geq D_{\text{wm}} y_{\text{eng}} + T_{\text{TO}} y_{\text{eng}} \quad (2.89)$$

$$L_{\text{vt}_{\text{EO}}} = \frac{1}{2} \rho_{\text{TO}} V_1^2 S_{\text{vt}} C_{L_{\text{vt}_{\text{EO}}}} \quad (2.90)$$

$$c_{l_{\text{vt}_{\text{EO}}}} \geq C_{L_{\text{vt}_{\text{EO}}}} \left(1 + \frac{c_{l_{\text{vt}_{\text{EO}}}}}{\pi e_{\text{vt}} AR_{\text{vt}}} \right) \quad (2.91)$$

$$D_{\text{wm}} \geq \frac{1}{2} \rho_{\text{TO}} V_1^2 A_{\text{fan}} C_{D_{\text{wm}}} \quad (2.92)$$

The second critical constraint for vertical tail sizing is crosswind landing. In this case, the vertical tail must provide sufficient yaw rate acceleration to maintain directional control dur-

ing touchdown. To model this condition, the lift coefficient is conservatively set to 85% of the maximum achievable lift coefficient, accounting for reduced aerodynamic effectiveness under crosswind conditions.

$$\frac{\dot{r}_{\text{req}}}{I_z} \leq \frac{1}{2} \rho_{\text{TO}} V_{\text{land}}^2 S_{\text{vt}} l_{\text{vt}} C_{\text{vt,landing}} \quad (2.93)$$

Similar to the wing, the vertical tail contributes to the total aircraft drag. To estimate this component, aerodynamic data from XFOIL simulations of NACA airfoils used to construct a drag model specific to the vertical tail. This model captures the profile drag characteristics under varying operating conditions and geometric parameters.

$$D_{\text{vt}} \geq \frac{1}{2} \rho_{\infty} V_{\infty}^2 S_{\text{vt}} C_{D_{p_{\text{vt}}}} \quad (2.94)$$

$$\begin{aligned} C_{D_{p_{\text{vt}}}}^{1.189} \geq & 2.44 \times 10^{-77} (Re_{\text{vt}})^{-0.528} (\tau_{\text{vt}})^{133.8} (M)^{1022.7} \\ & + 0.003 (Re_{\text{vt}})^{-0.410} (\tau_{\text{vt}})^{1.22} (M)^{1.55} \\ & + 1.967 \times 10^{-4} (Re_{\text{vt}})^{0.214} (\tau_{\text{vt}})^{-0.04} (M)^{-0.14} \\ & + 6.590 \times 10^{-50} (Re_{\text{vt}})^{-0.498} (\tau_{\text{vt}})^{1.56} (M)^{-114.6} \end{aligned} \quad (2.95)$$

$$Re_{\text{vt}} = \frac{\rho_{\infty} V_{\infty} \bar{c}_{\text{vt}}}{\mu} \quad (2.96)$$

2.4 Horizontal Tail Model

The purpose of the horizontal tail model is to ensure steady, level flight while providing the pilot with sufficient pitch control authority throughout the mission. As mentioned in the previous model, the assumption for the horizontal tail is the mounting on the fuselage rather than the vertical tail. This allows for the production of downforce in cruise. The horizontal tail must be sized to satisfy the aircraft's trim condition, which requires the net pitching moment coefficient

of the full configuration to be zero. In addition to trimming the aircraft the horizontal tail must maintain a positive static margin, ensuring longitudinal stability under all expected loading conditions. This is enforced by a constraint that prevents the stability margin from falling below a prescribed minimum value. As with other aerodynamic surfaces, the horizontal tail contributes to the aircraft's parasitic and induced drag, which is accounted for through drag modeling constraints and included in the total drag budget. The area and moment arm of the horizontal tail are constrained to ensure aerodynamic effectiveness while remaining within realistic design bounds.

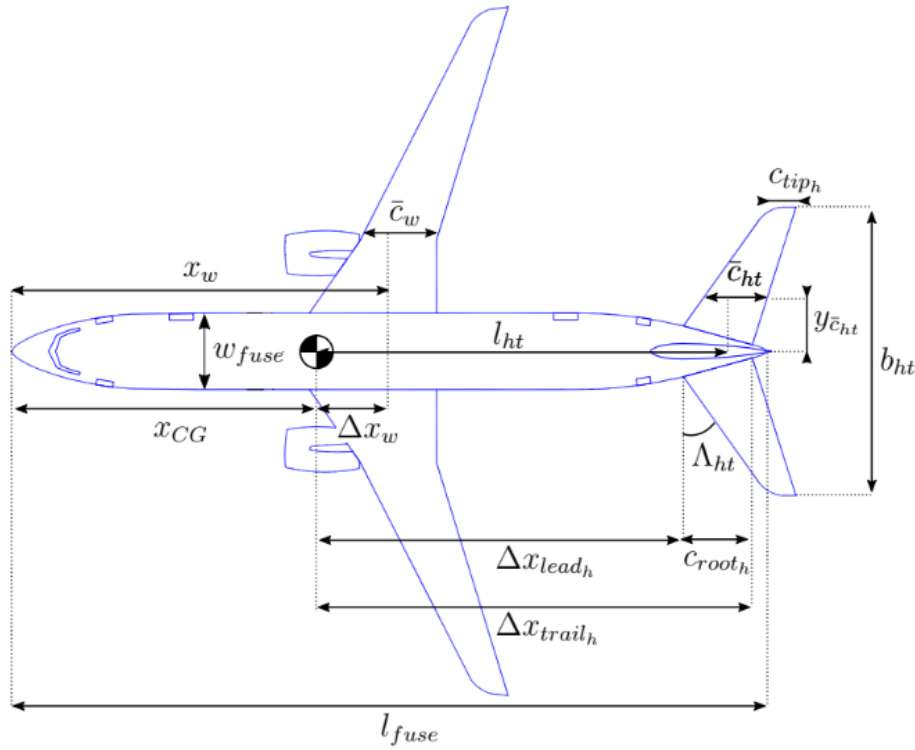


FIGURE 2.4. Horizontal Tail Geometry Model [3].

The horizontal tail geometry is governed by constraints similar to those applied in the wing and vertical tail models, with variable definitions adjusted to reflect the characteristics of the horizontal tail [3, 10, 11].

Like the vertical tail, it's sizing is based on a horizontal tail volume coefficient, which

relates the tail area to the wing's mean aerodynamic chord and the tail arm length.

$$S_{\text{ht}} = \frac{V_{\text{ht}} S_w \bar{c}_{\text{ht}}}{l_{\text{ht}}} \quad (2.97)$$

$$S_{\text{ht}} = b_{\text{ht}} \left(\frac{c_{\text{rootht}} + c_{\text{tipht}}}{2} \right) \quad (2.98)$$

$$\bar{c}_{\text{ht}} \geq \frac{2}{3} \left(\frac{1 + \lambda_{\text{ht}} + \lambda_{\text{ht}}^2}{q_{\text{ht}}} \right) c_{\text{rootht}} \quad (2.99)$$

$$y_{\bar{c}_{\text{ht}}} = \frac{b_{\text{ht}} q_{\text{ht}}}{3 p_{\text{ht}}} \quad (2.100)$$

$$q_{\text{ht}} = 1 + \lambda_{\text{ht}} \quad (2.101)$$

$$p_{\text{ht}} = 1 + 2\lambda_{\text{ht}} \quad (2.102)$$

$$\lambda_{\text{ht}} = \frac{c_{\text{tipht}}}{c_{\text{rootht}}} \quad (2.103)$$

$$\lambda_{\text{ht}} \geq \lambda_{\text{htmin}} \quad (2.104)$$

$$AR_{\text{ht}} = \frac{b_{\text{ht}}^2}{S_{\text{ht}}} \quad (2.105)$$

$$e(1 + AR_{\text{ht}} f(\lambda_{\text{ht}})) \leq 1 \quad (2.106)$$

$$f(\lambda_{\text{ht}}) \geq 0.0524\lambda_{\text{ht}}^4 - 0.15\lambda_{\text{ht}}^3 + 0.1659\lambda_{\text{ht}}^2 - 0.0706\lambda_{\text{ht}} + 0.0119 \quad (2.107)$$

The sizing requirement is necessary in order to properly refine the analysis of the aircraft's center of gravity, static stability, and control. To enhance the precision of the aerodynamic estimates, the model also incorporates the effects of taper ratio and Oswald efficiency.

The positioning of the horizontal tail follows a similar formulation to that of the vertical tail, with modifications made to account for the horizontal tail arm length relationship [10, 11].

$$l_{\text{ht}} \leq \Delta x_{\text{leadht}} + 0.25\bar{c}_{\text{ht}} \quad (2.108)$$

$$\Delta x_{\text{trail}_{\text{ht}}} \geq \Delta x_{\text{lead}_{\text{ht}}} + c_{\text{root}_{\text{ht}}} \quad (2.109)$$

$$l_{\text{fuse}} \geq x_{\text{CG}} + \Delta x_{\text{trail}_{\text{ht}}} \quad (2.110)$$

$$x_{\text{CG}_{\text{ht}}} \geq x_{\text{CG}} + \frac{1}{2}(\Delta x_{\text{lead}_{\text{ht}}} + \Delta x_{\text{trail}_{\text{ht}}}) \quad (2.111)$$

The lift generated by the horizontal tail is computed using the standard lift equation. To ensure adequate control authority, the lift and corresponding moment arm are constrained to counteract the pitching moment and drag forces produced by the engines.

$$L_{\text{ht}_{\text{max}}} = \frac{1}{2} \rho_{\text{TO}} S_{\text{ht}} c_{L_{\text{ht}_{\text{max}}}} V_{\text{ne}}^2 \quad (2.112)$$

$$L_{\text{ht}} l_{\text{ht}} \geq D_{\text{wm}} y_{\text{eng}} + T_{\text{TO}} y_{\text{eng}} \quad (2.113)$$

$$L_{\text{ht}} = \frac{1}{2} \rho_{\text{TO}} S_{\text{ht}} C_{L_{\text{ht}}} V_1^2 \quad (2.114)$$

The structural weight of the horizontal tail is modeled using the same framework as the wing weight formulation [6, 3], with parameter adjustments to reflect the geometry of the horizontal tail. Additionally, the weight equation accounts for fractional components specific to the tail structure.

$$W_{\text{ht}} \geq W_{\text{struct}_{\text{ht}}} (1 + f_{\text{flap}} + f_{\text{ribs}} + f_{\text{watt}}) \quad (2.115)$$

$$W_{\text{struct}_{\text{ht}}} \geq W_{\text{cap}_{\text{ht}}} + W_{\text{web}_{\text{ht}}} \quad (2.116)$$

$$W_{\text{cap}_{\text{ht}}} \geq \frac{8 \rho_{\text{cap}} g r_{\text{ht}_{\text{w/c}}} t_{\text{cap}_{\text{ht}}} v_{\text{ht}} S_{\text{ht}}^{1.5}}{3 A R_{\text{ht}}^{0.5}} \quad (2.117)$$

$$W_{\text{web}_{\text{ht}}} \geq \frac{8 \rho_{\text{web}} g r_{\text{ht}_{\text{h}}} \tau_{\text{ht}} t_{\text{web}_{\text{ht}}} v_{\text{ht}} S_{\text{ht}}^{1.5}}{3 A R_{\text{ht}}^{0.5}} \quad (2.118)$$

$$v_{\text{ht}}^{3.94} \geq 0.14 p_{\text{ht}}^{0.54} + \frac{0.84}{p_{\text{ht}}^{2.4}} \quad (2.119)$$

$$\frac{A R_{\text{ht}} M_{r_{\text{ht}}} N_{\text{lift}} \tau_{\text{ht}} q_{\text{ht}}^2}{I_{\text{cap}_{\text{ht}}} S_{\text{ht}} \sigma_{\text{max}}} \leq 8 \quad (2.120)$$

$$\frac{AR_{ht}L_{htmax}N_{lift}q_{ht}^2}{S_{ht}\sigma_{maxshear}\tau_{ht}t_{webht}} \leq 12 \quad (2.121)$$

$$M_{rht} \geq \frac{AR_{ht}L_{htmax}p_{ht}}{24} \quad (2.122)$$

The aircraft must satisfy the trim condition, which requires that the total moment coefficient about the center of gravity is zero. In typical analysis, thin airfoil theory is applied to estimate the lift-curve slope, which is then adjusted to account for downwash effects generated by the wing and fuselage. These corrections, often based on the downwash characteristics of an elliptically loaded wing, refine the effective lift-curve slope used in the trim condition. However, for the purposes of this study, these aerodynamic refinements are omitted to simplify the overall formulation. Instead, the trim condition is enforced directly through the balance of aerodynamic forces and moments without explicitly modeling lift-curve variations.

$$\frac{x_w}{\bar{c}_w} \leq \frac{x_{CG}}{\bar{c}_w} + \frac{C_{mac}}{C_{L_w}} + \frac{V_{ht}C_{L_{ht}}}{C_{L_w}} \quad (2.123)$$

Another key condition the horizontal tail must satisfy is maintaining a minimum static stability margin across both the forward and aft center of gravity limits. This ensures sufficient longitudinal stability throughout the flight envelope. The relationship between the lift characteristics of the horizontal tail and wing is captured using a signomial equality constraint, which accounts for the direction and magnitude of the aerodynamic forces, whether the tail is producing upward or downward lift.

$$SM_{min} + \frac{\Delta x_{CG}}{\bar{c}_w} + \frac{C_{mac}}{C_{L_{wmax}}} \leq V_{ht}m_{ratio} + \frac{V_{ht}C_{L_{htmax}}}{C_{L_{wmax}}} \quad (2.124)$$

$$1 + \frac{2}{AR_{ht}} = m_{ratio} \left(1 + \frac{2}{AR_w} \right) \quad (2.125)$$

To guarantee stable flight behavior, the static stability margin is constrained to remain above a defined minimum value for selected center of gravity locations. These conditions are im-

plemented through a set of stability constraints within the optimization framework.

$$SM \leq \frac{x_w - x_{CG}}{\bar{c}_w} \quad (2.126)$$

$$SM \geq SM_{\min} \quad (2.127)$$

The horizontal tail drag model follows a similar approach to the wing drag model utilizing XFOIL data from TASOPT T-series airfoils [4] designed for transonic conditions to refine the drag coefficient constraint equations.

$$D_{ht} \geq \frac{1}{2} \rho_{\infty} S_{ht} C_{D_{ht}} V_{\infty}^2 \quad (2.128)$$

$$C_{D_{ht}} \geq C_{D_{0_{ht}}} + C_{D_{i_{ht}}} \quad (2.129)$$

$$\begin{aligned} C_{D_{0_{ht}}}^{6.49} \geq & 5.288 \times 10^{-20} (Re_{ht})^{0.901} (\tau_{ht})^{0.912} (M)^{8.645} \\ & + 1.676 \times 10^{-28} (Re_{ht})^{0.351} (\tau_{ht})^{6.292} (M)^{10.256} \\ & + 7.098 \times 10^{-25} (Re_{ht})^{1.395} (\tau_{ht})^{1.962} (M)^{0.567} \\ & + 3.731 \times 10^{-14} (Re_{ht})^{-2.574} (\tau_{ht})^{3.128} (M)^{0.448} \\ & + 1.443 \times 10^{-12} (Re_{ht})^{3.910} (\tau_{ht})^{4.663} (M)^{7.689} \end{aligned} \quad (2.130)$$

$$Re_{ht} = \frac{\rho_{\infty} V_{\infty} \bar{c}_{ht}}{\mu} \quad (2.131)$$

$$C_{D_{i_{ht}}} \geq f_{tip} \frac{C_{L_{ht}}^2}{\pi e A R_{ht}} \quad (2.132)$$

2.5 Fuselage Model

The fuselage model is the largest and most structurally complex component in this formulation, due to its multifunctional role. It must serve as the central hub that connects all major

aircraft systems, including the wings, tail surfaces, and landing gear. For this analysis, the fuselage is modeled as a circular cross-section shell with uniform floor loading to represent the internal structural framework [3]. Additional assumptions include symmetric loading conditions and the use of a conservative yield stress to account for reinforcement materials. The model begins by defining the cross-sectional geometry, cylindrical shell, and tail cone. It must also satisfy structural constraints associated with internal pressurization loads, floor loading, and distributed aerodynamic forces. These are modeled using simplified stress constraints and empirical constraints. To maintain structural integrity, the fuselage design incorporates area moment of inertia constraints to ensure adequate stiffness under applied loads. Finally, the model includes estimates for fuselage weight and aerodynamic drag, which are incorporated into the overall aircraft performance and stability calculations.

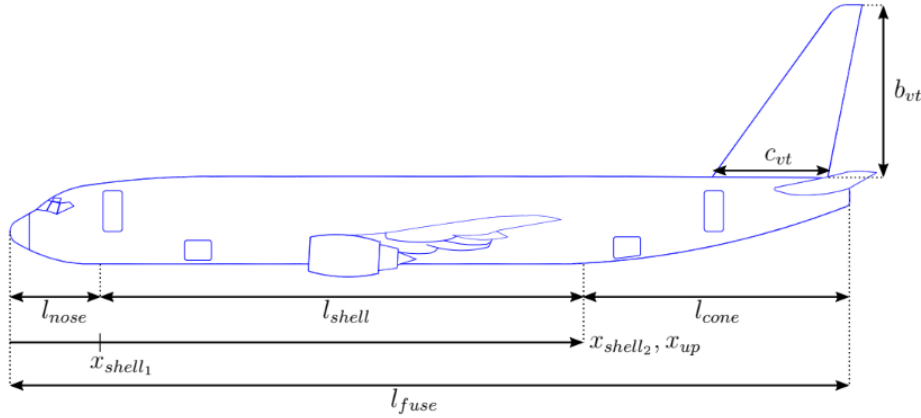


FIGURE 2.5. Fuselage Geometry Model [3].

The geometric constraints for the fuselage are based on the assumption of a circular cross-section. The fuselage diameter must be sufficient to accommodate the seat arrangement and aisle width. The structural constraint on the fuselage skin is derived from the thin-walled cylinder assumption, while the cross-sectional area is bounded as a function of the fuselage radius.

$$w_{\text{fuse}} = n_{\text{spr}} w_{\text{seat}} + w_{\text{aisle}} + 2w_{\text{sys}} \quad (2.133)$$

$$A_{\text{skin}} \geq 2\pi R_{\text{fuse}} t_{\text{skin}} \quad (2.134)$$

$$A_{\text{fuse}} = \pi R_{\text{fuse}}^2 \quad (2.135)$$

$$R_{\text{fuse}} \geq \frac{w_{\text{fuse}}}{2} \quad (2.136)$$

The axial and hoop stresses in the fuselage skin are constrained by internal pressurization loads, which result from the pressure differential between the cabin and the ambient atmosphere at cruise altitude. Based on these stresses, the required skin thickness is determined by applying the maximum allowable stress of the selected material.

$$\sigma_x = \frac{\Delta P_{\text{over}}}{2} \frac{R_{\text{fuse}}}{t_{\text{shell}}} \quad (2.137)$$

$$\sigma_\theta = \Delta P_{\text{over}} \frac{R_{\text{fuse}}}{t_{\text{skin}}} \quad (2.138)$$

$$\sigma_{\text{skin}} \geq \sigma_x \quad (2.139)$$

$$\sigma_{\text{skin}} \geq \sigma_\theta \quad (2.140)$$

The cabin floor structure must support the combined weight of the payload and seats, accounting for an appropriate landing load factor. The resulting shear force and bending moment are determined based on the design load distribution and floor width. The required cross-sectional area is then constrained by the allowable stresses in the tension caps and shear web.

$$P_{\text{floor}} = N_{\text{land}}(W_{\text{pay}} + W_{\text{seats}}) \quad (2.141)$$

$$S_{\text{floor}} = \frac{P_{\text{floor}}}{2} \quad (2.142)$$

$$M_{\text{floor}} = \frac{P_{\text{floor}} w_{\text{floor}}}{8} \quad (2.143)$$

$$A_{\text{floor}} = 1.5 \frac{S_{\text{floor}}}{\tau_{\text{floor}}} + 2 \frac{M_{\text{floor}}}{\sigma_{\text{floor}} h_{\text{floor}}} \quad (2.144)$$

This next section of constraints looks into the cylindrical shell of the fuselage that sit between the nose and tail cone. The number of seats is equal to the number of seats per row by the

number of rows. It is also assumed that there are the same number of passengers as there are seats. The seat pitch and the number of rows constrain the length of the shell. The floor length is constrained by the shell length and two times the radius in order to account for the pressure bulkheads. The length of the fuselage is then constrained by the sum of the nose, shell, and tail cone lengths. The wing midchord and the wingbox fore and aft bulkheads must be constrained in order to calculate bending loads. Cantrell's approximation is used to bound an approximation of the surface area of the nose [3]. This allows for the skin surface area, skin volume for the nose, main cabin, and the rear bulkhead to be constrained. The cabin volume is then constrained using a cylinder with hemispherical end caps for capturing buoyancy weight.

$$x_{\text{shell}_1} = l_{\text{nose}} \quad (2.145)$$

$$x_{\text{shell}_2} = l_{\text{nose}} + l_{\text{shell}} \quad (2.146)$$

$$n_{\text{seat}} = n_{\text{spr}} n_{\text{rows}} \quad (2.147)$$

$$n_{\text{pass}} = n_{\text{seat}} \quad (2.148)$$

$$l_{\text{shell}} \geq n_{\text{rows}} p_s \quad (2.149)$$

$$l_{\text{floor}} \geq 2R_{\text{fuse}} + l_{\text{shell}} \quad (2.150)$$

$$l_{\text{fuse}} = l_{\text{nose}} + l_{\text{shell}} + l_{\text{cone}} \quad (2.151)$$

$$S_{\text{nose}}^{8/5} = \left(2\pi R_{\text{fuse}}^2 \right)^{8/5} \left(\frac{1}{3} + \frac{2}{3} \left(\frac{l_{\text{nose}}}{R_{\text{fuse}}} \right)^{8/5} \right) \quad (2.152)$$

$$S_{\text{bulk}} = 2\pi R_{\text{fuse}}^2 \quad (2.153)$$

$$V_{\text{cyl}} = A_{\text{skin}} l_{\text{shell}} \quad (2.154)$$

$$V_{\text{nose}} = S_{\text{nose}} t_{\text{skin}} \quad (2.155)$$

$$V_{\text{bulk}} = S_{\text{bulk}} t_{\text{skin}} \quad (2.156)$$

$$V_{\text{cabin}} \geq A_{\text{fuse}} \left(\frac{2}{3} l_{\text{nose}} + l_{\text{shell}} + \frac{2}{3} R_{\text{fuse}} \right) \quad (2.157)$$

The tail cone transfers the aerodynamic and structural loads from the vertical tail to the main fuselage. The dominant load is the torsional moment induced by the vertical tail, which depends on the maximum aerodynamic force, tail span, and taper ratio. Assuming the cone cross-sectional area approximates that of the fuselage, and applying the allowable shear stress of the cone material, the required skin thickness can be determined. The cone skin volume is computed via integration from the base to the tip and incorporated into a signomial constraint. By aligning the formulation with the vertical tail model, the resulting constraints remain GP compatible [3]. Finally, the cone taper ratio imposes a constraint on cone length as a function of fuselage radius, driven by structural and geometric requirements.

$$Q_{\text{vt}} = \frac{L_{\text{vtmax}} b_{\text{vt}}}{3} \frac{1 + 2\lambda_{\text{vt}}}{1 + \lambda_{\text{vt}}} \quad (2.158)$$

$$t_{\text{cone}} = \frac{Q_{\text{vt}}}{2A_{\text{fuse}} \tau_{\text{cone}}} \quad (2.159)$$

$$R_{\text{fuse}} \tau_{\text{cone}} (1 + p_{\text{vt}}) V_{\text{cone}} \frac{1 + \lambda_{\text{cone}}}{4l_{\text{cone}}} \geq L_{\text{vtmax}} b_{\text{vt}} \frac{p_{\text{vt}}}{3} \quad (2.160)$$

$$\tau_{\text{cone}} = \sigma_{\text{skin}} \quad (2.161)$$

$$l_{\text{cone}} = \frac{R_{\text{fuse}}}{\lambda_{\text{cone}}} \quad (2.162)$$

The fuselage shell is composed of skin and stringers, and its ability to resist bending loads is governed by the area moment of inertia. Assuming uniform skin thickness and stringer spacing, the area moment of inertia remains constant about both the horizontal y-axis and vertical z-axis bending axes [3]. Horizontal bending moments are primarily induced by loads from the horizontal tail and the fuselage's inertial response, while vertical bending moments arise from vertical tail loads. The effective modulus weighted shell thickness accounts for both the skin and stringers, under the assumption that stringer weight scales proportionally with skin thickness. Pressurization introduces additional stress in the fuselage structure, effectively reducing the us-

able yield strength of the material in bending. As a result, the fuselage shell must satisfy both horizontal and vertical bending constraints, considering the pressurized condition and structural contribution of the skin-stringer combination.

$$t_{\text{shell}} = t_{\text{skin}} \left(1 + f_{\text{string}} r_E \frac{\rho_{\text{skin}}}{\rho_{\text{bend}}} \right) \quad (2.163)$$

$$\sigma_{\text{bend}} \geq \sigma_{M_h} + r_E \frac{\Delta P_{\text{over}} R_{\text{fuse}}}{2t_{\text{shell}}} \quad (2.164)$$

$$\sigma_{\text{bend}} \geq \sigma_{M_v} + r_E \frac{\Delta P_{\text{over}} R_{\text{fuse}}}{2t_{\text{shell}}} \quad (2.165)$$

$$I_{h_{\text{shell}}} \leq \pi R_{\text{fuse}}^3 t_{\text{shell}} \quad (2.166)$$

$$I_{v_{\text{shell}}} \leq \pi R_{\text{fuse}}^3 t_{\text{shell}} \quad (2.167)$$

The fuselage weight is constrained by the sum of all contributing components, excluding modeled weights. The fuselage skin weight is calculated as the product of skin volume and material density. The fuselage shell weight accounts for frames, stringers, and other structural elements. Floor weight includes the volume of floor beams multiplied by their material density, along with an assumed area based density for floor planking. Tail cone weight is estimated using assumed proportional weights for internal structures, such as frames and stringers. Window and insulation weights are based on assumed weight-per-length and weight-per-area values. Insulation is applied only to the passenger cabin, which is assumed to be approximately 55% of the total fuselage cross-sectional area. Additional weights from the auxiliary power unit and other payload-related components are captured using weight fractions. Miscellaneous systems are also included via weight fractions. The total seat weight is computed as the product of the number of seats and weight per seat. Buoyancy weight is calculated using the ideal gas law, specified cabin pressure, and an estimated cabin volume. This estimate assumes a sealed cabin and does not account for the difference between cabin and ambient air densities. Finally, fixed weight includes the pilots and required flight equipment, while payload weight is estimated based on an average

passenger weight including baggage.

$$\begin{aligned}
W_{\text{fuse}} &\geq W_{\text{apu}} + W_{\text{cone}} + W_{\text{floor}} \\
&\quad + W_{\text{insul}} + W_{\text{padd}} + W_{\text{seats}} \\
&\quad + W_{\text{shell}} + W_{\text{window}} + W_{\text{fix}}
\end{aligned} \tag{2.168}$$

$$W_{\text{skin}} \geq \rho_{\text{skin}} g (V_{\text{bulk}} + V_{\text{cyl}} + V_{\text{nose}}) \tag{2.169}$$

$$W_{\text{shell}} \geq W_{\text{skin}} (1 + f_{\text{fadd}} + f_{\text{frame}} + f_{\text{string}}) \tag{2.170}$$

$$V_{\text{floor}} \geq A_{\text{floor}} w_{\text{floor}} \tag{2.171}$$

$$W_{\text{floor}} \geq V_{\text{floor}} \rho_{\text{floor}} g + W_{\text{floor}}'' l_{\text{floor}} w_{\text{floor}} \tag{2.172}$$

$$W_{\text{cone}} \geq \rho_{\text{cone}} g V_{\text{cone}} (1 + f_{\text{fadd}} + f_{\text{frame}} + f_{\text{string}}) \tag{2.173}$$

$$W_{\text{window}} \geq W_{\text{window}}' l_{\text{shell}} \tag{2.174}$$

$$W_{\text{insul}} \geq W_{\text{insul}}'' (0.55(S_{\text{bulk}} + S_{\text{nose}}) + 1.1\pi R_{\text{fuse}} l_{\text{shell}}) \tag{2.175}$$

$$W_{\text{apu}} \geq W_{\text{pay}} f_{\text{apu}} \tag{2.176}$$

$$W_{\text{padd}} \geq W_{\text{pay}} f_{\text{padd}} \tag{2.177}$$

$$W_{\text{seats}} \geq W_{\text{seats}} n_{\text{seat}} \tag{2.178}$$

$$\rho_{\text{cabin}} = \frac{p_{\text{cabin}}}{RT_{\text{cabin}}} \tag{2.179}$$

$$W_{\text{buoy}} = \rho_{\text{cabin}} g V_{\text{cabin}} \tag{2.180}$$

$$W_{\text{pay}} \geq W_{\text{avg.pass}} n_{\text{pass}} \tag{2.181}$$

The fuselage drag is constrained using a drag coefficient calculated by TASOPT [4], which

employs a pseudo-axisymmetric viscous/inviscid method. This coefficient is then scaled by the fuselage reference area and dynamic pressure, which depends on the freestream Mach number and atmospheric conditions.

$$D_{\text{fuse}} = \frac{1}{2} \rho_{\infty} V_{\infty}^2 C_{D_{\infty}} \left(l_{\text{fuse}} R_{\text{fuse}} \frac{M^2}{M_{\text{fuse}_D}^2} \right) \quad (2.182)$$

The optimization results yield a set of design variables that define the aircraft's configuration at equilibrium between performance, weight, and aerodynamic constraints. The constants and variables are grouped according to their respective systems to better interpret their influence on the overall design.

CHAPTER 3

RESULTS

3.1 Performance Results

The results in this section demonstrate the performance of the signomial optimization framework applied to the baseline commercial aircraft configuration. The analysis focuses on the optimized geometric and performance parameters obtained from the coupled aerodynamic and structural formulation.

The results are organized by subsystem, with emphasis on the key drivers influencing aircraft weight, geometry, and aerodynamic performance. Each subsection also includes comparisons to values reported in prior studies to assess the validity of the proposed model.

To begin our model, we assume that we operate under certain circumstances that a typical commercial aircraft will operate in. It is important to note the weights and location of the Landing Gear and Miscellaneous systems. Due to those models being simplified and assumed constant, the value is directly taken from the optimized aircraft produced from the Kirschen [3] document in order to achieve reasonable designs. Upon integrating an updated landing gear model, the design parameters are subject to change and produce an entirely different optimized aircraft.

These parameters provide the foundation for all subsequent optimization cases and allow direct comparison with the results reported by Kirschen [3] and various outside sources [12, 13]. The constants shown in Table 3.1, ensure there is stability across the different configurations. Should we alter these constants, we produce a new optimal aircraft that will satisfy the change in requirements.

The optimized configuration closely matches the results in prior literature. There are minor discrepancies that can be attributed to the slight differences in constraint formulation and removal of certain subsystem models.

In Table 3.2, the performance variables are determined from the Breguet Range equation. The optimizer converged to an average drag coefficient of 0.13, which corresponds to a moderate cruise condition typical of a transonic transport aircraft. The resulting lift-to-drag ratio of approx-

TABLE 3.1. Constant Parameters and Reference Values used in Aircraft Model

Constant	Value	Units	Description
a	297	m/s	Speed of Sound at Cruise
e	0.9702	dimensionless	Oswald Efficiency Factor
$f_{\text{fuel}_{\text{res}}}$	0.1	dimensionless	Fuel Reserve Fraction
$f_{L_{\text{totw}}}$	1.2	dimensionless	Total Lift Divided by Wing Lift
M	0.78	dimensionless	Mach Number at Cruise
μ	1.4e-05	$\frac{Ns}{m^2}$	Dynamic Viscosity
ρ_{∞}	0.38	$\frac{kg}{m^3}$	Freestream Density
ρ_{fuel}	817	$\frac{kg}{m^3}$	Density of Fuel
ρ_{material}	2700	$\frac{kg}{m^3}$	Density of Spar and Web Material
V_{ne}	281.26	$\frac{m}{s}$	Never Exceed Velocity
W_{eng}	1e+04	N	Engine Weight
W_{lg}	14,696.92	N	Landing Gear Weight
W_{misc}	102,340.356	N	Miscellaneous Weight
$x_{\text{CG}_{\text{lg}}}$	19.3	m	Position of the Landing Gear
$x_{\text{CG}_{\text{misc}}}$	10	m	Position of Miscellaneous System

imately 11.56 indicates a balanced aerodynamic efficiency. Although the result is not quite the expected value, this could be due to the simplistic model for altitude thrust calculation without taking into account the throttle control. Additionally, the assumption of constant flight parameters may obscure dynamic effects that influence drag components. A more refined model incorporated throttle control, engine maps, and altitude dependent drag breakdown could yield a more representative lift-to-drag ratio and improve fidelity. With the Mach assumption of 0.78 and a speed of sound operating at an altitude approximately 40,000 ft, the freestream velocity of $237.6 \frac{m}{s}$ is key for solving the various lift and drag models through the various subsystems. With airport constraints, takeoff parameters will need to be taken into account, producing a takeoff distance and velocity of 2134.0 m and $74.93 \frac{m}{s}$, respectively.

The weight breakdown highlights the relationship between the total, dry, and fuel weights during a flight segment. The aircraft's average weight segment is gradually decreasing as fuel is consumed. The fuel fraction suggests a modest burn rate consistent with the aircraft's expected range per segment.

The total aircraft moment of inertia of $4.36e+07 \text{ kg } m^2$ takes into account where the fuse-

TABLE 3.2. Optimized Mission Variables

Variable	Value	Units	Description
C_D	0.13	dimensionless	Drag Coefficient (Avg)
D	7.994e+04	N	Total Aircraft Drag (Cruise Avg)
f_{fuel}	1.107	dimensionless	Percent Fuel Remaining each Segment (Avg)
I_z	4.364e+07	$kg\ m^2$	Total Aircraft Moment of Inertia
$I_{z_{\text{fuse}}}$	3.903e+07	$kg\ m^2$	Fuselage Moment of Inertia
$I_{z_{\text{tail}}}$	3.37e+06	$kg\ m^2$	Tail Moment of Inertia
$I_{z_{\text{wing}}}$	1.24e+06	$kg\ m^2$	Wing Moment of Inertia
$\frac{L}{D}$	11.56	dimensionless	Lift-to-Drag Ratio (Avg)
Range	9.647e+05	m	Segment Range
T	4.002e+04	N	Thrust per Engine in Cruise (Avg)
t	4.06e+03	s	Flight Time (per segment)
W_{avg}	9.24e+05	N	Average Aircraft Weight per Segment
W_{buoy}	2.688e+03	N	Buoyancy Weight
W_{dry}	7.241e+05	N	Aircraft Dry Weight
W_{end}	9.079e+05	N	Aircraft Weight at End-of-Flight Segment (Avg)
W_{fuel}	2.697e+04	N	Weight of Fuel burned per Flight Segment
W_{fuelprim}	8.091e+04	N	Primary Fuel Weight (excluding Reserves)
W_{max}	9.619e+05	N	Maximum Aircraft Weight
W_{start}	9.349e+05	N	Aircraft Weight at Start-of-Flight Segment (Avg)
x_{CG}	18.55	m	x Location of C.G.
x_{TO}	2.134e+03	m	Takeoff Distance
V_{TO}	74.93	$\frac{m}{s}$	Takeoff Velocity
V_{∞}	237.6	$\frac{m}{s}$	Freestream Velocity
y	6.771e-05	dimensionless	Takeoff Parameter
z_{bre}	0.02927	dimensionless	Breguet Parameter (Avg)
Δx_{CG}	0.05033	m	Change in x location of C.G.
ζ	9.946e-05	dimensionless	Takeoff Parameter

lage contributes the largest proportion. Due to the wing and tail's lower mass and distributed geometry, we expect those models to account for smaller moment of inertia fractions. This inertia distribution plays a key role in the aircraft's yaw and roll response.

Overall, the performance outputs demonstrate a coherent balance between aerodynamic efficiency, weight distribution, and dynamic properties. The resulting configuration achieves a reasonable range and fuel economy while maintaining feasible takeoff performance. These baseline results validate the optimization framework and provide a foundation for analyzing the following subsystem models.

3.2 Wing Model Results

The wing model defines the geometric, structural, and aerodynamic characteristics of the optimized configuration. The results presented in Tables 3.4-3.7, summarized the final parameters obtained from the optimization framework. They describe how the wing contributes to the overall aircraft performance and weight balance.

Starting with the constants, it is important to size the proper weight fractions of the various necessary pieces that are needed on the wing. It is also required to take into account the airport hanger allowable wingspan. Since the optimizer will optimize by minimizing drag which could lead to an infinite wingspan, therefore a proper constraint assist with leading the design to a feasible solution.

TABLE 3.3. Constants set in the Wing Model

Constant	Value	Units	Description
$b_{w_{\max}}$	35.9	m	Maximum Wing Span
f_{aileron}	0.005	dimensionless	Aileron Added Weight Fraction
f_{flap}	0.03	dimensionless	Flap Added Weight Fraction
$f_{\text{fuel}_{\text{use}}}$	0.9	dimensionless	Usability factor of Maximum Fuel Volume
f_{fuel_w}	0.05	dimensionless	Fraction of Total Fuel Stored in Wing
f_{lete}	0.05	dimensionless	Lete Added Weight Fraction
f_{ribs}	0.02	dimensionless	Wing Rib Added Weight Fraction
f_{slat}	0.02	dimensionless	Slat Added Weight Fraction
f_{spoiler}	0.01	dimensionless	Spoiler Added Weight Fraction
f_{tip}	0.065	dimensionless	Induced Drag Reduction from Wing-Tip Devices
f_{watt}	0.0001	dimensionless	Wing Attachment Hardware Added Weight Fraction
N_{lift}	2	dimensionless	Wing Loading Multiplier
r_h	0.75	dimensionless	Fractional Wing Thickness at Spar Web
$r_{w/c}$	0.5	dimensionless	Wing Box Width-to-Chord Ratio

The results are broken down into three main categories, Geometric Properties, Aerodynamic Characteristics, and Structural Characteristics.

The optimized wing geometry corresponds to an Aspect Ratio of 8.0 and a total area of 161.1 m^2 , which are consistent with medium range commercial aircraft configurations. The taper ratio of 0.2 and span of 35.9 m indicate a moderately tapered wing that balances drag reduction with structural simplicity. The aerodynamic center shift of 4.03 m captures the relative contribu-

TABLE 3.4. Optimized Wing Geometry Variables

Variable	Value	Units	Description
AR_w	8.0	dimensionless	Aspect Ratio of the Wing
b_w	35.9	m	Wingspan
$\bar{c}_w$	6.519	m	Wing Mean Aerodynamic Chord
c_{root_w}	7.479	m	Wing Root Chord
c_{tip_w}	1.496	m	Wing Tip Chord
f_{λ_w}	0.00356	dimensionless	Empirical Efficiency Function of Taper
p_w	1.4	dimensionless	Dummy Variable ($1 + 2 \lambda_w$)
q_w	0.9485	dimensionless	Dummy Variable ($1 + \lambda_w$)
S_w	161.1	m^2	Wing Area
Δx_{ac_w}	4.031	m	Wing Aerodynamic Center Shift
λ_w	0.2	dimensionless	Wing Taper Ratio

tion of the wing's geometry to longitudinal stability. The mean aerodynamic chord falls within typical range for wings of similar aspect ratio, suggesting that the optimizer successfully produced a feasible aerodynamic layout.

TABLE 3.5. Optimized Wing Lift Variables

Variable	Value	Units	Description
C_{L_w}	0.4704	dimensionless	Wing Lift Coefficient (Avg)
L_{total}	9.755e+05	N	Total Lift Generated by the Aircraft (Avg)
L_w	8.129e+05	N	Wing Lift (Avg)

The average wing lift coefficient is 0.4704, generating a wing lift of 8.129e+05 N, which represents approximately 83 % of the total aircraft lift in the segment. The remaining lift generation originates from the horizontal tail.

TABLE 3.6. Optimized Wing Drag Variables

Variable	Value	Units	Description
C_{D_w}	0.00623	dimensionless	Wing Drag Coefficient (Avg)
$C_{D_{i_w}}$	0.0005899	dimensionless	Wing Induced Drag Coefficient (Avg)
$C_{D_{p_w}}$	0.00564	dimensionless	Wing Parasitic Drag Coefficient (Avg)
D_w	1.077e+04	N	Wing Drag (Avg)
Re_w	4.204e+07	dimensionless	Reynolds Number of Wing

The total wing drag is 1.077e+04 N, that corresponds to an average drag coefficient of 0.00623. The drag coefficient is broken down into parasitic, 0.00564, and induced, 0.0005899,

drag coefficients that highlight the effects under cruise conditions with higher Reynolds numbers. This aligns with aerodynamic trends for high subsonic transports.

TABLE 3.7. Optimized Wing Weight Variables

Variable	Value	Units	Description
I_{cap}	9.082e-06	dimensionless	Nondimensional Spar Cap Area Moment of Inertia
M_r	1.452e+06	N	Root Moment per Root Chord
τ_w	0.14	dimensionless	Wing Thickness/Chord Ratio
t_{cap}	0.00227	dimensionless	Nondimensional Spar Cap Thickness
t_{web}	0.000991	dimensionless	Nondimensional Shear Web Thickness
W_{cap}	4.985e+04	N	Weight of Spar Cap
$W_{\text{fuel}_{\text{wing}}}$	1.016e+05	N	Maximum Fuel Weight Carried in the Wing
W_{struct_w}	5.442e+04	N	Weight Box Weight
W_{web}	4.571e+03	N	Weight of Shear Web
W_{wing}	6.178e+04	N	Wing Weight
$V_{\text{fuel}_{\text{max}}}$	28.64	m^3	Available Fuel Volume
ν	0.8602	dimensionless	Dummy Variable ($\frac{t^2+t+1}{(t+1)^2}$)

The structural model divides the wing into spar caps and shear webs to estimate the internal load bearing weight. The total wing weight is 6.178e+04 N, which includes Spar Cap Weight of 4.985e+04 N and Shear Web Weight of 4.571e+03 N. The nondimensional spar cap and shear web thicknesses indicate a structurally efficient design that minimizes bending loads while maintaining stiffness.

The root bending moment of 1.452e+06 N represents the dominant structural driver for sizing internal components. The available fuel volume of 28.64 m^3 allows for a maximum fuel weight of approximately 1.016e+05 N, showing that the wing tanks provide sufficient storage to meet the mission fuel requirements.

The resulting wing characteristics fall within the range of modern mid-range transport aircraft such as the Airbus A320 or Boeing 737 families, which both exhibit aspect ratios between 8-10 and comparable geometric proportions. The aerodynamic and structural outputs demonstrate that the optimizer captures realistic trade-offs between lift generation, drag minimization, and structural efficiency. These values serve as the foundation for evaluating the interactions between the wing and other major components, which are discussed in the following sections.

3.3 Vertical Tail Model Results

The vertical tail plays a key role in providing directional stability and yaw control for the aircraft. The optimization framework outputs both geometric and aerodynamic parameters for the vertical tail, summarized in Tables 3.8-3.10. These results allow assessments of the vertical tail's effectiveness during normal asymmetric flight conditions, such as engine out scenarios.

TABLE 3.8. Optimized Vertical Tail Geometric Variables

Variable	Value	Units	Description
AR_{vt}	0.4309	dimensionless	Aspect Ratio of Vertical Tail
b_{vt}	2.092	m	Wing Span of Vertical Tail
$\bar{c}_{vt}$	3.564	m	Vertical Tail Mean Aerochord
$c_{root_{vt}}$	5.0	m	Vertical Tail Root Chord
$c_{tip_{vt}}$	1.5	m	Vertical Tail Tip Chord
l_{vt}	37.0	m	Vertical Tail Moment Arm
p_{vt}	1.6	dimensionless	Dummy Variable ($1 + 2\lambda_{vt}$)
q_{vt}	1.3	dimensionless	Dummy Variable ($1 + \lambda_{vt}$)
S_{vt}	10.16	m^2	Vertical Tail Area
$x_{CG_{vt}}$	30.68	m	x Location of Vertical Tail C.G.
$\Delta x_{lead_{vt}}$	9.631	m	Distance from C.G. to VT Leading Edge
$\Delta x_{trail_{vt}}$	14.63	m	Distance from C.G. to VT Trailing Edge
λ_{vt}	0.3	dimensionless	VT Taper Ratio

The optimized vertical tail geometry results in an aspect ratio of 0.43 and a total area of $10.16 m^2$. The relatively low aspect ratio and compact planform are typical for conventional transport aircraft, where structural and stability requirements prioritize stiffness and surface effectiveness over aerodynamic efficiency. The span of 2.09 m and mean aerodynamic chord of 3.564 m yield a planform that is tall and moderately tapered. This geometry provides sufficient surface area for yaw stability while limiting torsional loads on the fuselage. The tail moment arm of 37.0 m indicates that the vertical tail is positioned far aft of the center of gravity of the vertical tail, 30.68 m. Offering a significant lever arm for restoring yaw moments. The aerodynamic center locations relative to the aircraft center of gravity demonstrate adequate placement for directional stability without excessive pitching interaction.

The structural sizing of the vertical tail is governed by the root bending moment of $3.677e+04$

TABLE 3.9. Optimized Vertical Tail Weight Variables

Variable	Value	Units	Description
$I_{\text{cap}_{\text{vt}}}$	3.689e-07	dimensionless	Nondimensional VT Spar Cap Area Moment of Inertia
$M_{r_{\text{vt}}}$	3.677e+04	N	VT Root Moment Per Root Chord
τ_{vt}	0.14	dimensionless	VT Thickness/Chord Ratio
$t_{\text{cap}_{\text{vt}}}$	0.0001393	dimensionless	Nondimensional Spar Cap Thickness
$t_{\text{web}_{\text{vt}}}$	0.000654	dimensionless	Nondimensional Shear Web Thickness
$W_{\text{cap}_{\text{vt}}}$	127.0	N	Weight of VT Spar Caps
$W_{\text{struct}_{\text{vt}}}$	322.7	N	Vertical Tail Box Weight
$W_{\text{web}_{\text{vt}}}$	195.7	N	Weight of VT Shear Webs
W_{vt}	358.2	N	Weight of the Vertical Tail
ν_{vt}	0.8176	dimensionless	Dummy Variable ($\frac{t^2+t+1}{(t+1)^2}$)

N and the internal spar cap and web thicknesses. The total vertical tail weight of 358.2 N, which includes contributions from spar caps, shear webs, and structural box components. The low overall weight reflects the tail's small surface area and modest loading requirements compared to the wing and horizontal tail. The nondimensional spar cap and web parameters demonstrate that the optimization framework produced a lightweight yet sufficiently stiff design. These values are consistent with typical vertical tail weight fractions less than 2% of the aircraft's total structural weight for similar configurations.

TABLE 3.10. Optimized Vertical Tail Aerodynamic Variables

Variable	Value	Units	Description
$C_{D_{p_{\text{vt}}}}$	0.0177	dimensionless	Viscous Drag Coefficient
$C_{L_{\text{vtEO}}}$	0.5677	dimensionless	VT Lift Coefficient during Engine Out
$C_{L_{\text{vtland}}}$	9.049e-12	dimensionless	VT Lift Coefficient during Landing
D_{wm}	3.617e+03	N	Engine Out Windmill Drag
D_{vt}	1.929e+03	N	Vertical Tail Drag
L_{vtEO}	1.731e+04	N	Vertical Tail Lift in Engine Out Condition
Re_{vt}	2.299e+07	dimensionless	Reynolds Number VT

The aerodynamic behavior of the vertical tail was analyzed under both nominal and engine out flight conditions. During engine-out operation, the vertical tail generates a lift coefficient of 0.568, producing Lift of approximately 1.731e+04 N of side force to counteract asymmetric thrust. This level of authority indicates sufficient control margin to maintain yaw equilibrium. In contrast, the lift coefficient during landing is effectively negligible, as expected since direc-

tional control in that regime is primarily managed through rudder deflection and nosewheel steering. The total drag attributed to the vertical tail is $1.929\text{e}+03$ N, dominated by parasitic effects. The induced drag contribution is minimal, reflecting the low lift loading of the vertical surface in steady flight. Additionally, the engine out windmill drag is captured in the model to represent asymmetric conditions that significantly influence tail sizing.

The resulting vertical tail design exhibits realistic proportions and aerodynamic performance when compared to reference data from mid-sized commercial aircraft. The tail area, aspect ratio, and structural weight fall within expected bounds, demonstrating that the optimization framework effectively balances directional stability, control authority, and structural efficiency. These outputs validate that the model's signomial formulation can capture the nonlinear dependencies between tail geometry, aerodynamic forces, and structural weight while maintaining a feasible solution consistent with established aircraft design practices.

3.4 Horizontal Tail Model Results

The horizontal tail model defines the aerodynamic and structural characteristics necessary to ensure longitudinal stability and control. Its design directly influences the trim balance, static margin, and pitching moment characteristics of the aircraft. Though the optimizer, key geometric, aerodynamic, and structural parameters were determined to meet stability and performance requirements while minimizing additional drag and weight penalties.

The horizontal tail configuration yielded an aspect ratio of 5.61 with a total area of 18.87 m^2 . Compared to the vertical tail, this design exhibits a higher aspect ratio and larger planform area, which are expected characteristics for providing longitudinal stability and trim control. The geometric parameters with a span of 10.29 m and a mean aerodynamic chord of 1.822 m is consistent with conventional tail plane configuration.

Aerodynamically, the horizontal tail produces a lift of $5.148\text{e}+04$ N with a Drag of $1.226\text{e}+03$ N. This is a result from a drag coefficient of 0.006058 comprised of a parasitic and induced drag model. The low drag indicates efficient flow, while the small stability margin, 0.1576, reflects a fine tuned trim condition rather than a conservative stability design. The Reynolds number of

TABLE 3.11. Optimized Horizontal Tail Geometric Variables

Variable	Value	Units	Description
AR_{ht}	5.611	dimensionless	Aspect Ratio of the Horizontal Tail
b_{ht}	10.29	m	Horizontal Tail Wing span
$\bar{c}_{ht}$	1.822	m	HT Mean Aerodynamic Chord
$c_{root_{ht}}$	2.645	m	HT Root Chord
$c_{tip_{ht}}$	0.5289	m	HT Tip Chord
l_{ht}	12.44	m	HT Moment Arm
p_{ht}	1.4	dimensionless	Dummy Variable ($1 + 2*\lambda_{ht}$)
q_{ht}	1.2	dimensionless	Dummy Variable ($1 + \lambda_{ht}$)
S_{ht}	18.87	m^2	HT Wing Area
$x_{CG_{ht}}$	31.86	m	x Location of the HT C.G.
$\Delta x_{lead_{ht}}$	11.99	m	Distance from C.G. to HT Leading Edge
$\Delta x_{trail_{ht}}$	14.63	m	Distance from C.G. to HT Trailing Edge

TABLE 3.12. Optimized Horizontal Tail Aerodynamic Variables

Variable	Value	Units	Description
$C_{D_{ht}}$	0.006058	dimensionless	Horizontal Tail Drag Coefficient (Avg)
$C_{D_{0_{ht}}}$	0.005812	dimensionless	Horizontal Tail Parasitic Drag Coefficient
$C_{D_{i_{ht}}}$	0.0002458	dimensionless	Horizontal Tail Induced Drag Coefficient (Avg)
$C_{L_{ht}}$	0.2543	dimensionless	Horizontal Tail Lift Coefficient
D_{ht}	1.226e+03	N	Horizontal Tail Drag
L_{ht}	5.148e+04	N	Horizontal Tail Lift
Re_{ht}	1.175e+07	dimensionless	Reynolds Number HT
SM	0.1576	dimensionless	Stability Margin

1.175e+07 further confirms turbulent flow conditions at the operating point.

TABLE 3.13. Optimized Horizontal Tail Weight Variables

Variable	Value	Units	Description
$I_{cap_{ht}}$	4.484e-05	dimensionless	Nondimensional HT Spar Cap Area Moment of Inertia
$M_{r_{ht}}$	7.482e+05	N	Horizontal Tail Root Moment per Root Chord
τ_{ht}	0.14	dimensionless	Horizontal Tail Thickness/Chord Ratio
$t_{cap_{ht}}$	0.0193	dimensionless	Nondimensional Spar Cap Thickness
$t_{web_{ht}}$	0.006977	dimensionless	Nondimensional Shear Web Thickness
$W_{cap_{ht}}$	1.616e+04	N	Weight of Horizontal Tail Spar Caps
W_{ht}	1.858e+04	N	Weight of the Horizontal Tail
$W_{struct_{ht}}$	1.769e+04	N	Horizontal Tail Box Weight
$W_{web_{ht}}$	1.534e+03	N	Weight of the Horizontal Tail Shear Web
v_{ht}	0.8562	dimensionless	Dummy Variable ($\frac{t^2+t+1}{(t+1)^2}$)

From a structural standpoint, the total weight of the horizontal tail was found to be 1.858e+04

N, distributed between the spar caps, shear web, and box components. The root moment per chord indicates that the tail is subject to substantial bending loads, necessitating the observed spar caps and shear web thickness values.

Overall, the horizontal tail configuration exhibits a well balanced design that satisfies the aircraft's longitudinal stability and trim conditions. The optimized geometry achieves sufficient lift generation with minimal drag contribution, supported by a structurally efficient layout capable of sustaining the expected aerodynamic loads. These results confirm that the horizontal tail contributes effectively to overall aircraft balance without compromising aerodynamic efficiency or structural feasibility.

3.5 Fuselage Model Results

With the lifting surfaces established, the next step examines the fuselage parameters predicted by the optimizer. The fuselage model defines the primary structural and aerodynamic characteristics of the aircraft's main body, encompassing the payload, cabin, and essential systems. This component significantly contributes to both the overall aerodynamic drag and structural weight. The optimizer evaluates key parameters while incorporating secondary structures like floor beams, bulkheads, and tail cone. The resulting configuration balances passenger accommodation and internal system integration with structural efficiency and aerodynamic smoothness.

TABLE 3.14. Optimized Fuselage Weight Variables

Variable	Value	Units	Description
W_{apu}	5.208e+03	N	APU Weight
W_{cone}	9.211e+03	N	Cone Weight
W_{floor}	9.678e+03	N	Floor Weight
W_{fuse}	5.063e+05	N	Fuselage Weight
W_{insul}	4.305e+03	N	Insulation Weight
W_{padd}	5.952e+04	N	Miscellaneous Weights (galley, toilets, doors, etc)
W_{pay}	1.488e+05	N	Payload Weight
W_{seats}	3.311e+04	N	Seating Weight
W_{shell}	3.717e+04	N	Shell Weight
W_{skin}	2.065e+05	N	Skin Weight
W_{window}	1.061e+04	N	Window Weight

The total fuselage weight is estimated at approximately $5.063\text{e}+05$ N, which dominates the aircraft's overall structural weight. This total accounts for the contributions of several key components like the payload, APU, cone, floor, etc. The skin and shell weights reflect the structural integrity required to handle cabin pressurization and bending loads, with the inclusion of insulation, flooring, and seating, which provide a realistic distribution of secondary system masses.

TABLE 3.15. Optimized Fuselage Geometric Variables

Variable	Value	Units	Description
A_{floor}	0.05326	m^2	Floor Beam Cross-Sectional Area
A_{fuse}	10.81	m^2	Fuselage Cross-Sectional Area
A_{skin}	0.2554	m^2	Skin Cross-Sectional Area
l_{cone}	18.55	m	Cone Length
l_{floor}	28.11	m	Floor Length
l_{shell}	24.4	m	Shell Length
n_{pass}	186	dimensionless	Number of Passengers
n_{rows}	31	dimensionless	Number of Rows
S_{bulk}	21.62	m^2	Bulkhead Surface Area
S_{nose}	49.82	m^2	Nose Surface Area
t_{cone}	0.0004915	m	Cone Thickness
t_{shell}	0.02958	m	Shell Thickness
t_{skin}	0.02191	m	Skin Thickness
V_{bulk}	0.4737	m^3	Bulkhead Skin Volume
V_{cone}	0.1932	m^3	Cone Volume
V_{cyl}	6.23	m^3	Cylinder Skin Volume
V_{cabin}	314.6	m^3	Cabin Volume
V_{floor}	0.1664	m^3	Floor Volume
w_{fuse}	3.71	m	Fuselage Width
x_{shell_1}	5.2	m	Start of Cylinder Section
x_{shell_2}	29.6	m	End of Cylinder Section

Geometrically, the fuselage exhibits a cylindrical section length of 24.4 m with a total width of 3.71 m and a cross-sectional area of 10.81 m^2 . These values are representative of a medium-capacity commercial transport. The cabin volume of 314.6 m^3 accommodates 186 passenger across 31 seating rows, suggesting a single aisle configuration. The floor beam cross-sectional area and skin thickness were sized to meet bending and hoop stress requirements associated with pressurization and flight loads. Additionally, the cone and shell volumes indicate a structurally efficient tapering design toward the aft section.

TABLE 3.16. Optimized Fuselage Structural and Drag Variables

Variable	Value	Units	Description
D_{fuse}	6.602e+04	N	Fuselage Drag
$I_{h_{\text{shell}}}$	3.088e-27	m^4	Shell Horizontal Bending Inertia
$I_{v_{\text{shell}}}$	3.088e-27	m^4	Shell Vertical Bending Inertia
M_{floor}	4.263e+05	$N * m$	Maximum Bending Moment in Floor Beams
P_{floor}	1.091e+06	N	Distributed Floor Load
S_{floor}	5.457e+05	N	Maximum Shear in Floor Beams
ρ_{cabin}	0.8711	$\frac{kg}{m^3}$	Cabin Air Density
σ_{M_h}	3.088e-05	$\frac{N}{m^2}$	Horizontal Bending Material Stress
σ_{M_v}	3.088e-05	$\frac{N}{m^2}$	Vertical Bending Material Stress
σ_x	1.85e+06	$\frac{N}{m^2}$	Axial Stress in Skin
σ_{θ}	4.995e+06	$\frac{N}{m^2}$	Skin Hoop Stress

From a structural standpoint, the maximum bending moment in the floor beams and the distributed floor load remain within typical bounds for aluminum alloy transport aircraft. The axial and hoop stresses in the fuselage skin show that the skin primarily resists internal pressure loads with a significant circumferential stress component. The relatively small horizontal and vertical bending stresses indicate that the overall structure provides ample rigidity against flight induced bending.

Aerodynamically, the fuselage contributes a total drag of 6.602e+04 N, which is consistent with the expected parasitic drag for a blended body transport aircraft at cruise conditions. This value confirms that the fuselage's surface finish and geometry produce a streamlined profile while maintaining sufficient internal volume for passengers and systems.

The optimized fuselage model achieves a structurally sound configuration that supports the expected payload and internal pressurization loads while maintaining aerodynamic efficiency. The calculated stresses and bending moments remain within allowable material limits, and the resulting drag contribution aligns with typical values for transport class aircraft. Overall, the fuselage design demonstrates an effective trade-off between, strength, weight and aerodynamic performance, ensuring compatibility with the major aircraft subsystems.

3.6 Aircraft Model Comparison

To assess the accuracy and credibility of the proposed optimization framework, the resulting aircraft parameters are compared against those reported in literature and publicly available data for existing commercial aircraft. In particular, comparisons are made with the benchmark configuration presented by Kirschen [3] as well as the Boeing 737 family [14, 12, 13], which serves as a representative narrow body transport aircraft of similar mission class.

TABLE 3.17. Performance Variable Comparison

Performance Variable	Solution Value	Kirschen	Reference
W_{dry} (N)	7.241e+05	4.038e+05	6.1529e+05 [14]
D (N)	7.994e+04	3.421e+04	N/A
$\frac{L}{D}$	11.56	19.7	N/A

The optimized model produced a dry weight of approximately 7.241e+05 N, which is higher than both the Kirschen reference [3] and the available Boeing 737-800 data [14, 12, 13]. This discrepancy likely stems from conservative structural weight assumptions and the lack of detailed subsystem modeling. The resulted drag force exceeds that of the Kirschen model, which could be attributed to the differences in aerodynamic efficiency. The computed lift-to-drag ratio is lower than the reference, suggesting that further refinement to the thrust and drag parameters may be necessary for alignment with real world data.

TABLE 3.18. Wing Variable Comparison

Wing Variable	Solution Value	Kirschen	Reference
AR_w	8.0	10.2	9.45 [14]
b_w (m)	35.9	35.8	34.32 [13]
S_w (m^2)	161.1	126.0	124.58 [14]
W_{wing} (N)	6.178e+04	9.1335e+04	N/A
D_{wing} (N)	1.077e+04	1.7344e+04	N/A

The optimization model resulted in an Aspect Ratio of 8.0, which is lower than both the Kirschen [3] and Boeing reference [14]. The calculated wing area exceeded both the Kirschen [3] and reference data[14], suggesting that the optimizer favored a larger surface area to maintain sufficient lift. The overall wing weight is smaller than the Kirschen model which is likely due to

the simplification of the structural and moment assumptions. The predicted drag is slightly lower but remains consistent with the expectation for a moderate aspect ratio design.

TABLE 3.19. Vertical Tail Variable Comparison

VT Variable	Solution Value	Kirschen	Reference
AR_{vt}	0.4309	2	1.91 [14]
b_{vt} (m)	2.092	8.35	7.06 [13]
S_{vt} (m^2)	10.16	34.9	26.44 [14]
W_{vt} (N)	358.2	1.7748e+04	N/A
D_{vt} (N)	1.929e+03	6.548e+03	N/A

The vertical tail appears to have the largest discrepancies. Such results may have come from constraints or simplifications in the tail volume coefficient formulation. Consequently, both the predicted tail weight and drag are much smaller than those in the Kirschen baseline [3]. Future refinements may include updated stability and control derivatives to better constrain vertical tail sizing.

TABLE 3.20. Horizontal Tail Variable Comparison

HT Variable	Solution Value	Kirschen	Reference
AR_{ht}	5.611	6.4	6.16 [14]
b_{ht} (m)	10.29	12.3	14.35 [13]
S_{ht} (m^2)	18.87	23	32.78 [14]
W_{ht} (N)	1.858e+04	2.797e+03	N/A
D_{ht} (N)	1.226e+03	1.505e+03	N/A
SM	0.01046	0.15	N/A

The horizontal aspect ratio closely aligns with the Boeing reference data [14], however the surface area is smaller than both the reference datasets. This suggests a more compact tailplane optimized under simplified stability requirements. The resulting static margin is positive but lower than typical values, implying that the configuration may operate near the stability limit. The tail's predicted weight is higher expected, possibly due to overestimated structural load factors or conservative material assumptions.

The optimized fuselage radius is nearly identical to both references, however the length is slightly smaller suggesting that some tradeoffs were made to balance weight and aerodynamic

TABLE 3.21. Fuselage Variable Comparison

Fuselage Variable	Solution Value	Kirschen	Reference
R_{fuse} (m)	1.855	1.85	1.88 [13]
l_{fuse} (m)	33.18	36.6	38.02 [13]
W_{fuse} (N)	5.063e+05	1.5734e+05	N/A
D_{fuse} (N)	6.602e+04	8.844e+03	N/A

drag. The total fuselage weight and drag are larger than the Kirschen results [3], likely due to simplified aerodynamic modeling and the absence of the detailed bending models. These discrepancies highlight the sensitivity of fuselage performance to drag estimation methods.

3.7 Overall Analysis and Limitations

Overall, the comparison between the optimized configuration, the Kirschen baseline model [3], and the Boeing 737-800 reference data [14, 12, 13] reveals consistent geometric trends but notable deviations in weight and aerodynamic performance. The optimized design generally produced larger lifting surfaces and higher total drag, reflecting a more conservative aerodynamic tradeoff within the current constraint framework. Differences in tail sizing and fuselage mass suggest that simplified stability, control, and structural assumptions in the optimized process led to over-predicted component weights.

Despite these discrepancies, the results demonstrate that the framework successfully captures the fundamental relationships among key aircraft parameters. The trends align with expected aerodynamic behavior, supporting the model's validity as a conceptual design and optimization tool. Future refinements are expected to improve alignment with both empirical data and established design references. Particularly in drag modeling, structural weight estimation and stability constraints.

CHAPTER 4

CONCLUSIONS AND OUTLOOK

This study presented an integrated optimization based approach to aircraft design, leveraging signomial programming to analyze the relationships among aerodynamic performance, structural efficiency, and overall configuration geometry. By modeling the primary components, the framework provided a comprehensive view of how design variables interact to achieve feasible and balanced solutions.

The optimized results, when compared to the Kirschen baseline and reference data from the Boeing 737 families, demonstrate that the methodology captures realistic aerodynamic and geometric trends. While the current formulation tends to overpredict component weights and drag, these outcomes highlight the tradeoffs inherent in simplified aerodynamic and structural models. The general consistency in geometric ratios, aspect ratios, and fuselage dimensions suggests that the optimization framework provides a credible foundation for conceptual design studies.

Beyond its immediate findings, this work illustrates the value of mathematical programming as a unifying language for early-stage aircraft design. By embedding aerodynamic, structural, and geometric considerations into a transparent design process, one can reduce reliance on iteration and instead promote data driven decision making. Such an approach has the potential to accelerate design cycles, improve traceability of assumptions, and ultimately support the development of more efficient and sustainable aircraft. In practice, this framework could be integrated into preliminary design workflows, enabling engineers to evaluate unconventional configurations more systematically and with greater confidence.

Future work should focus on refining the drag and stability modeling, improving structural weight correlations, and incorporating higher-fidelity aerodynamic data. Expanding the multi-fidelity framework will enable tighter coupling between geometry generation, aerodynamic analysis, and optimization. In addition, extending the methodology to include emerging design objectives, like fuel burn reduction, noise minimization, or hybrid electric propulsion integration, can

further demonstrate its adaptability to the evolving priorities of the aerospace industry.

In a broader context, this research highlights the growing role of optimization in addressing the aerospace industry's most pressing challenges, from reducing environmental impact to accelerating the evaluation of novel configurations. Beyond its technical contributions, the framework also holds educational value, by unifying aerodynamic, structural, and geometric considerations into a single optimization environment, it can serve as a powerful teaching tool for conceptual design courses, helping students visualize tradeoffs and develop intuition for integrated aircraft design. This dual role as both a research tool and an educational platform underscores its versatility and long term relevance.

In addition to these contributions, a key novelty of this research lies in the transition from GPKit to Pyomo. This methodological shift provided greater solver control, extensibility, and the ability to incorporate nonlinear formulations and custom routines, establishing a more robust and adaptable foundation for signomial-based optimization in aircraft design.

Ultimately, this research underscores the promise of optimization based methods in shaping the next generation of aircraft design. By bridging mathematical rigor with engineering intuition, the framework developed here lays the groundwork for more integrated, efficient, and forward looking design practices that can meet both current and future challenges in aerospace engineering. As the industry advances toward sustainable aviation, autonomous systems, and novel configurations, the principles demonstrated here will remain central to guiding innovation.

APPENDICES

APPENDIX A
FULL MODEL

$$R = \frac{\eta}{\text{SFC}} \frac{L}{D} \left(\frac{W_{\text{initial}}}{W_{\text{final}}} \right) \quad (\text{A.1})$$

$$\sum_{n=1}^N R_n \geq R_{\text{req}} \quad (\text{A.2})$$

$$W_{\text{fuel}_n} \geq \left(z_{\text{bre}_n} + \frac{z_{\text{bre}_n}^2}{2} + \frac{z_{\text{bre}_n}^3}{6} \right) W_{\text{end}_n} \quad (\text{A.3})$$

$$z_{\text{bre}_n} \geq c_T t_n \frac{D_n}{W_{\text{avg}_n}} \quad (\text{A.4})$$

$$t_n = \frac{R_n}{V_{\infty_n}} \quad (\text{A.5})$$

$$\left(\frac{L}{D} \right)_n = \frac{W_{\text{avg}_n}}{D_n} \quad (\text{A.6})$$

$$D_n \leq n_{\text{eng}} T_n \quad (\text{A.7})$$

$$T_n = T_{\text{TO}} \left(\frac{\rho_{\infty}}{\rho_{\text{TO}}} \right) \quad (\text{A.8})$$

$$W_{\text{avg}_n} \geq \sqrt{W_{\text{start}_n} W_{\text{end}_n}} + W_{\text{buoy}_n} \quad (\text{A.9})$$

$$W_{\text{start}_0} = W_{\text{max}} \quad (\text{A.10})$$

$$W_{\text{start}_n} \geq W_{\text{end}_n} + W_{\text{fuel}_n} \quad (\text{A.11})$$

$$W_{\text{start}_{n+1}} = W_{\text{end}_n} \quad (\text{A.12})$$

$$W_{\text{end}_n} \geq W_{\text{dry}} + W_{\text{pay}} + f_{\text{fuel}_{\text{res}}} W_{\text{fuel}_{\text{prim}}} \quad (\text{A.13})$$

$$W_{\text{max}} \geq W_{\text{dry}} + W_{\text{pay}} + W_{\text{fuel}_{\text{prim}}} (1 + f_{\text{fuel}_{\text{res}}}) \quad (\text{A.14})$$

$$W_{\text{fuel}_{\text{prim}}} \geq \sum_{n=1}^N W_{\text{fuel}_n} \quad (\text{A.15})$$

$$W_{\text{dry}} \geq W_{\text{wing}} + W_{\text{fuse}} + W_{\text{vt}} + W_{\text{ht}} + W_{\text{lg}} + n_{\text{eng}} W_{\text{eng}} + W_{\text{misc}} \quad (\text{A.16})$$

$$D_n \geq D_{\text{wing}_n} + D_{\text{fuse}_n} + D_{\text{vt}_n} + D_{\text{ht}_n} \quad (\text{A.17})$$

$$M = \frac{V_{\infty}}{a} \quad (\text{A.18})$$

$$M = 0.78 \quad (\text{A.19})$$

$$x_{\text{TO}} \leq l_r \quad (\text{A.20})$$

$$1 + y \leq 2 \frac{g x_{\text{TO}} T_{\text{TO}}}{V_{\text{TO}}^2 W_{\text{max}}} \quad (\text{A.21})$$

$$1 \geq 0.0464 \frac{\zeta^{2.7}}{y^{2.9}} + 1.044 \frac{\zeta^{0.3}}{y^{0.049}} \quad (\text{A.22})$$

$$\zeta \geq \frac{1}{2} \frac{\rho_{\text{TO}} V_{\text{TO}}^2 S_w C_D}{T_{\text{TO}}} \quad (\text{A.23})$$

$$V_{\text{TO}} = 1.2 \sqrt{\frac{2 W_{\text{max}}}{C_{L_w, \text{max}} S_w \rho_{\text{TO}}}} \quad (\text{A.24})$$

$$\begin{aligned} W_{\text{end} x_{\text{CG}_n}} &\geq W_{\text{wing}}(x_{\text{wing}} + \Delta x_{\text{ac}_w}) + W_{\text{fuel}_{\text{prim}}}(f_{\text{fuel}_n} + f_{\text{fuel}_{\text{res}}})(x_{\text{wing}} + \Delta x_{\text{ac}_w} f_{\text{fuel}_n}) \\ &+ \frac{1}{2}(W_{\text{fuse}} + W_{\text{pay}})l_{\text{fuse}} + W_{\text{ht}}x_{\text{CG}_{\text{ht}}} + W_{\text{vt}}x_{\text{CG}_{\text{vt}}} + n_{\text{eng}}W_{\text{eng}}x_b + W_{\text{lg}}x_{\text{CG}_{\text{lg}}} + W_{\text{misc}}x_{\text{CG}_{\text{misc}}} \end{aligned} \quad (\text{A.25})$$

$$f_{\text{fuel}_n} \geq \frac{\sum_{n=1}^n W_{\text{fuel}_n}}{W_{\text{fuel}_{\text{prim}}}} \quad (\text{A.26})$$

$$I_z = I_{z_{\text{wing}}} + I_{z_{\text{fuse}}} + I_{z_{\text{tail}}} \quad (\text{A.27})$$

$$I_{z_{\text{wing}}} = \frac{n_{\text{eng}} W_{\text{eng}} y_{\text{eng}}^2}{g} + \left(\frac{W_{\text{fuel}_{\text{wing}}} + W_{\text{wing}}}{g} \right) \frac{b_w^3 c_{\text{root}_w}}{16 S_w} \left(\lambda_w + \frac{1}{3} \right) \quad (\text{A.28})$$

$$I_{z_{\text{fuse}}} = \left(\frac{W_{\text{fuse}} + W_{\text{pay}}}{g} \right) \left(\frac{x_{\text{wing}}^3 + l_{\text{vt}}^3}{3 l_{\text{fuse}}} \right) \quad (\text{A.29})$$

$$I_{z_{\text{tail}}} = \left(\frac{W_{\text{apu}} + W_{\text{vt}} + W_{\text{ht}}}{g} \right) l_{\text{vt}}^2 \quad (\text{A.30})$$

$$W_{\text{lg}} = 14696.92 \quad \text{N} \quad (\text{A.31})$$

$$W_{\text{misc}} = 102340.356 \quad \text{N} \quad (\text{A.32})$$

$$x_{\text{CG}_{\text{lg}}} = 19.3 \quad \text{m} \quad (\text{A.33})$$

$$x_{\text{CG}_{\text{misc}}} = 10.0 \quad \text{m} \quad (\text{A.34})$$

$$S_w = b_w \left(\frac{c_{\text{root}_w} + c_{\text{tip}_w}}{2} \right) \quad (\text{A.35})$$

$$p_w = 1 + 2\lambda_w \quad (\text{A.36})$$

$$q_w = 1 + \lambda_w \quad (\text{A.37})$$

$$\bar{c}_w \geq \frac{2}{3} \left(\frac{1 + \lambda_w + \lambda_w^2}{q_w} \right) c_{\text{root}_w} \quad (\text{A.38})$$

$$y_{\bar{c}_w} = \frac{b_w q_w}{3 p_w} \quad (\text{A.39})$$

$$\lambda_w = \frac{c_{\text{tip}_w}}{c_{\text{root}_w}} \quad (\text{A.40})$$

$$\lambda_w = \lambda_{w_{\text{min}}} \quad (\text{A.41})$$

$$b_w \leq b_{w_{\text{max}}} \quad (\text{A.42})$$

$$AR_w = \frac{b_w^2}{S_w} \quad (\text{A.43})$$

$$e(1 + AR_w f(\lambda_w)) \leq 1 \quad (\text{A.44})$$

$$f(\lambda_w) \geq 0.0524\lambda_w^4 - 0.15\lambda_w^3 + 0.1659\lambda_w^2 - 0.0706\lambda_w + 0.0119 \quad (\text{A.45})$$

$$L_{\text{total}} \geq W_{\text{avg}} + L_{\text{ht}} \quad (\text{A.46})$$

$$L_{\text{total}} = f(L_{\text{total}_w})L_w \quad (\text{A.47})$$

$$L_{\text{total}} = \frac{1}{2}\rho_{\infty}S_w C_{L_w} V_{\infty}^2 \quad (\text{A.48})$$

$$W_{\text{wing}} \geq W_{\text{struct}_w} (1 + f_{\text{flap}} + f_{\text{slat}} + f_{\text{aileron}} + f_{\text{lete}} + f_{\text{ribs}} + f_{\text{spoiler}} + f_{\text{watt}}) \quad (\text{A.49})$$

$$W_{\text{struct}_w} \geq W_{\text{cap}} + W_{\text{web}} \quad (\text{A.50})$$

$$W_{\text{cap}} \geq \frac{8\rho_{\text{cap}} g r_{w/c} t_{\text{cap}} S_w^{1.5} \nu}{3AR_w^{0.5}} \quad (\text{A.51})$$

$$W_{\text{web}} \geq \frac{8\rho_{\text{web}} g r_h \tau_w t_{\text{web}} S_w^{1.5} \nu}{3AR_w^{0.5}} \quad (\text{A.52})$$

$$\nu^{3.94} \geq 0.14 p_w^{0.56} + \frac{0.86}{p_w^{2.4}} \quad (\text{A.53})$$

$$0.4232\tau_w^2 t_{\text{cap}} r_{w/c} \geq 0.92\tau_w t_{\text{cap}}^2 r_{w/c} + I_{\text{cap}} \quad (\text{A.54})$$

$$\frac{AR_w M_r N_{\text{lift}} \tau_w q_w^2}{I_{\text{cap}} S_w \sigma_{\text{max}}} \leq 8 \quad (\text{A.55})$$

$$\frac{AR_w L_{w_{\text{max}}} N_{\text{lift}} q_w^2}{S_w \sigma_{\text{max}_{\text{shear}}} \tau_w t_{\text{web}}} \leq 12 \quad (\text{A.56})$$

$$M_r \geq \frac{AR_w L_{w_{\text{max}}} p_w}{24} \quad (\text{A.57})$$

$$D_w = \frac{1}{2}\rho_{\infty} V_{\infty}^2 S_w C_{D_w} \quad (\text{A.58})$$

$$C_{D_w} \geq C_{D_{p_w}} + C_{D_{l_w}} \quad (\text{A.59})$$

$$\begin{aligned} C_{D_{p_w}}^{1.65} \geq & 1.61 \left(\frac{Re_w}{1000} \right)^{-0.55} \tau_w^{1.29} (M \cos(\Lambda))^{3.04} C_{L_w}^{1.78} \\ & + 0.0466 \left(\frac{Re_w}{1000} \right)^{-0.389} \tau_w^{0.784} (M \cos(\Lambda))^{-0.340} C_{L_w}^{0.951} \\ & + 191 \left(\frac{Re_w}{1000} \right)^{-0.219} \tau_w^{3.95} (M \cos(\Lambda))^{19.3} C_{L_w}^{1.15} \\ & + 2.82e - 12 \left(\frac{Re_w}{1000} \right)^{1.18} \tau_w^{-1.76} (M \cos(\Lambda))^{0.105} C_{L_w}^{-1.44} \end{aligned} \quad (\text{A.60})$$

$$Re_w = \frac{\rho_\infty V_\infty \bar{c}_w}{\mu} \quad (\text{A.61})$$

$$C_{D_{l_w}} \geq f_{\text{tip}} \frac{C_{L_w}^2}{\pi e A R_w} \quad (\text{A.62})$$

$$\Delta x_{\text{ac}_w} \geq \frac{1}{2} \tan \Lambda A R_w c_{\text{root}_w} \left(\frac{1}{3} + \frac{2}{3} \lambda_w \right) \quad (\text{A.63})$$

$$V_{\text{fuel}_{\max}} \leq 0.303 \bar{c}_w^2 b_w \tau_w \quad (\text{A.64})$$

$$W_{\text{fuel}_w} \leq \rho_{\text{fuel}} V_{\text{fuel}_{\max}} g \quad (\text{A.65})$$

$$W_{\text{fuel}_w} \geq \frac{f_{\text{fuel}_w} W_{\text{fuel}_{\text{total}}}}{f_{\text{fuel}_{\text{use}}}} \quad (\text{A.66})$$

$$l_{\text{vt}} \leq \Delta x_{\text{lead}_{\text{vt}}} + z_{\bar{c}_{\text{vt}}} \tan(\Lambda_{\text{vt}}) + 0.25 \bar{c}_{\text{vt}} \quad (\text{A.67})$$

$$\Delta x_{\text{trail}_{\text{vt}}} \geq \Delta x_{\text{lead}_{\text{vt}}} + c_{\text{root}_{\text{vt}}} \quad (\text{A.68})$$

$$l_{\text{fuse}} \geq x_{\text{CG}} + \Delta x_{\text{trail}_{\text{vt}}} \quad (\text{A.69})$$

$$x_{\text{CG}_{\text{vt}}} \geq x_{\text{CG}} + \frac{1}{2} (\Delta x_{\text{lead}_{\text{vt}}} + \Delta x_{\text{trail}_{\text{vt}}}) \quad (\text{A.70})$$

$$L_{\text{vt}_{\text{max}}} = \frac{1}{2} \rho_{\text{TO}} V_{\text{ne}}^2 S_{\text{vt}} C_{L_{\text{vt}_{\text{max}}}} \quad (\text{A.71})$$

$$S_{\text{vt}} = \frac{V_{\text{vt}} S_w b_w}{l_{\text{vt}}} \quad (\text{A.72})$$

$$S_{\text{vt}} \geq b_{\text{vt}} \left(\frac{c_{\text{root}_{\text{vt}}} + c_{\text{tip}_{\text{vt}}}}{2} \right) \quad (\text{A.73})$$

$$\bar{c}_{\text{vt}} = \frac{2}{3} \left(\frac{1 + \lambda_{\text{vt}} + \lambda_{\text{vt}}^2}{q_{\text{vt}}} \right) c_{\text{root}_{\text{vt}}} \quad (\text{A.74})$$

$$q_{\text{vt}} = 1 + \lambda_{\text{vt}} \quad (\text{A.75})$$

$$p_{\text{vt}} = 1 + 2\lambda_{\text{vt}} \quad (\text{A.76})$$

$$y_{\bar{c}_{\text{vt}}} = \frac{b_{\text{vt}} q_{\text{vt}}}{3p_{\text{vt}}} \quad (\text{A.77})$$

$$\lambda_{\text{vt}} = \frac{c_{\text{tip}_{\text{vt}}}}{c_{\text{root}_{\text{vt}}}} \quad (\text{A.78})$$

$$\lambda_{\text{vt}} \geq \lambda_{\text{vt}_{\text{min}}} \quad (\text{A.79})$$

$$AR_{\text{vt}} = \frac{b_{\text{vt}}^2}{S_{\text{vt}}} \quad (\text{A.80})$$

$$W_{\text{vt}} \geq W_{\text{struct}_{\text{vt}}} (1 + f_{\text{flap}} + f_{\text{lete}} + f_{\text{ribs}} + f_{\text{spoiler}} + f_{\text{watt}}) \quad (\text{A.81})$$

$$W_{\text{struct}_{\text{vt}}} \geq W_{\text{cap}_{\text{vt}}} + W_{\text{web}_{\text{vt}}} \quad (\text{A.82})$$

$$W_{\text{cap}_{\text{vt}}} \geq \frac{8\rho_{\text{cap}} g r_{\text{vt}_{w/c}} t_{\text{cap}_{\text{vt}}} v_{\text{vt}} S_{\text{vt}}^{1.5}}{3AR_{\text{vt}}^{0.5}} \quad (\text{A.83})$$

$$W_{\text{web}_{\text{vt}}} \geq \frac{8\rho_{\text{web}} g r_{\text{vt}_h} \tau_{\text{vt}} t_{\text{web}_{\text{vt}}} v_{\text{vt}} S_{\text{vt}}^{1.5}}{3AR_{\text{vt}}^{0.5}} \quad (\text{A.84})$$

$$v_{\text{vt}}^{3.94} \geq 0.14 p_{\text{vt}}^{0.54} + \frac{0.84}{p_{\text{vt}}^{2.4}} \quad (\text{A.85})$$

$$\frac{AR_{\text{vt}} M_{r_{\text{vt}}} N_{\text{lift}} \tau_{\text{vt}} q_{\text{vt}}^2}{I_{\text{cap}_{\text{vt}}} S_{\text{vt}} \sigma_{\text{max}}} \leq 8 \quad (\text{A.86})$$

$$\frac{AR_{\text{vt}}L_{\text{vtmax}}N_{\text{lift}}q_{\text{vt}}^2}{S_{\text{vt}}\sigma_{\text{maxshear}}\tau_{\text{vt}}t_{\text{webvt}}} \leq 12 \quad (\text{A.87})$$

$$M_{r_{\text{vt}}} \geq \frac{AR_{\text{vt}}L_{\text{vtmax}}p_{\text{vt}}}{24} \quad (\text{A.88})$$

$$L_{\text{vtEO}}l_{\text{vt}} \geq D_{\text{wm}}y_{\text{eng}} + T_{\text{TO}}y_{\text{eng}} \quad (\text{A.89})$$

$$L_{\text{vtEO}} = \frac{1}{2}\rho_{\text{TO}}V_1^2S_{\text{vt}}C_{L_{\text{vtEO}}} \quad (\text{A.90})$$

$$c_{l_{\text{vtEO}}} \geq C_{L_{\text{vtEO}}} \left(1 + \frac{c_{l_{\text{vtEO}}}}{\pi e_{\text{vt}}AR_{\text{vt}}} \right) \quad (\text{A.91})$$

$$D_{\text{wm}} \geq \frac{1}{2}\rho_{\text{TO}}V_1^2A_{\text{fan}}C_{D_{\text{wm}}} \quad (\text{A.92})$$

$$\frac{\dot{r}_{\text{req}}}{I_z} \leq \frac{1}{2}\rho_{\text{TO}}V_{\text{land}}^2S_{\text{vt}}l_{\text{vt}}C_{v_{\text{tlanding}}} \quad (\text{A.93})$$

$$D_{\text{vt}} \geq \frac{1}{2}\rho_{\infty}V_{\infty}^2S_{\text{vt}}C_{D_{p_{\text{vt}}}} \quad (\text{A.94})$$

$$\begin{aligned} C_{D_{p_{\text{vt}}}}^{1.189} &\geq 2.44 \times 10^{-77} (Re_{\text{vt}})^{-0.528} (\tau_{\text{vt}})^{133.8} (M)^{1022.7} \\ &+ 0.003 (Re_{\text{vt}})^{-0.410} (\tau_{\text{vt}})^{1.22} (M)^{1.55} \\ &+ 1.967 \times 10^{-4} (Re_{\text{vt}})^{0.214} (\tau_{\text{vt}})^{-0.04} (M)^{-0.14} \\ &+ 6.590 \times 10^{-50} (Re_{\text{vt}})^{-0.498} (\tau_{\text{vt}})^{1.56} (M)^{-114.6} \end{aligned} \quad (\text{A.95})$$

$$Re_{\text{vt}} = \frac{\rho_{\infty}V_{\infty}\bar{c}_{\text{vt}}}{\mu} \quad (\text{A.96})$$

$$S_{\text{ht}} = \frac{V_{\text{ht}}S_w\bar{c}_{\text{ht}}}{l_{\text{ht}}} \quad (\text{A.97})$$

$$S_{\text{ht}} = b_{\text{ht}} \left(\frac{c_{\text{root}_{\text{ht}}} + c_{\text{tip}_{\text{ht}}}}{2} \right) \quad (\text{A.98})$$

$$\bar{c}_{\text{ht}} \geq \frac{2}{3} \left(\frac{1 + \lambda_{\text{ht}} + \lambda_{\text{ht}}^2}{q_{\text{ht}}} \right) c_{\text{root}_{\text{ht}}} \quad (\text{A.99})$$

$$y_{\bar{c}_{\text{ht}}} = \frac{b_{\text{ht}} q_{\text{ht}}}{3 p_{\text{ht}}} \quad (\text{A.100})$$

$$q_{\text{ht}} = 1 + \lambda_{\text{ht}} \quad (\text{A.101})$$

$$p_{\text{ht}} = 1 + 2\lambda_{\text{ht}} \quad (\text{A.102})$$

$$\lambda_{\text{ht}} = \frac{c_{\text{tip}_{\text{ht}}}}{c_{\text{root}_{\text{ht}}}} \quad (\text{A.103})$$

$$\lambda_{\text{ht}} \geq \lambda_{\text{ht}_{\min}} \quad (\text{A.104})$$

$$AR_{\text{ht}} = \frac{b_{\text{ht}}^2}{S_{\text{ht}}} \quad (\text{A.105})$$

$$e(1 + AR_{\text{ht}} f(\lambda_{\text{ht}})) \leq 1 \quad (\text{A.106})$$

$$f(\lambda_{\text{ht}}) \geq 0.0524\lambda_{\text{ht}}^4 - 0.15\lambda_{\text{ht}}^3 + 0.1659\lambda_{\text{ht}}^2 - 0.0706\lambda_{\text{ht}} + 0.0119 \quad (\text{A.107})$$

$$l_{\text{ht}} \leq \Delta x_{\text{lead}_{\text{ht}}} + 0.25\bar{c}_{\text{ht}} \quad (\text{A.108})$$

$$\Delta x_{\text{trail}_{\text{ht}}} \geq \Delta x_{\text{lead}_{\text{ht}}} + c_{\text{root}_{\text{ht}}} \quad (\text{A.109})$$

$$l_{\text{fuse}} \geq x_{\text{CG}} + \Delta x_{\text{trail}_{\text{ht}}} \quad (\text{A.110})$$

$$x_{\text{CG}_{\text{ht}}} \geq x_{\text{CG}} + \frac{1}{2}(\Delta x_{\text{lead}_{\text{ht}}} + \Delta x_{\text{trail}_{\text{ht}}}) \quad (\text{A.111})$$

$$L_{\text{ht}_{\max}} = \frac{1}{2} \rho_{\text{TO}} S_{\text{ht}} c_{L_{\text{ht}_{\max}}} V_{\text{ne}}^2 \quad (\text{A.112})$$

$$L_{\text{ht}} l_{\text{ht}} \geq D_{\text{wm}} y_{\text{eng}} + T_{\text{TO}} y_{\text{eng}} \quad (\text{A.113})$$

$$L_{\text{ht}} = \frac{1}{2} \rho_{\text{TO}} S_{\text{ht}} C_{L_{\text{ht}}} V_{\text{l}}^2 \quad (\text{A.114})$$

$$W_{\text{ht}} \geq W_{\text{struct}_{\text{ht}}} (1 + f_{\text{flap}} + f_{\text{ribs}} + f_{\text{watt}}) \quad (\text{A.115})$$

$$W_{\text{struct}_{\text{ht}}} \geq W_{\text{cap}_{\text{ht}}} + W_{\text{web}_{\text{ht}}} \quad (\text{A.116})$$

$$W_{\text{cap}_{\text{ht}}} \geq \frac{8\rho_{\text{cap}} g r_{\text{ht}_{w/c}} t_{\text{cap}_{\text{ht}}} v_{\text{ht}} S_{\text{ht}}^{1.5}}{3AR_{\text{ht}}^{0.5}} \quad (\text{A.117})$$

$$W_{\text{web}_{\text{ht}}} \geq \frac{8\rho_{\text{web}} g r_{\text{ht}_h} \tau_{\text{ht}} t_{\text{web}_{\text{ht}}} v_{\text{ht}} S_{\text{ht}}^{1.5}}{3AR_{\text{ht}}^{0.5}} \quad (\text{A.118})$$

$$v_{\text{ht}}^{3.94} \geq 0.14 p_{\text{ht}}^{0.54} + \frac{0.84}{p_{\text{ht}}^{2.4}} \quad (\text{A.119})$$

$$\frac{AR_{\text{ht}} M_{r_{\text{ht}}} N_{\text{lift}} \tau_{\text{ht}} q_{\text{ht}}^2}{I_{\text{cap}_{\text{ht}}} S_{\text{ht}} \sigma_{\text{max}}} \leq 8 \quad (\text{A.120})$$

$$\frac{AR_{\text{ht}} L_{\text{ht}_{\text{max}}} N_{\text{lift}} q_{\text{ht}}^2}{S_{\text{ht}} \sigma_{\text{max}_{\text{shear}}} \tau_{\text{ht}} t_{\text{web}_{\text{ht}}}} \leq 12 \quad (\text{A.121})$$

$$M_{r_{\text{ht}}} \geq \frac{AR_{\text{ht}} L_{\text{ht}_{\text{max}}} p_{\text{ht}}}{24} \quad (\text{A.122})$$

$$\frac{x_w}{\bar{c}_w} \leq \frac{x_{\text{CG}}}{\bar{c}_w} + \frac{C_{m_{\text{ac}}}}{C_{L_w}} + \frac{V_{\text{ht}} C_{L_{\text{ht}}}}{C_{L_w}} \quad (\text{A.123})$$

$$SM_{\text{min}} + \frac{\Delta x_{\text{CG}}}{\bar{c}_w} + \frac{C_{m_{\text{ac}}}}{C_{L_{w_{\text{max}}}}} \leq V_{\text{ht}} m_{\text{ratio}} + \frac{V_{\text{ht}} C_{L_{\text{ht}_{\text{max}}}}}{C_{L_{w_{\text{max}}}}} \quad (\text{A.124})$$

$$1 + \frac{2}{AR_{\text{ht}}} = m_{\text{ratio}} \left(1 + \frac{2}{AR_w} \right) \quad (\text{A.125})$$

$$SM \leq \frac{x_w - x_{\text{CG}}}{\bar{c}_w} \quad (\text{A.126})$$

$$SM \geq SM_{\text{min}} \quad (\text{A.127})$$

$$D_{\text{ht}} \geq \frac{1}{2} \rho_{\infty} S_{\text{ht}} C_{D_{\text{ht}}} V_{\infty}^2 \quad (\text{A.128})$$

$$C_{D_{\text{ht}}} \geq C_{D_{0_{\text{ht}}}} + C_{D_{i_{\text{ht}}}} \quad (\text{A.129})$$

$$\begin{aligned} C_{D_{0_{\text{ht}}}}^{6.49} &\geq 5.288 * 10^{-20} (Re_{\text{ht}})^{0.901} (\tau_{\text{ht}})^{0.912} (M)^{8.645} \\ &+ 1.676 * 10^{-28} (Re_{\text{ht}})^{0.351} (\tau_{\text{ht}})^{6.292} (M)^{10.256} \\ &+ 7.098 * 10^{-25} (Re_{\text{ht}})^{1.395} (\tau_{\text{ht}})^{1.962} (M)^{0.567} \\ &+ 3.731 * 10^{-14} (Re_{\text{ht}})^{-2.574} (\tau_{\text{ht}})^{3.128} (M)^{0.448} \\ &+ 1.443 * 10^{-12} (Re_{\text{ht}})^{3.910} (\tau_{\text{ht}})^{4.663} (M)^{7.689} \quad (\text{A.130}) \end{aligned}$$

$$Re_{\text{ht}} = \frac{\rho_{\infty} V_{\infty} \bar{c}_{\text{ht}}}{\mu} \quad (\text{A.131})$$

$$C_{D_{i_{\text{ht}}}} \geq f_{\text{tip}} \frac{C_{L_{\text{ht}}}^2}{\pi e A R_{\text{ht}}} \quad (\text{A.132})$$

$$w_{\text{fuse}} = n_{\text{spr}} w_{\text{seat}} + w_{\text{aisle}} + 2w_{\text{sys}} \quad (\text{A.133})$$

$$A_{\text{skin}} \geq 2\pi R_{\text{fuse}} t_{\text{skin}} \quad (\text{A.134})$$

$$A_{\text{fuse}} = \pi R_{\text{fuse}}^2 \quad (\text{A.135})$$

$$R_{\text{fuse}} \geq \frac{w_{\text{fuse}}}{2} \quad (\text{A.136})$$

$$\sigma_x = \frac{\Delta P_{\text{over}}}{2} \frac{R_{\text{fuse}}}{t_{\text{shell}}} \quad (\text{A.137})$$

$$\sigma_{\theta} = \Delta P_{\text{over}} \frac{R_{\text{fuse}}}{t_{\text{skin}}} \quad (\text{A.138})$$

$$\sigma_{\text{skin}} \geq \sigma_x \quad (\text{A.139})$$

$$\sigma_{\text{skin}} \geq \sigma_{\theta} \quad (\text{A.140})$$

$$P_{\text{floor}} = N_{\text{land}}(W_{\text{pay}} + W_{\text{seats}}) \quad (\text{A.141})$$

$$S_{\text{floor}} = \frac{P_{\text{floor}}}{2} \quad (\text{A.142})$$

$$M_{\text{floor}} = \frac{P_{\text{floor}} w_{\text{floor}}}{8} \quad (\text{A.143})$$

$$A_{\text{floor}} = 1.5 \frac{S_{\text{floor}}}{\tau_{\text{floor}}} + 2 \frac{M_{\text{floor}}}{\sigma_{\text{floor}} h_{\text{floor}}} \quad (\text{A.144})$$

$$x_{\text{shell}_1} = l_{\text{nose}} \quad (\text{A.145})$$

$$x_{\text{shell}_2} = l_{\text{nose}} + l_{\text{shell}} \quad (\text{A.146})$$

$$n_{\text{seat}} = n_{\text{spr}} n_{\text{rows}} \quad (\text{A.147})$$

$$n_{\text{pass}} = n_{\text{seat}} \quad (\text{A.148})$$

$$l_{\text{shell}} \geq n_{\text{rows}} p_s \quad (\text{A.149})$$

$$l_{\text{floor}} \geq 2R_{\text{fuse}} + l_{\text{shell}} \quad (\text{A.150})$$

$$l_{\text{fuse}} = l_{\text{nose}} + l_{\text{shell}} + l_{\text{cone}} \quad (\text{A.151})$$

$$S_{\text{nose}}^{8/5} = \left(2\pi R_{\text{fuse}}^2 \right)^{8/5} \left(\frac{1}{3} + \frac{2}{3} \left(\frac{l_{\text{nose}}}{R_{\text{fuse}}} \right)^{8/5} \right) \quad (\text{A.152})$$

$$S_{\text{bulk}} = 2\pi R_{\text{fuse}}^2 \quad (\text{A.153})$$

$$V_{\text{cyl}} = A_{\text{skin}} l_{\text{shell}} \quad (\text{A.154})$$

$$V_{\text{nose}} = S_{\text{nose}} t_{\text{skin}} \quad (\text{A.155})$$

$$V_{\text{bulk}} = S_{\text{bulk}} t_{\text{skin}} \quad (\text{A.156})$$

$$V_{\text{cabin}} \geq A_{\text{fuse}} \left(\frac{2}{3} l_{\text{nose}} + l_{\text{shell}} + \frac{2}{3} R_{\text{fuse}} \right) \quad (\text{A.157})$$

$$Q_{\text{vt}} = \frac{L_{\text{vtmax}} b_{\text{vt}}}{3} \frac{1 + 2\lambda_{\text{vt}}}{1 + \lambda_{\text{vt}}} \quad (\text{A.158})$$

$$t_{\text{cone}} = \frac{Q_{\text{vt}}}{2A_{\text{fuse}}\tau_{\text{cone}}} \quad (\text{A.159})$$

$$R_{\text{fuse}}\tau_{\text{cone}}(1 + p_{\text{vt}})V_{\text{cone}}\frac{1 + \lambda_{\text{cone}}}{4l_{\text{cone}}} \geq L_{\text{vtmax}} b_{\text{vt}} \frac{p_{\text{vt}}}{3} \quad (\text{A.160})$$

$$\tau_{\text{cone}} = \sigma_{\text{skin}} \quad (\text{A.161})$$

$$l_{\text{cone}} = \frac{R_{\text{fuse}}}{\lambda_{\text{cone}}} \quad (\text{A.162})$$

$$t_{\text{shell}} = t_{\text{skin}} \left(1 + f_{\text{string}} r_E \frac{\rho_{\text{skin}}}{\rho_{\text{bend}}} \right) \quad (\text{A.163})$$

$$\sigma_{\text{bend}} \geq \sigma_{M_h} + r_E \frac{\Delta P_{\text{over}} R_{\text{fuse}}}{2t_{\text{shell}}} \quad (\text{A.164})$$

$$\sigma_{\text{bend}} \geq \sigma_{M_v} + r_E \frac{\Delta P_{\text{over}} R_{\text{fuse}}}{2t_{\text{shell}}} \quad (\text{A.165})$$

$$I_{h_{\text{shell}}} \leq \pi R_{\text{fuse}}^3 t_{\text{shell}} \quad (\text{A.166})$$

$$I_{v_{\text{shell}}} \leq \pi R_{\text{fuse}}^3 t_{\text{shell}} \quad (\text{A.167})$$

$$\begin{aligned} W_{\text{fuse}} &\geq W_{\text{apu}} + W_{\text{cone}} + W_{\text{floor}} \\ &\quad + W_{\text{insul}} + W_{\text{padd}} + W_{\text{seats}} \\ &\quad + W_{\text{shell}} + W_{\text{window}} + W_{\text{fix}} \end{aligned} \quad (\text{A.168})$$

$$W_{\text{skin}} \geq \rho_{\text{skin}} g (V_{\text{bulk}} + V_{\text{cyl}} + V_{\text{nose}}) \quad (\text{A.169})$$

$$W_{\text{shell}} \geq W_{\text{skin}} (1 + f_{\text{add}} + f_{\text{frame}} + f_{\text{string}}) \quad (\text{A.170})$$

$$V_{\text{floor}} \geq A_{\text{floor}} w_{\text{floor}} \quad (\text{A.171})$$

$$W_{\text{floor}} \geq V_{\text{floor}} \rho_{\text{floor}} g + W_{\text{floor}}'' l_{\text{floor}} w_{\text{floor}} \quad (\text{A.172})$$

$$W_{\text{cone}} \geq \rho_{\text{cone}} g V_{\text{cone}} (1 + f_{\text{fadd}} + f_{\text{frame}} + f_{\text{string}}) \quad (\text{A.173})$$

$$W_{\text{window}} \geq W'_{\text{window}} l_{\text{shell}} \quad (\text{A.174})$$

$$W_{\text{insul}} \geq W''_{\text{insul}} (0.55(S_{\text{bulk}} + S_{\text{nose}}) + 1.1\pi R_{\text{fuse}} l_{\text{shell}}) \quad (\text{A.175})$$

$$W_{\text{apu}} \geq W_{\text{pay}} f_{\text{apu}} \quad (\text{A.176})$$

$$W_{\text{padd}} \geq W_{\text{pay}} f_{\text{padd}} \quad (\text{A.177})$$

$$W_{\text{seats}} \geq W_{\text{seats}} n_{\text{seat}} \quad (\text{A.178})$$

$$\rho_{\text{cabin}} = \frac{p_{\text{cabin}}}{RT_{\text{cabin}}} \quad (\text{A.179})$$

$$W_{\text{buoy}} = \rho_{\text{cabin}} g V_{\text{cabin}} \quad (\text{A.180})$$

$$W_{\text{pay}} \geq W_{\text{avg.pass}} n_{\text{pass}} \quad (\text{A.181})$$

$$D_{\text{fuse}} = \frac{1}{2} \rho_{\infty} V_{\infty}^2 C_{D_{\infty}} \left(l_{\text{fuse}} R_{\text{fuse}} \frac{M^2}{M_{\text{fuse}_D}^2} \right) \quad (\text{A.182})$$

APPENDIX B
PYTHON SCRIPT

```

# =====
# Import Statements
# =====
import numpy as np
import pyomo
from formulation import Formulation
from pyomo.environ import units

# =====
# Declare Formulation
# =====
f = Formulation()

f.Objective(W_max) # Minimize Max weight at start

Constraints = []
# # Flight Performance Constraints
Constraints += [
    R[0] + R[1] + R[2] >= R_req,
    R[0] == R[1],
    R[1] == R[2],
]

# Flight Performance
for i in [0,1,2]:
    Constraints += [
        W_fuel[i] >= W_end[i] * (z_bre[i] + (z_bre[i]**2)/2 + (z_bre[i]**3)/6),
        z_bre[i] >= c_T * t[i] * (D[i] / W_avg[i]),
        t[i] >= R[i] / V_free[i],
        L_D[i] == W_avg[i] / D[i],
        D[i] <= n_eng * T[i],
        T[i] == T_TO * (rho_free / rho_TO), # # eta_throttle, # Simplified thrust model at cruise
        # L_D[i] <= 18,
        W_avg[i] == (W_start[i] * W_end[i])**0.5 + W_buoy,
        W_start[i] >= W_end[i] + W_fuel[i],
        W_end[i] >= W_dry + W_pay + f_fuelres * W_fuelprim,
        D[i] >= D_w[i] + D_fuse[i] + D_vt[i] + D_ht[i],
        M[i] == V_free[i] / a,
        M[i] >= M_crit,
        M[i] >= 0.75,
        1 >= 0.0464 * (zeta[i]**2.7 / y**2.9) + 1.044 * (zeta[i]**0.3 / y**0.049),
        zeta[i] >= 0.5 * ((rho_TO * S_w * C_D[i]) / T_TO),
        C_D[i] == C_Dfuse[i] + C_Dw[i] + C_Dht[i] + C_Dpvt[i],
    ]
    Constraints += [
        C_Lw[i] <= 1.2, # replacing the W_s for now
    ]

Constraints += [
    W_start[0] == W_max,
    W_start[1] == W_end[0],
    W_start[2] == W_end[1],
    W_fuelprim >= W_fuel[0] + W_fuel[1] + W_fuel[2],
    W_dry >= W_wing + W_fuse + W_vt + W_ht + W_lg + W_misc + (n_eng * W_eng),
    W_max >= W_dry + W_pay + W_fuelprim * (1 + f_fuelres), # MTOW constraint
    x_TO <= l_r, # Converted into one takeoff distance
    1 + y <= 2 * ((g * x_TO * T_TO) / (W_max * V_TO**2)),
    V_TO == 1.2 * ((2 * W_max) / (c_Lwmax * S_w * rho_TO))**0.5, # One takeoff velocity in a mission
]

# System Level Properties
for i in [0,1,2]:
    Constraints += [
        W_end[i] * x_CG >= W_wing * (x_wing + delt_xacw) +
        W_fuelprim * (f_fuel[i] + f_fuelres) * (x_wing + delt_xacw * f_fuel[i]) +

```

```

                                0.5 * (W_fuse + W_pay) * l_fuse + W_ht * x_CGht + W_vt * x_CGvt +
                                n_eng * W_eng * x_b + W_lg * x_CGlg + W_misc * x_CGmisc,
f_fuel[i]                      >= (W_fuel[0] + W_fuel[1] + W_fuel[2]) / W_fuelprim,
# # W_lg * x_CGlg[i]           >= W_mg * x_mg[i] + W_ng * x_ng[i],
# # W_misc * x_CGmisc[i]      >= W_hpesys * x_hpesys[i],
# # x_hpesys[i] >= 1,
I_z[i]                        == I_zwing[i] + I_zfuse[i] + I_ztail[i], # Converted from inequality to equality
I_zwing[i]                    == ((n_eng * W_eng * y_eng**2) / g) +
                                ((W_fuelwing + W_wing) / g) * ((c_rootw * b_w**3) / (16*S_w)) * (lambda_w + (1/3)),
I_zfuse[i]                    == ((W_fuse + W_pay) / g) * ((x_wing**3 + l_vt**3) / (3*l_fuse)), # look to replace x_wing with the proper variable
I_ztail[i]                    == ((W_apu + W_vt + W_ht) / g) * l_vt**2,
# Converted I_z to equality from inequality
]

# Wing Model Constraints
# Geometry
Constraints += [
    S_w          == b_w * ((c_rootw + c_tipw) / 2),
    cbar_w       == (2/3) * ((1 + lambda_w + lambda_w**2) / q_w) * c_rootw,
    y_cbarw      == (b_w * q_w) / (3 * p_w),
    lambda_w     == c_tipw / c_rootw,
    p_w         == 1 + 2 * lambda_w,
    q_w         == 1 + lambda_w,
    lambda_w     >= lambda_wmin,
    b_w         <= b_wmax,
    AR_w        == (b_w**2) / S_w,
    AR_w >= 8,
    e * (1 + (AR_w * f_lambdaw)) <= 1,
    f_lambdaw   >= 0.0524 * lambda_w**4 - 0.15 * lambda_w**3 + 0.1659 * lambda_w**2 - 0.0706 * lambda_w + 0.0119,
]

# Wing Lift
for i in [0,1,2]:
    Constraints += [
        L_total[i] == W_avg[i] + L_ht[i],
        L_total[i] == f_Ltotw * L_w[i],
        L_w[i]     == 0.5 * rho_free * S_w * C_Lw[i] * V_free[i]**2,
        W_s[i]     == 0.5 * rho_free * C_Lw[i] * V_free[i]**2,
        W_s[i]     <= W_smax,
    ]

# Wing Weight
Constraints += [
    W_wing      >= W_structw * (1 + f_flap + f_slats + f_aileron + f_lete + f_ribs + f_spoiler + f_watt),
    W_structw   >= (W_cap + W_web),
    W_cap       >= (8 * rho_cap * g * r_wc * t_cap * v * S_w**1.5) / (3 * AR_w**0.5),
    W_web       >= (8 * rho_web * g * r_h * tau_w * t_web * v * S_w**1.5) / (3 * AR_w**0.5),
    v**3.94     >= (0.14 * p_w**0.56) + (0.86 / p_w**2.4),
    # 2 * q_w    >= 1 + p_w,
    0.4232 * tau_w**2 * t_cap * r_wc >= 0.92 * tau_w * t_cap**2 * r_wc + I_cap,
    8          >= (AR_w * M_r * N_lift * tau_w * q_w**2) / (I_cap * S_w * sigma_max),
    12         >= (AR_w * L_wmax * N_lift * q_w**2) / (S_w * sigma_maxsh * tau_w * t_web),
    tau_w <= 0.14,
    M_r        >= (AR_w * L_wmax * p_w) / 24, # from hoburg.
    # M_r * c_rootw >= (L_wmax - N_lift * (W_wing + f_fuelw * W_fueltotal)) * ((b_w**2 / (12*S_w)) * (c_rootw + 2 * c_tipw)) - (N_lift * W_eng * y_eng),
]

# Wing Drag
for i in [0,1,2]:
    Constraints += [
        D_w[i] == 0.5 * rho_free * S_w * C_Dw[i] * V_free[i]**2,
        # D_w[i] <= 20000,
        C_Dw[i] >= C_Dpw[i] + C_Diw[i], # + C_Dww[i],
        # C_Dww[i]*10.0 >= ( np.exp(-86.20313638873553) * (M[i])**((116.07558760138559) +
        #                    np.exp(-18.409861023575278) * (M[i])**((27.77453413127811) +
        #                    np.exp(-46.13163108464548) * (M[i])**((65.53485981024158) +

```



```

#                               np.exp(0.252540692919293) * (M[i])**(-0.5243391493897814) ),

C_Dpw[i]**1.65 >= 1.61 * (Re_w[i] / 1000)**-0.55 * tau_w**1.29 * (M[i] * cos_chord)**3.04 * C_Lw[i]**1.78 +
0.0466 * (Re_w[i] / 1000)**-0.389 * tau_w**0.784 * (M[i] * cos_chord)**-0.34 * C_Lw[i]**0.951 +
191 * (Re_w[i] / 1000)**-0.219 * tau_w**3.95 * (M[i] * cos_chord)**19.3 * C_Lw[i]**1.15 +
(2.82 * (10**-12) ) * (Re_w[i] / 1000)**1.18 * tau_w**-1.76 * (M[i] * cos_chord)**0.105 * C_Lw[i]**-1.44,

Re_w[i]      == (rho_free * V_free[i] * cbar_w) / mu,
C_Diw[i]      >= f_tip * (C_Lw[i]**2 / (pi * e * AR_w)),
# # e        <= 1 / (1 + (AR_w * f_lambdaw[i])), # ignore this for now
# e * (1 + (AR_w * f_lambdaw[i]))          <= 1, # moved to geometry
# f_lambdaw[i] >= 0.0524 * lambda_w**4 - 0.15 * lambda_w**3 + 0.1659 * lambda_w**2 - 0.0706 * lambda_w + 0.0119, # moved to geometry
]

# Wing Aerodynamic Center
Constraints += [
    delt_xacw    >= (1/4) * tan_chord * AR_w * c_rootw * ((1/3) + (2/3) * lambda_w),
]

# Fuel Volume
Constraints += [
    V_fuelmax    <= 0.303 * cbar_w**2 * b_w * tau_w,
    W_fuelwing    <= rho_fuel * V_fuelmax * g,
    W_fuelwing    >= (f_fuelw * W_fuelprim) / f_fueluse,
]

# Vertical Tail Model Constraints
# Vertical Tail Geometry and Structure
Constraints += [
    l_vt         <= delt_xlvt + z_cbarvt * tan_chdvt + 0.25 * cbar_vt,
    delt_xtvt     >= delt_xlvt + c_rootvt,
    c_rootvt     >= 5, # added constraint
    l_fuse        >= x_CG + delt_xtvt,
    x_CGvt        >= x_CG + 0.5 * (delt_xlvt + delt_xtvt),
    L_vtmax[1]    == 0.5 * rho_TO * S_vt * c_lvtmax * V_ne**2, # originally using S_vtcorrected but used S_vt for now
]

# Vertical Tail Geometry and Structure
Constraints += [
    # Review VT geometry equations. Should be halfspan.
    S_vt == (V_vt * S_w * b_w) / l_vt, # Added to properly size VT
    S_vt >= b_vt * ((c_rootvt + c_tipvt) / 2),
    cbar_vt == (2/3) * ((1 + lambda_vt + lambda_vt**2) / q_vtd) * c_rootvt,
    q_vtd == l+lambda_vt,
    y_cbarvt == (b_vt * q_vtd) / (3 * p_vt),
    p_vt == 1+2 * lambda_vt,
    lambda_vt == c_tipvt / c_rootvt,
    lambda_vt >= lambda_vtmin,
    AR_vt      == (b_vt**2) / S_vt,
    # # b_vt_corrected == b_vt / 2, # to get half span
    # S_vtcorrected == S_vt / 2, # to get half area
]

# Vertical Tail Weight
Constraints += [
    W_vt      >= W_structvt * (1 + f_flap + f_lete + f_ribs + f_spoiler + f_watt), # Need to update with correct terms, rudder, etc.
    W_structvt >= W_capvt + W_webvt,
    W_capvt    >= (8 * rho_cap * g * r_vtvc * t_capvt * vvt * S_vt**1.5) / (3 * AR_vt**0.5),
    W_webvt    >= (8 * rho_web * g * r_vth * tau_vt * t_webvt * vvt * S_vt**1.5) / (3 * AR_vt**0.5),
    vvt**3.94  >= (0.14 * p_vt**0.54) + (0.84 / p_vt**2.4), # Don't know if a different dummy variable is needed here or we can reuse from wing
    # # 2 * q_vtd      >= 1 + p_vt,
    0.4232 * tau_vt**2 * t_capvt * r_vtvc >= 0.92 * tau_vt * t_capvt**2 * r_vtvc + I_capvt, # What is tau_rvtc?
    8          >= (AR_vt * M_rvt * N_lift * tau_vt * q_vtd**2) / (I_capvt * S_vt * sigma_max),
    12         >= (AR_vt * L_vtmax[0] * N_lift * q_vtd**2) / (S_vt * sigma_maxsh * tau_vt * t_webvt), # Using L_vtmax[0] since its constant
    tau_vt <= 0.14,
    M_rvt      >= (AR_vt * L_vtmax[0] * p_vt) / 24, # I believe this constraint is needed since its not carrying engines
    # # M_rvt[i] * c_rootvt >= (L_vtmax[i]) * ((b_vt / 12) * (c_rootvt + c_tipvt)) * (b_vt / S_vt), #Not this one

```

```

        # W_vtcorrected == W_vt / 2, # to get half weight
        # # AR_vtcorrected == AR_vt / 2, # to get half aspect ratio
    ]

# Engine-Out Condition
for i in [0,1,2]:
    Constraints += [
        L_vtEO[i] * l_vt    >= D_wm[i] * y_eng + T_TO * y_eng,
        L_vtEO[i]          == 0.5 * rho_TO * S_vt * C_LvtEO[i] * V1**2, # originally using S_vtcorrected but used S_vt for now
        c_lvt_EO           >= C_LvtEO[i] * (1 + (c_lvt_EO / (pi * e * AR_vt))), # originally using e_vt but using e for now
        D_wm[i]            >= 0.5 * rho_TO * A_fan * c_Dwm * V1**2,
    ]

# Crosswind Landing Condition
for i in [0,1,2]:
    Constraints += [
        rdot_req / I_z[i]  <= 0.5 * rho_TO * S_vt * l_vt * C_Lvtland[i] * V_land**2,
    ]

# Vertical Tail Drag
for i in [0,1,2]:
    Constraints += [
        D_vt[i]            >= 0.5 * rho_free * S_vt * C_Dpvt[i] * V_free[i]**2,
        C_Dpvt[i]**1.189   >= 2.44e-77 * Re_vt[i]**-0.528 * tau_vt**133.8 * M[i]**1022.7 +
        0.003 * Re_vt[i]**-0.41 * tau_vt**1.22 * M[i]**1.55 +
        1.967e-4 * Re_vt[i]**0.214 * tau_vt**0.04 * M[i]**-0.14 +
        6.59e-50 * Re_vt[i]**-0.498 * tau_vt**1.56 * M[i]**114.6,
        Re_vt[i]           == (rho_free * V_free[i] * cbar_vt) / mu,
    ]

# Fuselage Model Constraints
Constraints += [
    wdt_fuse    == n_spr * w_seatwidth + w_aisle + 2 * w_sys,
    A_skin     >= 2 * pi * R_fuse * t_skin,
    A_fuse     == pi * R_fuse**2, # Converted from inequality to equality
    A_skin >= 0.01,
    R_fuse >= wdt_fuse / 2, # added this constraint since wdt_fuse was not used elsewhere
]

# Shell Geometry
Constraints += [
    x_shell1    == l_nose,
    x_shell2    == l_nose + l_shell, # Converted from inequality to equality
    n_seats     == n_spr * n_rows,
    n_pass      == n_seats,
    l_shell     >= n_rows * p_s,
    l_floor     >= 2 * R_fuse + l_shell, # Converted from inequality to equality
    l_fuse      == l_nose + l_shell + l_cone,
    # #      # x_fwb      <= x_wing[i] + 0.5 * c0 * r_wc,
    # #      # x_wing[i]   <= x_b[i] + 0.5 * c0 * r_wc,
    S_nose**(8/5) == (2 * pi * R_fuse**2)**(8/5) * ((1/3) + (2/3) * (l_nose / R_fuse)**(8/5)),
    S_bulk       == 2 * pi * R_fuse**2,
    V_cyl        == A_skin * l_shell,
    V_nose       == S_nose * t_skin,
    V_bulk       == S_bulk * t_skin,
    V_cabin      == A_fuse * ((2/3) * l_nose + l_shell + (2/3) * R_fuse), # Converted from inequality to equality
]

# Fuselage Weight
Constraints += [
    W_fuse >= W_apu + W_cone + W_floor + W_insul + W_padd + W_seats + W_shell + W_window + W_fix, # + W_hbend[i] + W_vbend[i],
    W_skin >= rho_skin * g * (V_bulk + V_cyl + V_nose),
    W_shell >= W_skin * (1 + f_fadd + f_frame + f_string),
    V_floor >= A_floor * wdt_floor,
    W_floor >= V_floor * rho_floor * g + d2W_floor * l_floor * wdt_floor,
    W_cone >= rho_cone * g * V_cone * (1 + f_fadd + f_frame + f_string),

```

```

    ## W_hbend[i] >= rho_bend * g * V_hbend[i],
    ## ## V_hbend[i] >= 1,
    ## W_vbend[i] >= rho_bend * g * V_vbend[i],
    ## ## V_vbend[i] >= 1,
    W_window >= dW_window * l_shell, # converted from equality to inequality
    W_insul >= d2W_insul * (0.55 * (S_bulk + S_nose) + 1.1 * pi * R_fuse * l_shell),
    W_apu   >= W_pay * f_apu,
    W_padd  >= W_pay * f_padd,
    W_seats >= W_seat * n_seats,
    rho_cabin == p_cabin / (R_sheat * T_cabin),
    W_buoy  == rho_cabin * g * V_cabin,
    W_pay   >= W_avgpass * n_pass,
]

# Pressure Loading
Constraints += [
    sigma_x      == 0.5 * delt_Pover * (R_fuse / t_shell),
    sigma_thta   == delt_Pover * (R_fuse / t_skin),
    sigma_skin   >= sigma_x,
    sigma_skin   >= sigma_thta,
]

# Floor Loading
Constraints += [
    P_floor      == N_land * (W_pay + W_seats), # Converted from inequality to equality
    S_floor      == P_floor / 2,
    M_floor      == (1/8) * P_floor * wdt_floor,
    A_floor      == 1.5 * (S_floor / tau_floor) + 2 * (M_floor / (sigma_floor * h_floor)), # Converted from inequality to equality
]

# Tail Cone Sizing
BigQvt = f.Variable(name='BigQvt', guess=1000, units='N*m', description='Fuselage bending moment at the vertical tail root')
lambda_cone = f.Constant(name='lambda_cone', value=0.1, units='-', description='Fuselage tail cone taper ratio')
t_cone = f.Variable(name='t_cone', guess=0.015, units='m', description='Fuselage tail cone skin thickness')
tau_cone = f.Variable(name='tau_cone', guess=0.07, units='-', description='Fuselage tail cone thickness to chord ratio')

Constraints += [
    BigQvt * 3 == L_vtmax[0] * b_vt * (p_vt/q_vtd), # rewritten form of (154)
    t_cone == BigQvt / (2 * A_fuse * tau_cone),
    R_fuse * tau_cone * (1 + p_vt) * V_cone * ((1 + lambda_cone)/(4 * l_cone)) >= L_vtmax[0] * b_vt * (p_vt/3),
    tau_cone == sigma_skin,
    lambda_cone == R_fuse / l_cone, # l_cone is currently constant. updated 10/10/25
    # Maybe i need to make lambda_cone a constant and l_cone into a variable?
    # Maybe convert BigQvt into a constant?
]

# Fuselage Area Moment of Inertia
Constraints += [
    t_shell      == t_skin * (1 + f_string * (rho_skin / rho_bend)), # converted from inequality to equality
    sigma_bend   >= sigma_Mh + ((delt_Pover * R_fuse) / (2 * t_shell)),
    sigma_bend   >= sigma_Mv + ((delt_Pover * R_fuse) / (2 * t_shell)),
    I_hshell     <= pi * t_shell * R_fuse**3,
    I_vshell     <= pi * t_shell * R_fuse**3,
]

# Fuselage Drag
for i in [0,1,2]:
    Constraints += [
        D_fuse[i] >= 0.5 * rho_free * C_Dfuse[i] * V_free[i]**2 * (l_fuse * R_fuse * (M[i]**2 / M_fuseD**2)),
        # D_fuse[i] <= 9000,
    ]

# Horizontal Tail Geometry and Structure
Constraints += [
    S_ht         == V_ht * S_w * cbar_ht / l_ht, # Added to properly size HT
    S_ht         == b_ht * ((c_rootht + c_tiptht) / 2),

```

```

        cbar_ht      == (2/3) * ((1 + lambda_ht + lambda_ht**2) / q_htd) * c_rootht,
        q_htd == 1 + lambda_ht,
        y_cbarht     == (b_ht * q_htd) / (3 * p_ht),
        p_ht == 1 + 2* lambda_ht,
        lambda_ht     == c_tipht / c_rootht,
        lambda_ht >= lambda_htmin,
        AR_ht        == (b_ht**2) / S_ht,
        e * (1 + (AR_ht * f_lambdaht)) <= 1,
        f_lambdaht >= 0.0524 * lambda_ht**4 - 0.15 * lambda_ht**3 + 0.1659 * lambda_ht**2 - 0.0706 * lambda_ht + 0.0119,
    ]

# Trim Condition (with corrected Lift curve slopes)
Constraints += [
    (x_wing / cbar_w) <= (x_CG / cbar_w) + (cbar_ht / C_Lw[0]) + ((V_ht * C_Lht[0]) / C_Lw[0]),
    # C_Lht == C_alphaht[0] * alpha_ht,
    # C_Lalphaht[i] + ((2 * C_Lalphaw[i]) / (pi * AR_w)) * eta_ht * C_Lalphaht[0] <= C_Lalphaht[0] * eta_ht,
]

# Minimum Stability Margin
Constraints += [
    SM_min + (delt_xCG / cbar_w) + (cbar_ht / c_Lwmax) <= V_ht * m_ratio + ((V_ht * c_Lhtmax) / c_Lwmax),
    m_ratio * (1 + (2/AR_w)) == 1 + (2 / AR_ht),
]

# Stability Margin
Constraints += [
    SM <= (x_wing - x_CG) / cbar_w,
    SM >= SM_min,
]

# HT Position and Lift?
Constraints += [
    l_ht <= delt_xlht + 0.25 * cbar_ht,
    delt_xtht >= delt_xlht + c_rootht,
    l_fuse >= x_CG + delt_xtht,
    x_CGht >= x_CG + 0.5 * (delt_xlht + delt_xtht),
    # Converted into equality constraints. Not sure if this is a correct assumption.
]

for i in [0,1,2]:
    Constraints += [
        L_htmax[i] == 0.5 * rho_TO * S_ht * c_Lhtmax * V_ne**2,
        L_ht[i] * l_ht >= D_wm[i] * y_eng + T_TO * y_eng,
        L_ht[i] == 0.5 * rho_free * S_ht * C_Lht[i] * V_free[i]**2,
    ]

# Horizontal Tail Weight
Constraints += [
    W_ht >= W_structht * (1 + f_flap + f_ribs + f_watt), # Need to update with correct terms, elevator, etc.
    W_structht >= W_capht + W_webht,
    W_capht >= (8 * rho_cap * g * r_htc * t_capht * vht * S_ht**1.5) / (3 * AR_ht**0.5),
    W_webht >= (8 * rho_web * g * r_hht * tau_ht * t_webht * vht * S_ht**1.5) / (3 * AR_ht**0.5),
    vht**3.94 >= (0.14 * p_ht**0.54) + (0.84 / p_ht**2.4),
    # 2 * q_htd >= 1 + p_ht,
    0.4232 * tau_ht**2 * t_capht * r_htc >= 0.92 * tau_ht * t_capht**2 * r_htc + I_capht, # What is tau_rwc?
    8 >= (AR_ht * M_rht * N_lift * tau_ht * q_htd**2) / (I_capht * S_ht * sigma_max),
    12 >= (AR_ht * L_htmax[0] * N_lift * q_htd**2) / (S_ht * sigma_maxsh * tau_ht * t_webht),
    tau_ht <= 0.14, # Check after to remove
    M_rht >= (AR_ht * L_htmax[0] * p_ht) / 24, # use this one from hoburg
    # # M_rht[i] * c_rootht >= (L_htmax[i]) * ((b_ht / 12) * (c_rootht + 2 * c_tipht)) * (b_ht / S_ht), # No engines on HT. No fuel storage on HT
]

# Horizontal Tail Drag
for i in [0,1,2]:
    Constraints += [
        D_ht[i] >= 0.5 * rho_free * S_ht * C_Dht[i] * V_free[i]**2,
        C_Dht[i] >= C_D0ht[i] + C_Diht[i],
    ]

```

```

C_D0ht[i]**6.49    >= 5.288e-20 * Re_ht[i]**0.901 * tau_ht**0.912 * M[i]**8.645 +
                  1.676e-28 * Re_ht[i]**0.351 * tau_ht**6.292 * M[i]**10.256 +
                  7.098e-25 * Re_ht[i]**1.395 * tau_ht**1.962 * M[i]**0.567 +
                  3.731e-14 * Re_ht[i]**-2.574 * tau_ht**3.128 * M[i]**0.448 +
                  1.443e-12 * Re_ht[i]**-3.91 * tau_ht**4.663 * M[i]**7.689, # copy over from other file
Re_ht[i]           == (rho_free * V_free[i] * cbar_ht) / mu,
C_Diht[i]          >= f_tip * (C_Lht[i]**2 / (pi * e * AR_ht)),
# e * (1 + (AR_ht * f_lambdaht[i]))    <= 1, # moved to geometry
# f_lambdaht[i]    >= 0.0524 * lambda_ht**4 - 0.15 * lambda_ht**3 + 0.1659 * lambda_ht**2 - 0.0706 * lambda_ht + 0.0119, # moved to geometry
]

# =====
# Code for displaying
# =====

f.ConstraintList(Constraints)
var_list = f.get_variables()
con_list = f.get_constants()

variable_names = []
variable_values = []
variable_units = []
variable_description = []

constant_names = []
constant_values = []
constant_units = []
constant_description = []

ctr = 0

from solver import cvxopt_solve
res = cvxopt_solve(f)
# print(res)

import pandas as pd
for var in var_list:
    if isinstance(var, pyomo.core.base.var.IndexedVar):
        for ix in var.index_set():
            lbls = ['out', 'ret', 'sprint']
            variable_names.append(f'{var.name} [{lbls[ix]}]')
            variable_values.append(res['x'][ctr])
            variable_units.append(var._units)
            variable_description.append(var.doc)
            ctr += 1
    else:
        variable_names.append(var.name)
        variable_values.append(res['x'][ctr])
        variable_units.append(var._units)
        variable_description.append(var.doc)
        ctr += 1

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4f}'.format
vdf = pd.DataFrame({
    'Variable Name': variable_names,
    'Value': variable_values,
    'Units': variable_units,
    'Description': variable_description
})

print(vdf)

ctr2 = 0

for con in con_list:

```

```

        constant_names.append(con.name)
        constant_values.append(con.values)
        constant_units.append(con._units)
        constant_description.append(con.doc)
        ctr += 1

pd.set_option('display.max_rows', None)
pd.options.display.float_format = '{:.4}'.format
cdf = pd.DataFrame({
    'Constant Name': constant_names,
    'Value': constant_values,
    'Units': constant_units,
    'Description': constant_description
})

print(cdf)

# Save into separate txt file
with open("variables.txt", "w") as f:
    f.write(vdf.to_string())

with open("constants.txt", "w") as f:
    f.write(cdf.to_string())

```

APPENDIX C
VARIABLES AND CONSTANTS

```

# # # Flight Performance Variables
a = f.Constant(name='a', value=297, units='m/s', description='Speed of sound') # Assumed constant at altitude
C_D = f.Variable(name='C_D', guess=0.02911, units='-', size=3, description='Drag Coefficient')
D = f.Variable(name='D', guess=34241, units='N', size=3, description='Total aircraft drag (cruise)')
L_D = f.Variable(name='L_D', guess=19.29, units='-', size=3, description='Lift/drag ratio')
M = f.Constant(name='M', value=0.8, units='-', size=3, description='Cruise mach number')
R = f.Variable(name='R', guess=50000, units='m', size=3, description='Segment range')
t = f.Variable(name='t', guess=5000, units='s', size=3, description='Flight time')
V_free = f.Variable(name='V_free', guess=231.7, units='m/s', size=3, description='Freestream Velocity')
V_TO = f.Variable(name='V_TO', guess=73.38, units='m/s', size=3, description='Takeoff Velocity')
W_avg = f.Variable(name='W_avg', guess=50000, units='N', size=3, description='Average aircraft weight during flight segment') # Changed from lbf
W_buoy = f.Variable(name='W_buoy', guess=1517, units='N', size=3, description='Buoyancy weight')
W_dry = f.Variable(name='W_dry', guess=406261.53, units='N', size=3, description='Aircraft dry weight') # Changed from lbf
W_end = f.Variable(name='W_end', guess=1000, units='N', size=3, description='Aircraft weight at end-of-flight segment')
W_fuel = f.Variable(name='W_fuel', guess=50000, units='N', size=3, description='Weight of fuel burned per flight segment') # Changed from lbf
W_fuelprim = f.Variable(name='W_fuelprim', guess=5.164e4, units='N', size=3, description='Primary fuel weight (excludes reserves)')
W_lg = f.Variable(name='W_lg', value=14696.92, units='N', size=3, description='Landing gear weight') # Assumed from landing gear model
W_max = f.Variable(name='W_max', guess=500000, units='N', size=3, description='Maximum aircraft weight') # Changed from lbf
W_misc = f.Variable(name='W_misc', value=102340.356, units='N', size=3, description='Miscellaneous system weight') # Changed from lbf, assumed from paper
W_start = f.Variable(name='W_start', guess=5.162e5, units='N', size=3, description='Aircraft weight at start of flight segment') # Changed from lbf
x_TO = f.Variable(name='x_TO', guess=1524, units='m', size=3, description='Takeoff distance')
y = f.Variable(name='y', guess=0.1318, units='-', size=3, description='Takeoff parameter')
z_bre = f.Variable(name='z_bre', guess=0.1036, units='-', size=3, description='Breguet parameter')
zeta = f.Variable(name='zeta', guess=0.07295, units='-', size=3, description='Takeoff parameter')

c_Lwmax = f.Constant(name='c_Lwmax', value=2.5, units='-', description='Maximum lift coefficient, wing')
c_T = f.Constant(name='c_T', value=0.00008333, units='1/s', description='thrust specific fuel consumption') # Changed from 0.3 1/h
f_fuelres = f.Constant(name='f_fuelres', value=0.1, units='-', description='Fuel reserve fraction')
g = f.Constant(name='g', value=9.81, units='m/s^2', description='Gravitational acceleration')
l_r = f.Constant(name='l_r', value=2134, units='m', description='Maximum runway length') # Changed from ft
M_min = f.Constant(name='M_min', value=0.7, units='-', description='Minimum mach number')
n_eng = f.Constant(name='n_eng', value=2, units='-', description='Number of engines')
R_req = f.Constant(name='R_req', value=5.56e6, units='m', description='Required total range')
rho_TO = f.Constant(name='rho_TO', value=1.225, units='kg/m^3', description='Takeoff density')
T = f.Variable(name='T', guess=1.29e5, units='N', size=3, description='Thrust per engine in cruise')
T_TO = f.Variable(name='T_TO', value=1.29e5, units='N', size=3, description='Thrust per engine at takeoff')
W_eng = f.Constant(name='W_eng', value=1e4, units='N', description='Engine weight')

# System Level Variables
f_fuel = f.Variable(name='f_fuel', guess=1, units='-', size=3, description='Percent fuel remaining')
I_z = f.Variable(name='I_z', guess=1000, units='kg*m^2', size=3, description='Total aircraft moment of inertia')
I_zfuse = f.Variable(name='I_zfuse', guess=1000, units='kg*m^2', size=3, description='Fuselage moment of inertia')
I_ztail = f.Variable(name='I_ztail', guess=1000, units='kg*m^2', size=3, description='Tail moment of inertia')
I_zwing = f.Variable(name='I_zwing', guess=1000, units='kg*m^2', size=3, description='Wing moment of inertia')
# # W_hpesys = f.Variable(name='W_hpesys', guess=5000, units='N', size=3, description='Power system weight') # Changed from lbf
# # W_mg = f.Constant(name='W_mg', value=1.1e4, units='N', size=3, description='Main landing gear weight')
# # W_ng = f.Constant(name='W_ng', value=1113, units='N', size=3, description='Nose landing gear weight')
x_b = f.Variable(name='x_b', guess=1000, units='m', size=3, description='Wing box forward bulkhead location') # Removed index, assumed constant
x_CGlg = f.Constant(name='x_CGlg', value=19.3, units='m', size=3, description='x location of landing gear c.g.') # Removed index, assumed constant
x_CGmisc = f.Constant(name='x_CGmisc', value=10, units='m', size=3, description='x location of miscellaneous systems c.g.') # Removed index, assumed constant
# # x_hpesys = f.Variable(name='x_hpesys', guess=1000, units='m', size=3, description='Power systems centroid') # Removed index, assumed constant
# # x_mg = f.Variable(name='x_mg', guess=20.75, units='m', size=3, description='Main landing gear centroid') # Removed index, assumed constant
# # x_ng = f.Variable(name='x_ng', guess=5, units='m', size=3, description='Nose landing gear centroid') # Removed index, assumed constant
x_wing = f.Constant(name='x_wing', value=19.6, units='m', size=3, description='Wing centroid') # Removed index, assumed constant, may need to revert back to variable

# Wing Model Variables and Constants
# Geometry Variables
b_w = f.Variable(name='b_w', guess=30.19, units='m', size=3, description='wingspan')
b_wmax = f.Constant(name='b_wmax', value=35.9, units='m', size=3, description='maximum allowed wingspan')
cbar_w = f.Variable(name='cbar_w', guess=3.728, units='m', size=3, description='mean aerodynamic chord(wing)') # not sure why it is constant
c_rootw = f.Variable(name='c_rootw', guess=5.411, units='m', size=3, description='Wing root chord')
c_tipw = f.Variable(name='c_tipw', guess=1.082, units='m', size=3, description='Wing tip chord')
lambda_w = f.Variable(name='lambda_w', guess=0.2, units='-', size=3, description='wing taper ratio')
lambda_wmin = f.Constant(name='lambda_wmin', value=0.2, units='-', size=3, description='Minimum wing taper ratio')
S_w = f.Variable(name='S_w', guess=98, units='m^2', size=3, description='Wing area')

```



```

y_cbarw = f.Constant(name='y_cbarw', value=8.624, units='m', description='Spanwise location of mean aerodynamic chord') # not sure why it is constant

# Wing Lift Variables
C_Lw = f.Variable(name='C_Lw', guess=0.5616, units='-', size=3, description='Lift Coefficient, wing')
f_Ltotw = f.Constant(name='f_Ltotw', value=1.2, units='-', description='Total lift divided by wing lift')
L_total = f.Variable(name='L_total', guess=6.399e5, units='N', size=3, description='Total Lift generated by aircraft')
L_w = f.Variable(name='L_w', guess=5.612e5, units='N', size=3, description='Wing Lift')
# W_s = f.Variable(name='W_s', guess=15, units='N/m^2', size=3, description='Wing Loading')
# W_smax = f.Constant(name='W_smax', value=6222, units='N/m^2', description='Maximum Wing loading')

# Wing Weight Variables
AR_w = f.Variable(name='AR_w', guess=9.298, units='-', description='Wing Aspect Ratio') # From Geometry model, converted from constant to variable
f_aileron = f.Constant(name='f_aileron', value=0.005, units='-', description='Aileron Added weight fraction')
f_flap = f.Constant(name='f_flap', value=0.03, units='-', description='Flap Added weight fraction')
f_fuelw = f.Constant(name='f_fuelw', value=0.5, units='-', description='Fraction of total fuel stored in wing')
f_lete = f.Constant(name='f_lete', value=0.05, units='-', description='Lete added weight fraction')
f_ribs = f.Constant(name='f_ribs', value=0.02, units='-', description='Wing rib added weight fraction')
f_slat = f.Constant(name='f_slat', value=0.02, units='-', description='Slat added weight fraction')
f_spoiler = f.Constant(name='f_spoiler', value=0.01, units='-', description='Spoiler added weight fraction')
f_watt = f.Constant(name='f_watt', value=0.0001, units='-', description='Wing attachment hardware added weight fraction')
I_cap = f.Variable(name='I_cap', guess=3.458e-5, units='-', description='Nondimensional spar cap area moment of inertia')
L_wmax = f.Constant(name='L_wmax', value=3.112e6, units='N', description='Maximum lift generated by wing') # From Wing Lift model
M_r = f.Variable(name='M_r', guess=1.688e6, units='N', description='Root moment per root chord')
N_lift = f.Constant(name='N_lift', value=2, units='-', description='Wing loading multiplier')
p_w = f.Variable(name='p_w', guess=1.4, units='-', description='Dummy variable (1+2*lambda_w)')
q_w = f.Variable(name='q_w', guess=1.2, units='-', description='Dummy Variable (1+lambda_w)')
r_h = f.Constant(name='r_h', value=0.75, units='-', description='Fractional Wing thickness at spar web')
r_wc = f.Constant(name='r_wc', value=0.5, units='-', description='Wing box width-to-chord ratio') # within 0.5 - 0.6, dont go too much higher
rho_cap = f.Constant(name='rho_cap', value=2700, units='kg/m^3', description='Density of spar cap material')
rho_web = f.Constant(name='rho_web', value=2700, units='kg/m^3', description='Density of shear web material')
sigma_max = f.Constant(name='sigma_max', value=2.5e8, units='Pa', description='Allowable tensile stress')
sigma_maxsh = f.Constant(name='sigma_maxsh', value=1.67e8, units='Pa', description='Allowable shear stress')
tau_w = f.Variable(name='tau_w', guess=0.14, units='-', description='wing thickness/chord ratio')
t_cap = f.Variable(name='t_cap', guess=0.008251, units='-', description='Nondimensional spar cap thickness')
t_web = f.Variable(name='t_web', guess=0.002828, units='-', description='Nondimensional shear web thickness')
W_cap = f.Variable(name='W_cap', guess=7.984e4, units='N', description='Weight of Spar caps')
W_fueltotal = f.Constant(name='W_fueltotal', value=5.164e4, units='N', description='Total fuel weight')
W_structw = f.Variable(name='W_structw', guess=3927, units='N', description='Wing box weight')
W_web = f.Variable(name='W_web', guess=6158, units='N', description='Weight of shear web')
W_wing = f.Variable(name='W_wing', guess=1.204e5, units='N', description='Wing Weight')
v = f.Variable(name='v', guess=0.8612, units='-', description='Dummy variable ((t^2+t+1)/(t+1)^2)')
y_eng = f.Constant(name='y_eng', value=4.83, units='m', description='Engine moment arm')

# Wing drag Variables
C_Dw = f.Variable(name='C_Dw', guess=0.01677, units='-', size=3, description='Drag coefficient,wing')
C_Diw = f.Variable(name='C_Diw', guess=0.005, units='-', size=3, description='Wing Induced Drag Coefficient')
C_Dpw = f.Variable(name='C_Dpw', guess=0.005637, units='-', size=3, description='Wing Parasitic Drag Coefficient')
cos_chord = f.Constant(name='cos_chord', value=0.866, units='-', description='Cosine of quarter-chord sweep angle')
D_w = f.Variable(name='D_w', guess=17344, units='N', size=3, description='Wing Drag')
f_lambda_w = f.Variable(name='f_lambda_w', guess=0.0033, units='-', description='Empirical efficiency function of taper')
f_tip = f.Constant(name='f_tip', value=0.065, units='-', description='Induced drag reduction from wing-tip devices')
e = f.Constant(name='e', value=0.9702, units='-', description='Oswald efficiency factor') # Assumed constant
mu = f.Constant(name='mu', value=1.4e-5, units='N*s/m^2', description='Dynamic Viscosity')
pi = np.pi
Re_w = f.Variable(name='Re_w', guess=2.344e7, units='-', size=3, description='Cruise Reynolds number (wing)')
rho_free = f.Constant(name='rho_free', value=0.38, units='kg/m^3', description='Freestream density') # Removed index, Assumed Constant at altitude

# Wing aerodynamic center variables
delt_xacw = f.Variable(name='delt_xacw', guess=50, units='m', description='Wing aerodynamic center shift')
tan_chord = f.Constant(name='tan_chord', value=0.5774, units='-', description='Tangent of quarter-chord sweep angle')

# Fuel Volume Variables
f_fueluse = f.Constant(name='f_fueluse', value=0.9, units='-', description='Usability factor of maximum fuel volume')
rho_fuel = f.Constant(name='rho_fuel', value=817, units='kg/m^3', description='Density of fuel')
V_fuelmax = f.Variable(name='V_fuelmax', guess=93.38, units='m^3', description='Available Fuel Volume')
W_fuelwing = f.Variable(name='W_fuelwing', guess=5000, units='N', description='Maximum fuel weight carried in wing') # Changed from lbf

```

```

# Vertical Tail Variables and Constants
# Geometrical Position Variables
c_Lvtmax = f.Constant(name='c_Lvtmax', value=2.6, units='-', description='Maximum lift coefficient')
delt_xlvt = f.Variable(name='delt_xlvt', guess=25.73, units='m', description='Distance from c.g. to VT leading edge')
delt_xtvt = f.Variable(name='delt_xtvt', guess=34.88, units='m', description='Distance from c.g. to VT trailing edge')
l_fuse = f.Variable(name='l_fuse', guess=52.47, units='m', description='Fuselage length') # Constant pulled from fuselage model?
l_vt = f.Variable(name='l_vt', guess=28.2, units='m', description='Vertical tail moment arm')
tan_chdvt = f.Constant(name='tan_chdvt', value=0.8391, units='', description='Tangent of leading edge sweep (40 deg)')
V_ne = f.Constant(name='V_ne', value=281.26, units='m/s', description='Never exceed velocity') # changed from 144 m/s
x_CG = f.Variable(name='x_CG', guess=17.6, units='m', description='x location of c.g.') # Constant pulled from performance model?
x_CGvt = f.Variable(name='x_CGvt', guess=47.9, units='m', description='x location of vertical tail c.g.')
z_cbarvt = f.Constant(name='z_cbarvt', value=1.004, units='m', description='Vertical location of mean aerodynamic chord') # assume for now

# Geometrical Variables
b_vt = f.Variable(name='b_vt', guess=3.707, units='m', description='Vertical tail full-span')
cbar_vt = f.Variable(name='cbar_vt', guess=6.519, units='m', description='Vertical tail mean aerochord') # assume for now
c_rootvt = f.Variable(name='c_rootvt', guess=9.146, units='m', description='Vertical tail root chord')
c_tipvt = f.Variable(name='c_tipvt', guess=2.744, units='m', description='Vertical tail tip chord')
lambda_vt = f.Variable(name='lambda_vt', guess=0.3, units='-', description='Span and Taper ratio')
lambda_vtmin = f.Constant(name='lambda_vtmin', value=0.3, units='-', description='Minimum vertical tail taper ratio')
p_vt = f.Variable(name='p_vt', guess=1.6, units='-', description='Dummy variable (1+2*lambda_dvt)')
q_vtd = f.Variable(name='q_vtd', guess=1.3, units='-', description='Dummy Variable (1+lambda_dvt)')
S_vt = f.Variable(name='S_vt', guess=19.38, units='m^2', description='Vertical tail reference area (half)')
y_cbarvt = f.Variable(name='y_cbarvt', guess=4.5, units='m', description='Spanwise location of vertical tail mean aerodynamic chord') # assume for now

# VT Weight Variables
AR_vt = f.Variable(name='AR_vt', guess=4, units='-', description='Vertical tail aspect ratio') # from geometry model
I_capvt = f.Variable(name='I_capvt', guess=3.458e-5, units='-', description='Nondimensional vertical tail spar cap area moment of inertia')
L_vtmax = f.Constant(name='L_vtmax', value=1.28e6, units='N', size=3, description='Maximum load for structural sizing') # from lift model
M_rvt = f.Variable(name='M_rvt', guess=1.688e6, units='N', description='Vertical tail root moment per root chord')
r_vth = f.Variable(name='r_vth', value=0.75, units='-', description='Fractional vertical tail thickness at spar web')
r_vtwc = f.Variable(name='r_vtwc', value=0.32, units='-', description='Vertical tail box width-to-chord ratio')
tau_vt = f.Variable(name='tau_vt', guess=0.09958, units='-', description='Vertical tail thickness/chord ratio')
t_capvt = f.Variable(name='t_capvt', guess=0.008251, units='-', description='Nondimensional spar cap thickness')
t_webvt = f.Variable(name='t_webvt', guess=0.002828, units='-', description='Nondimensional shear web thickness')
W_capvt = f.Variable(name='W_capvt', guess=1000, units='N', description='Weight of vertical tail spar caps')
W_webvt = f.Variable(name='W_webvt', guess=1000, units='N', description='Weight of vertical tail shear web')
W_structvt = f.Variable(name='W_structvt', guess=1.2e4, units='N', description='Vertical tail box weight')
W_vt = f.Variable(name='W_vt', guess=1963, units='N', description='Vertical tail weight')
vvt = f.Variable(name='vvt', guess=0.8612, units='-', description='Dummy variable ((t^2+t+1)/(t+1)^2)')

# # Engine Out Condition Variables
A_fan = f.Constant(name='A_fan', value=2.41, units='m^2', description='Engine reference area')
c_Dwm = f.Constant(name='c_Dwm', value=0.5, units='-', description='Windmill drag coefficient')
c_lvt_EO = f.Constant(name='c_lvt_EO', value=1, units='-', description='Sectional lift force coefficient (engine out)')
C_LvtEO = f.Variable(name='C_LvtEO', guess=0.3905, units='-', size=3, description='Vertical tail lift coefficient during engine out')
D_wm = f.Variable(name='D_wm', guess=3609, units='N', size=3, description='Engine out windmill drag')
L_vtEO = f.Variable(name='L_vtEO', guess=2.271e4, units='N', size=3, description='Vertical tail lift in engine out')
V1 = f.Constant(name='V1', value=70, units='m/s', description='Minimum takeoff velocity')

# Crosswind Variables
C_Lvtland = f.Variable(name='C_Lvtland', guess=3, units='-', size=3, description='Vertical tail lift coefficient during landing')
rdot_req = f.Constant(name='rdot_req', value=2, units='s^-2', description='Maximum required yaw rate acceleration at landing')
V_land = f.Constant(name='V_land', value=65, units='m/s', description='Landing velocity')

# Vertical Tail Drag Variables
C_Dpvt = f.Variable(name='C_Dpvt', guess=0.004781, units='-', size=3, description='Viscous drag coefficient')
D_vt = f.Variable(name='D_vt', guess=6548, units='N', size=3, description='Vertical tail drag')
Re_vt = f.Variable(name='Re_vt', guess=4.099e7, units='-', size=3, description='Vertical tail Reynolds number')

# Fuselage Model Variables and Constants
# Geometry Constraints
A_fuse = f.Variable(name='A_fuse', guess=10.81, units='m^2', description='Fuselage cross-sectional area')
A_skin = f.Variable(name='A_skin', guess=0.01087, units='m^2', description='Skin cross-sectional area')
n_spr = f.Constant(name='n_spr', value=6, units='-', description='Number of seats per row')

```

```

R_fuse      = f.Variable(name='R_fuse',      guess=1.855,    units='m',      description='Fuselage radius')
t_skin      = f.Variable(name='t_skin',      guess=0.0009324, units='m',      description='Skin thickness')
wdt_fuse    = f.Variable(name='wdt_fuse',    guess=3.71,    units='m',      description='Fuselage width')
w_aisle     = f.Constant(name='w_aisle',    value=0.51,    units='m',      description='Aisle width')
w_seatwdth  = f.Constant(name='w_seatwdth',  value=0.5,     units='m',      description='Seat width')
w_sys       = f.Constant(name='w_sys',      value=0.1,     units='m',      description='Width between cabin and skin for systems')

# Shell Geometry Variables
l_cone      = f.Variable(name='l_cone',      guess=22.86,    units='m',      description='Cone length') # from cone model?
l_floor     = f.Variable(name='l_floor',      guess=28.12,    units='m',      description='Floor length')
l_nose      = f.Constant(name='l_nose',      value=5.2,     units='m',      description='Noselength')
l_shell     = f.Variable(name='l_shell',      guess=24.41,    units='m',      description='Shell length')
n_pass      = f.Variable(name='n_pass',      guess=167,     units='-',      description='Number of passengers')
n_rows      = f.Variable(name='n_rows',      guess=31,      units='-',      description='Number of rows')
n_seats     = f.Constant(name='n_seats',     value=186,     units='-',      description='Number of seats')
p_s         = f.Constant(name='p_s',         value=0.787,    units='m',      description='Seat pitch')
# # r_wc     = f.Constant(name='r_wc',         value=0.25,    units='-',      description='Wing box width-to-chord ratio')
S_bulk      = f.Variable(name='S_bulk',      guess=21.62,    units='m^2',    description='Bulkhead surface area')
S_nose      = f.Variable(name='S_nose',      guess=49.82,    units='m^2',    description='Nose surface area')
V_bulk      = f.Variable(name='V_bulk',      guess=0.02016, units='m^3',    description='Bulkhead skin volume')
V_cyl       = f.Variable(name='V_cyl',      guess=0.2653,  units='m^3',    description='Cylinder skin volume')
V_cabin     = f.Variable(name='V_cabin',      guess=315,     units='m^3',    description='Cabin Volume')
V_nose      = f.Variable(name='V_nose',      guess=0.04645, units='m^3',    description='Nose skin volume')
# # x_b      = f.Variable(name='x_b',         guess=1000,    units='m',      size=3, description='Wing box forward bulkhead location')
# # x_fwb    = f.Variable(name='x_fwb',      guess=10,      units='m',      description='x location of front of wing box')
x_shell1    = f.Variable(name='x_shell1',    guess=5.2,     units='m',      description='Start of cylinder section')
x_shell2    = f.Variable(name='x_shell2',    guess=29.61,   units='m',      description='End of cylinder section')

# # Weight Variables
A_floor     = f.Variable(name='A_floor',      guess=0.05549, units='m^2',    description='Floor beam cross-sectional area') # from loading model
d2W_floor   = f.Constant(name='d2W_floor',   value=60,      units='N/m^2',  description='Floor weight per unit area')
d2W_insul   = f.Constant(name='d2W_insul',   value=22,      units='N/m^2',  description='Insulation material weight per unit area')
dW_window   = f.Constant(name='dW_window',   value=435,     units='N/m',    description='Window weight per unit length')
f_apu       = f.Constant(name='f_apu',       value=0.035,   units='-',      description='APU weight as fraction of payload weight')
f_fadd      = f.Constant(name='f_fadd',      value=0.2,     units='-',      description='Fractional add weight of local reinforcements')
f_frame     = f.Constant(name='f_frame',     value=0.25,    units='-',      description='Fractional frame weight')
f_padd      = f.Constant(name='f_padd',      value=0.4,     units='-',      description='Other misc weight as fraction of payload weight')
f_stringer  = f.Constant(name='f_stringer',  value=0.35,    units='-',      description='Fractional stringer weight')
p_cabin     = f.Constant(name='p_cabin',     value=7.5e4,   units='N/m^2',  description='Cabin Air Pressure')
R_sheat     = f.Constant(name='R_sheat',     value=287,     units='J/kg*K', description='Air specific heat')
rho_cabin   = f.Variable(name='rho_cabin',    guess=1.225,   units='kg/m^3', description='Cabin air density')
rho_cone    = f.Constant(name='rho_cone',    value=2700,    units='kg/m^3', description='Cone material density')
rho_floor   = f.Constant(name='rho_floor',   value=2700,    units='kg/m^3', description='Floor material density')
rho_skin    = f.Constant(name='rho_skin',    value=2700,    units='kg/m^3', description='Skin density')
T_cabin     = f.Constant(name='T_cabin',     value=300,     units='K',      description='Cabin Air Temperature')
V_cone      = f.Variable(name='V_cone',      guess=0.1657,  units='m^3',    description='Cone skin Volume') # goes into negative for some reason
V_floor     = f.Variable(name='V_floor',      guess=0.1734,  units='m^3',    description='Floor Volume')
W_apu       = f.Variable(name='W_apu',       guess=5657,    units='N',      description='APU weight')
W_avgpass   = f.Constant(name='W_avgpass',   value=800,     units='N',      description='Average passenger weight, including luggage') # Converted from 180 lbf
W_cone      = f.Variable(name='W_cone',      guess=7898,    units='N',      description='Cone weight')
wdt_floor   = f.Constant(name='wdt_floor',   value=3.125,   units='m',      description='Floor half width') # from geometry model?
W_fix       = f.Constant(name='W_fix',       value=3000,    units='lbf',    description='Fixed weights (pilots,cockpit seats,navcom)')
W_floor     = f.Variable(name='W_floor',     guess=9865,    units='N',      description='Floor weight')
W_fuse      = f.Variable(name='W_fuse',      guess=1.616e5, units='N',      description='Fuselage weight')
# # # W_hbend = f.Variable(name='W_hbend',    guess=5000,    units='N',      size=3, description='Horizontal bending material weight') # Changed from lbf
W_insul     = f.Variable(name='W_insul',     guess=4307,    units='N',      description='Insulation material weight')
W_padd      = f.Variable(name='W_padd',      guess=6.465e4, units='N',      description='Miscellaneous weights (galley,toilets,doors,etc)')
W_pay       = f.Variable(name='W_pay',       guess=1.616e5, units='N',      description='Payload weight')
W_seat      = f.Constant(name='W_seat',     value=178,     units='N',      description='Weight per seat')
W_seats     = f.Variable(name='W_seats',     guess=2.79e4,  units='N',      description='Seating weight')
W_shell     = f.Variable(name='W_shell',     guess=1.582e4, units='N',      description='Shell weight')
W_skin      = f.Variable(name='W_skin',     guess=8791,    units='N',      description='Skin weight')
# # # W_vbend = f.Variable(name='W_vbend',    guess=5000,    units='N',      size=3, description='Vertical bending material weight') # Changed from lbf
W_window    = f.Variable(name='W_window',    guess=1.062e4, units='N',      description='Window weight')

# # Pressure Loading Variables

```

```

delt_Pover = f.Constant(name='delt_Pover', value=5.9e4, units='Pa', description='Cabin overpressure') # not entirely sure about this value
sigma_skin = f.Constant(name='sigma_skin', value=1.034e8, units='N/m^2', description='Maximum allowable skin stress')
sigma_thta = f.Variable(name='sigma_thta', guess=1.034e8, units='N/m^2', description='Skin hoop stress')
sigma_x = f.Variable(name='sigma_x', guess=3.831e7, units='N/m^2', description='Axial stress in skin')
t_shell = f.Variable(name='t_shell', guess=0.001259, units='m', description='Shell thickness')

# Floor Loading Variables
h_floor = f.Constant(name='h_floor', value=0.08359, units='m', description='Floor beam height')
M_floor = f.Variable(name='M_floor', guess=4.442e5, units='N*m', description='Maximum bending moment in floor beams')
N_land = f.Constant(name='N_land', value=6, units='-', description='Emergency lading load factor')
P_floor = f.Variable(name='P_floor', guess=1.137e6, units='N', description='Distributed floor load')
S_floor = f.Variable(name='S_floor', guess=5.686e5, units='N', description='Maximum shear in floor beams')
sigma_floor = f.Constant(name='sigma_floor', value=2.069e8, units='N/m^2', description='Maximum allowable floor stress')
tau_floor = f.Constant(name='tau_floor', value=2.069e8, units='N/m^2', description='Maximum allowable shear web stress')

# Fuselage Area Moment of Inertia Variables
I_hshell = f.Variable(name='I_hshell', guess=1000, units='m^4', description='Shell horizontal bending inertia')
I_vshell = f.Variable(name='I_vshell', guess=1000, units='m^4', description='Shell vertical bending inertia')
rho_bend = f.Constant(name='rho_bend', value=2700, units='kg/m^3', description='Stringer density')
sigma_bend = f.Variable(name='sigma_bend', value=1.85e6, units='N/m^2', description='Bending material stress')
sigma_Mh = f.Variable(name='sigma_Mh', guess=10, units='N/m^2', description='Horizontal bending material stress')
sigma_Mv = f.Variable(name='sigma_Mv', guess=10, units='N/m^2', description='Vertical bending material stress')

# # Drag Variables
C_Dfuse = f.Constant(name='C_Dfuse', value=0.1, units='-', size=3, description='Fuselage drag coefficient') # should be from TASOPT, from 0.001 to 0.1.
D_fuse = f.Variable(name='D_fuse', guess=8844, units='N', size=3, description='Fuselage Drag')
M_fuseD = f.Constant(name='M_fuseD', value=0.8, units='-', description='Fuselage drag reference Mach number')

# Horizontal Tail Variables and Constants
# Geometrical Variables
b_ht = f.Variable(name='b_ht', guess=10.68, units='m', description='Horizontal tail half-span')
cbar_ht = f.Variable(name='cbar_ht', guess=1.591, units='m', description='Horizontal tail mean aerodynamic chord')
c_rootht = f.Variable(name='c_rootht', guess=2.31, units='m', description='Horizontal tail root chord')
c_tipt = f.Variable(name='c_tipt', guess=0.462, units='m', description='Horizontal tail tip chord')
lambda_ht = f.Variable(name='lambda_ht', guess=0.2, units='-', description='Horizontal tail taper ratio')
lambda_htmin = f.Constant(name='lambda_htmin', value=0.2, units='-', description='Minimum horizontal tail taper ratio')
p_ht = f.Variable(name='p_ht', guess=1.4, units='N/m', description='Horizontal tail theoretical wing loading')
q_htd = f.Variable(name='q_htd', guess=1.2, units='-', description='Dummy Variable (1+lambda)')
S_ht = f.Variable(name='S_ht', guess=14.81, units='m^2', description='Horizontal tail cross-sectional area')
y_cbarht = f.Constant(name='y_cbarht', value=3.052, units='m', description='Spanwise location of horizontal tail mean aerodynamic chord')

# HT Position and Lift Variables?
C_Lht = f.Variable(name='C_Lht', guess=0.1785, units='-', size=3, description='Horizontal tail lift coefficient')
C_Lhtmax = f.Constant(name='C_Lhtmax', value=2.5, units='-', description='Maximum horizontal tail lift coefficient')
delt_xlht = f.Variable(name='delt_xlht', guess=32.57, units='m', description='Distance from c.g. to HT leading edge')
delt_xtht = f.Variable(name='delt_xtht', guess=34.88, units='m', description='Distance from c.g. to HT trailing edge')
l_ht = f.Variable(name='l_ht', guess=34.73, units='m', description='Horizontal tail moment arm')
L_ht = f.Variable(name='L_ht', guess=2.695e4, units='N', size=3, description='Horizontal tail downforce')
L_htmax = f.Variable(name='L_htmax', guess=4.702e5, units='N', size=3, description='Maximum horizontal tail downforce')
x_CGht = f.Variable(name='x_CGht', guess=51.32, units='m', description='x location of horizontal tail c.g.')

# # HT Weight Variables
AR_ht = f.Variable(name='AR_ht', guess=7.707, units='-', description='Horizontal tail aspect ratio') # from geometry model
I_capht = f.Variable(name='I_capht', guess=3.458e-5, units='-', description='Nondimensional horizontal tail spar cap area moment of inertia')
M_rht = f.Variable(name='M_rht', guess=1.688e6, units='N', description='Horizontal tail root moment per root chord')
r_htc = f.Constant(name='r_htc', value=0.40, units='-', description='Horizontal tail box width-to-chord ratio')
r_hht = f.Constant(name='r_hht', value=0.75, units='-', description='Fractional horizontal tail thickness at spar web')
tau_ht = f.Variable(name='tau_ht', guess=0.14, units='-', description='Horizontal tail thickness/chord ratio')
t_capht = f.Variable(name='t_capht', guess=0.008251, units='-', description='Nondimensional spar cap thickness')
t_webht = f.Variable(name='t_webht', guess=0.002828, units='-', description='Nondimensional shear web thickness')
W_capht = f.Variable(name='W_capht', guess=1000, units='N', description='Weight of horizontal tail spar caps')
W_webht = f.Variable(name='W_webht', guess=1000, units='N', description='Weight of horizontal tail shear web')
W_ht = f.Variable(name='W_ht', guess=5196, units='N', description='Horizontal tail weight')
W_structht = f.Variable(name='W_structht', guess=1.2e4, units='N', description='Horizontal tail box weight')
vht = f.Variable(name='vht', guess=0.8612, units='-', description='Dummy variable ((t^2+t+1)/(t+1)^2)')

```

```

# HT Drag Variables
C_Dht = f.Variable(name='C_Dht', guess=0.005, units='-', size=3, description='Horizontal tail drag coefficient')
C_D0ht = f.Variable(name='C_D0ht', guess=0.005348, units='-', size=3, description='Horizontal tail parasitic drag coefficient')
C_Diht = f.Variable(name='C_Diht', guess=0.005, units='-', size=3, description='Horizontal tail induced drag coefficient')
D_ht = f.Variable(name='D_ht', guess=1505, units='N', size=3, description='Horizontal tail drag')
f_lambdaht = f.Variable(name='f_lambdaht', guess=0.0524, units='-', description='Horizontal tail taper ratio function')
Re_ht = f.Variable(name='Re_ht', guess=1.001e7, units='-', size=3, description='Horizontal tail Reynolds number')

# Added Variables
# C_Dww = f.Variable(name='C_Dww', guess=0.03, units='-', size=3, description='Wave')
# c_mac = f.Constant(name='c_mac', value=0.1, units='-', description='Mean aerodynamic chord')
delt_xCG = f.Variable(name='delt_xCG', guess=0.01, units='m', description='Change in x location of c.g.')
eta_throttle = f.Constant(name='eta_throttle', value=0.3, units='-', description='Throttle setting at cruise')
M_crit = f.Constant(name='M_crit', value=0.5, units='-', description='Critical Mach Number')
m_ratio = f.Variable(name='m_ratio', guess=5, units='-', description='Ratio of HT and Wing Lift Curve Slopes')
SM = f.Variable(name='SM', guess=0.1, units='-', description='Stability Margin')
SM_min = f.Variable(name='SM_min', guess=0.05, units='-', description='Minimum Stability Margin')
V_ht = f.Constant(name='V_ht', value=0.8, units='-', description='Horizontal Tail Volume Coefficient')
V_vt = f.Constant(name='V_vt', value=0.065, units='-', description='Vertical Tail Volume Coefficient')

```

APPENDIX D

RESULTS

	Variable Name	Value	Units	Description
0	C_D[out]	0.13	dimensionless	Drag Coefficient
1	C_D[ret]	0.13	dimensionless	Drag Coefficient
2	C_D[sprint]	0.1299	dimensionless	Drag Coefficient
3	D[out]	8.003e+04	N	Total aircraft drag (cruise)
4	D[ret]	7.994e+04	N	Total aircraft drag (cruise)
5	D[sprint]	7.986e+04	N	Total aircraft drag (cruise)
6	L_D[out]	11.88	dimensionless	Lift/drag ratio
7	L_D[ret]	11.56	dimensionless	Lift/drag ratio
8	L_D[sprint]	11.23	dimensionless	Lift/drag ratio
9	R[out]	9.647e+05	m	Segment range
10	R[ret]	9.647e+05	m	Segment range
11	R[sprint]	9.647e+05	m	Segment range
12	t[out]	4.06e+03	s	Flight time
13	t[ret]	4.06e+03	s	Flight time
14	t[sprint]	4.06e+03	s	Flight time
15	V_free[out]	237.6	m/s	Freestream Velocity
16	V_free[ret]	237.6	m/s	Freestream Velocity
17	V_free[sprint]	237.6	m/s	Freestream Velocity
18	V_TO	74.93	m/s	Takeoff Velocity
19	W_avg[out]	9.51e+05	N	Average aircraft weight during flight segment
20	W_avg[ret]	9.24e+05	N	Average aircraft weight during flight segment
21	W_avg[sprint]	8.97e+05	N	Average aircraft weight during flight segment
22	W_buoy	2.688e+03	N	Buoyancy weight
23	W_dry	7.241e+05	N	Aircraft dry weight
24	W_end[out]	9.349e+05	N	Aircraft weight at end-of-flight segment
25	W_end[ret]	9.079e+05	N	Aircraft weight at end-of-flight segment
26	W_end[sprint]	8.81e+05	N	Aircraft weight at end-of-flight segment
27	W_fuel[out]	2.7e+04	N	Weight of fuel burned per flight segment
28	W_fuel[ret]	2.697e+04	N	Weight of fuel burned per flight segment
29	W_fuel[sprint]	2.694e+04	N	Weight of fuel burned per flight segment
30	W_fuelprim	8.091e+04	N	Primary fuel weight (excludes reserves)
31	W_max	9.619e+05	N	Maximum aircraft weight
32	W_start[out]	9.619e+05	N	Aircraft weight at start of flight segment
33	W_start[ret]	9.349e+05	N	Aircraft weight at start of flight segment
34	W_start[sprint]	9.079e+05	N	Aircraft weight at start of flight segment
35	x_TO	2.134e+03	m	Takeoff distance
36	y	6.771e-05	dimensionless	Takeoff parameter
37	z_bre[out]	0.02847	dimensionless	Breguet parameter
38	z_bre[ret]	0.02927	dimensionless	Breguet parameter
39	z_bre[sprint]	0.03012	dimensionless	Breguet parameter
40	zeta[out]	9.948e-05	dimensionless	Takeoff parameter
41	zeta[ret]	9.946e-05	dimensionless	Takeoff parameter
42	zeta[sprint]	9.945e-05	dimensionless	Takeoff parameter
43	T[out]	4.002e+04	N	Thrust per engine in cruise
44	T[ret]	4.002e+04	N	Thrust per engine in cruise
45	T[sprint]	4.002e+04	N	Thrust per engine in cruise
46	f_fuel[out]	1.204	dimensionless	Percent fuel remaining
47	f_fuel[ret]	1.107	dimensionless	Percent fuel remaining
48	f_fuel[sprint]	1.0	dimensionless	Percent fuel remaining
49	I_z[out]	4.364e+07	kg*m**2	Total aircraft moment of inertia
50	I_z[ret]	4.364e+07	kg*m**2	Total aircraft moment of inertia
51	I_z[sprint]	4.364e+07	kg*m**2	Total aircraft moment of inertia
52	I_zfuse[out]	3.903e+07	kg*m**2	Fuselage moment of inertia
53	I_zfuse[ret]	3.903e+07	kg*m**2	Fuselage moment of inertia
54	I_zfuse[sprint]	3.903e+07	kg*m**2	Fuselage moment of inertia
55	I_ztail[out]	3.37e+06	kg*m**2	Tail moment of inertia
56	I_ztail[ret]	3.37e+06	kg*m**2	Tail moment of inertia
57	I_ztail[sprint]	3.37e+06	kg*m**2	Tail moment of inertia
58	I_zwing[out]	1.24e+06	kg*m**2	Wing moment of inertia
59	I_zwing[ret]	1.24e+06	kg*m**2	Wing moment of inertia
60	I_zwing[sprint]	1.24e+06	kg*m**2	Wing moment of inertia
61	x_b	1.109e-06	m	Wing box forward bulkhead location
62	b_w	35.9	m	wingspan
63	cbar_w	6.519	m	mean aerodynamic chord(wing)
64	c_rootw	7.479	m	Wing root chord
65	c_tipw	1.496	m	Wing tip chord
66	lambda_w	0.2	dimensionless	wing taper ratio
67	S_w	161.1	m**2	Wing area
68	y_cbarw	8.107	m	Spanwise location of mean aerodynamic chord
69	C_Lw[out]	0.4834	dimensionless	Lift Coefficient, wing
70	C_Lw[ret]	0.4704	dimensionless	Lift Coefficient, wing
71	C_Lw[sprint]	0.4574	dimensionless	Lift Coefficient, wing
72	L_total[out]	1.002e+06	N	Total Lift generated by aircraft
73	L_total[ret]	9.755e+05	N	Total Lift generated by aircraft
74	L_total[sprint]	9.485e+05	N	Total Lift generated by aircraft
75	L_w[out]	8.354e+05	N	Wing Lift
76	L_w[ret]	8.129e+05	N	Wing Lift
77	L_w[sprint]	7.904e+05	N	Wing Lift
78	AR_w	8.0	dimensionless	Wing Aspect Ratio
79	I_cap	9.082e-06	dimensionless	Nondimensional spar cap area moment of inertia
80	M_r	1.452e+06	N	Root moment per root chord
81	p_w	1.4	dimensionless	Dummy variable (1+2*lambda_w)
82	q_w	0.9485	dimensionless	Dummy Variable (1+lambda_w)
83	tau_w	0.14	dimensionless	wing thickness/chord ratio
84	t_cap	0.00227	dimensionless	Nondimensional spar cap thickness
85	t_web	0.000991	dimensionless	Nondimensional shear web thickness
86	W_cap	4.985e+04	N	Weight of Spar caps
87	W_structw	5.442e+04	N	Wing box weight
88	W_web	4.571e+03	N	Weight of shear web
89	W_wing	6.178e+04	N	Wing Weight
90	v	0.8602	dimensionless	Dummy variable ((t^2+t+1)/(t+1)^2)

91	C_Dw[out]	0.006283	dimensionless	Drag coefficient,wing
92	C_Dw[ret]	0.00623	dimensionless	Drag coefficient,wing
93	C_Dw[sprint]	0.006182	dimensionless	Drag coefficient,wing
94	C_Diw[out]	0.000623	dimensionless	Wing Induced Drag Coefficient
95	C_Diw[ret]	0.0005899	dimensionless	Wing Induced Drag Coefficient
96	C_Diw[sprint]	0.0005577	dimensionless	Wing Induced Drag Coefficient
97	C_Dpw[out]	0.00566	dimensionless	Wing Parasitic Drag Coefficient
98	C_Dpw[ret]	0.00564	dimensionless	Wing Parasitic Drag Coefficient
99	C_Dpw[sprint]	0.005624	dimensionless	Wing Parasitic Drag Coefficient
100	D_w[out]	1.086e+04	N	Wing Drag
101	D_w[ret]	1.077e+04	N	Wing Drag
102	D_w[sprint]	1.068e+04	N	Wing Drag
103	f_lambdaw	0.003561	dimensionless	Empirical efficiency function of taper
104	Re_w[out]	4.204e+07	dimensionless	Cruise Reynolds number (wing)
105	Re_w[ret]	4.204e+07	dimensionless	Cruise Reynolds number (wing)
106	Re_w[sprint]	4.204e+07	dimensionless	Cruise Reynolds number (wing)
107	delt_xacw	4.031	m	Wing aerodynamic center shift
108	V_fuelmax	28.64	m**3	Available Fuel Volume
109	W_fuelwing	1.016e+05	N	Maximum fuel weight carried in wing
110	delt_xlvt	9.631	m	Distance from c.g. to VT leading edge
111	delt_xtvt	14.63	m	Distance from c.g. to VT trailing edge
112	l_fuse	33.18	m	Fuselage length
113	l_vt	37.0	m	Vertical tail moment arm
114	x_CG	18.55	m	x location of c.g.
115	x_CGvt	30.68	m	x location of vertical tail c.g.
116	b_vt	2.092	m	Vertical tail full-span
117	char_vt	3.564	m	Vertical tail mean aerochord
118	c_rootvt	5.0	m	Vertical tail root chord
119	c_tipvt	1.5	m	Vertical tail tip chord
120	lambda_vt	0.3	dimensionless	Span and Taper ratio
121	p_vt	1.6	dimensionless	Dummy variable (1+2*lambda_dvt)
122	q_vtd	1.3	dimensionless	Dummy Variable (1+lambda_dvt)
123	S_vt	10.16	m**2	Vertical tail reference area(half)
124	y_cbarvt	0.5667	m	Spanwise location of vertical tail mean aerodynamic chord
125	AR_vt	0.4309	dimensionless	Vertical tail aspect ratio
126	I_capvt	3.689e-07	dimensionless	Nondimensional vertical tail spar cap area moment of inertia
127	M_rvt	3.677e+04	N	Vertical tail root moment per root chord
128	tau_vt	0.14	dimensionless	Vertical tail thickness/chord ratio
129	t_capvt	0.0001393	dimensionless	Nondimensional spar cap thickness
130	t_webvt	0.000654	dimensionless	Nondimensional shear web thickness
131	W_capvt	127.0	N	Weight of vertical tail spar caps
132	W_webvt	195.7	N	Weight of vertical tail shear web
133	W_structvt	322.7	N	Vertical tail box weight
134	W_vt	358.2	N	Vertical tail weight
135	vvt	0.8176	dimensionless	Dummy variable ((t^2+t+1)/(t+1)^2)
136	C_LvtEO[out]	0.5677	dimensionless	Vertical tail lift coefficient during engine out
137	C_LvtEO[ret]	0.5677	dimensionless	Vertical tail lift coefficient during engine out
138	C_LvtEO[sprint]	0.5677	dimensionless	Vertical tail lift coefficient during engine out
139	D_wm[out]	3.617e+03	N	Engine out windmill drag
140	D_wm[ret]	3.617e+03	N	Engine out windmill drag
141	D_wm[sprint]	3.617e+03	N	Engine out windmill drag
142	L_vtEO[out]	1.731e+04	N	Vertical tail lift in engine out
143	L_vtEO[ret]	1.731e+04	N	Vertical tail lift in engine out
144	L_vtEO[sprint]	1.731e+04	N	Vertical tail lift in engine out
145	C_Lvtland[out]	2.582e+11	dimensionless	Vertical tail lift coefficient during landing
146	C_Lvtland[ret]	2.582e+11	dimensionless	Vertical tail lift coefficient during landing
147	C_Lvtland[sprint]	2.582e+11	dimensionless	Vertical tail lift coefficient during landing
148	C_Dpvt[out]	0.0177	dimensionless	Viscous drag coefficient
149	C_Dpvt[ret]	0.0177	dimensionless	Viscous drag coefficient
150	C_Dpvt[sprint]	0.0177	dimensionless	Viscous drag coefficient
151	D_vt[out]	1.929e+03	N	Vertical tail drag
152	D_vt[ret]	1.929e+03	N	Vertical tail drag
153	D_vt[sprint]	1.929e+03	N	Vertical tail drag
154	Re_vt[out]	2.299e+07	dimensionless	Vertical tail Reynolds number
155	Re_vt[ret]	2.299e+07	dimensionless	Vertical tail Reynolds number
156	Re_vt[sprint]	2.299e+07	dimensionless	Vertical tail Reynolds number
157	A_fuse	10.81	m**2	Fuselage cross-sectional area
158	A_skin	0.2554	m**2	Skin cross-sectional area
159	R_fuse	1.855	m	Fuselage radius
160	t_skin	0.02191	m	Skin thickness
161	wdt_fuse	3.71	m	Fuselage width
162	l_cone	18.55	m	Cone length
163	l_floor	28.11	m	Floor length
164	l_shell	24.4	m	Shell length
165	n_pass	186.0	dimensionless	Number of passengers
166	n_rows	31.0	dimensionless	Number of rows
167	S_bulk	21.62	m**2	Bulkhead surface area
168	S_nose	49.82	m**2	Nose surface area
169	V_bulk	0.4737	m**3	Bulkhead skin volume
170	V_cyl	6.23	m**3	Cylinder skin volume
171	V_cabin	314.6	m**3	Cabin Volume
172	V_nose	1.092	m**3	Nose skin volume
173	x_shell1	5.2	m	Start of cylinder section
174	x_shell2	29.6	m	End of cylinder section
175	A_floor	0.05326	m**2	Floor beam cross-sectional area
176	rho_cabin	0.8711	kg/m**3	Cabin air density
177	V_cone	0.1932	m**3	Cone skin Volume
178	V_floor	0.1664	m**3	Floor Volume
179	W_apu	5.208e+03	N	APU weight
180	W_cone	9.211e+03	N	Cone weight
181	W_floor	9.678e+03	N	Floor weight
182	W_fuse	5.063e+05	N	Fuselage weight

183	W_insul	4.305e+03	N	Insulation material weight
184	W_padd	5.952e+04	N	Miscellaneous weights (galley,toilets,doors,etc)
185	W_pay	1.488e+05	N	Payload weight
186	W_seats	3.311e+04	N	Seating weight
187	W_shell	3.717e+05	N	Shell weight
188	W_skin	2.065e+05	N	Skin weight
189	W_window	1.061e+04	N	Window weight
190	sigma_thta	4.995e+06	N/m**2	Skin hoop stress
191	sigma_x	1.85e+06	N/m**2	Axial stress in skin
192	t_shell	0.02958	m	Shell thickness
193	M_floor	4.263e+05	m*N	Maximum bending moment in floor beams
194	P_floor	1.091e+06	N	Distributed floor load
195	S_floor	5.457e+05	N	Maximum shear in floor beams
196	I_hshell	1.082e-25	m**4	Shell horizontal bending inertia
197	I_vshell	1.082e-25	m**4	Shell vertical bending inertia
198	sigma_Mh	1.245e-05	N/m**2	Horizontal bending material stress
199	sigma_Mv	1.245e-05	N/m**2	Vertical bending material stress
200	D_fuse[out]	6.602e+04	N	Fuselage Drag
201	D_fuse[ret]	6.602e+04	N	Fuselage Drag
202	D_fuse[sprint]	6.602e+04	N	Fuselage Drag
203	b_ht	10.29	m	Horizontal tail half-span
204	cbar_ht	1.822	m	Horizontal tail mean aerodynamic chord
205	c_rootht	2.645	m	Horizontal tail root chord
206	c_tipht	0.5289	m	Horizontal tail tip chord
207	lambda_ht	0.2	dimensionless	Horizontal tail taper ratio
208	p_ht	1.4	N/m	Horizontal tail theoretical wing loading
209	q_htd	1.2	dimensionless	Dummy Variable (1+lambda)
210	S_ht	18.87	m**2	Horizontal tail cross-sectional area
211	y_cbarht	2.94	m	Spanwise location of horizontal tail mean aerodynamic chord
212	C_Lht[out]	0.2543	dimensionless	Horizontal tail lift coefficient
213	C_Lht[ret]	0.2543	dimensionless	Horizontal tail lift coefficient
214	C_Lht[sprint]	0.2543	dimensionless	Horizontal tail lift coefficient
215	delt_xlht	11.99	m	Distance from c.g. to HT leading edge
216	delt_xtht	14.63	m	Distance from c.g. to HT trailing edge
217	l_ht	12.44	m	Horizontal tail moment arm
218	L_ht[out]	5.148e+04	N	Horizontal tail downforce
219	L_ht[ret]	5.148e+04	N	Horizontal tail downforce
220	L_ht[sprint]	5.148e+04	N	Horizontal tail downforce
221	L_htmax[out]	2.286e+06	N	Maximum horizontal tail downforce
222	L_htmax[ret]	2.286e+06	N	Maximum horizontal tail downforce
223	L_htmax[sprint]	2.286e+06	N	Maximum horizontal tail downforce
224	x_CGht	31.86	m	x location of horizontal tail c.g.
225	AR_ht	5.611	dimensionless	Horizontal tail aspect ratio
226	I_capht	4.484e-05	dimensionless	Nondimensional horizontal tail spar cap area moment of inertia
227	M_rht	7.482e+05	N	Horizontal tail root moment per root chord
228	tau_ht	0.14	dimensionless	Horizontal tail thickness/chord ratio
229	t_capht	0.0193	dimensionless	Nondimensional spar cap thickness
230	t_webht	0.006977	dimensionless	Nondimensional shear web thickness
231	W_capht	1.616e+04	N	Weight of horizontal tail spar caps
232	W_webht	1.534e+03	N	Weight of horizontal tail shear web
233	W_ht	1.858e+04	N	Horizontal tail weight
234	W_structht	1.769e+04	N	Horizontal tail box weight
235	vht	0.8562	dimensionless	Dummy variable ((t^2+t+1)/(t+1)^2)
236	C_Dht[out]	0.006058	dimensionless	Horizontal tail drag coefficient
237	C_Dht[ret]	0.006058	dimensionless	Horizontal tail drag coefficient
238	C_Dht[sprint]	0.006058	dimensionless	Horizontal tail drag coefficient
239	C_D0ht[out]	0.005812	dimensionless	Horizontal tail parasitic drag coefficient
240	C_D0ht[ret]	0.005812	dimensionless	Horizontal tail parasitic drag coefficient
241	C_D0ht[sprint]	0.005812	dimensionless	Horizontal tail parasitic drag coefficient
242	C_Diht[out]	0.0002458	dimensionless	Horizontal tail induced drag coefficient
243	C_Diht[ret]	0.0002458	dimensionless	Horizontal tail induced drag coefficient
244	C_Diht[sprint]	0.0002458	dimensionless	Horizontal tail induced drag coefficient
245	D_ht[out]	1.226e+03	N	Horizontal tail drag
246	D_ht[ret]	1.226e+03	N	Horizontal tail drag
247	D_ht[sprint]	1.226e+03	N	Horizontal tail drag
248	f_lambdaht	0.004297	dimensionless	Horizontal tail taper ratio function
249	Re_ht[out]	1.175e+07	dimensionless	Horizontal tail Reynolds number
250	Re_ht[ret]	1.175e+07	dimensionless	Horizontal tail Reynolds number
251	Re_ht[sprint]	1.175e+07	dimensionless	Horizontal tail Reynolds number
252	delt_xCG	0.07175	m	Change in x location of c.g.
253	m_ratio	1.085	dimensionless	Ratio of HT and Wing Lift Curve Slopes
254	SM	0.1576	dimensionless	Stability Margin
255	SM_min	0.1538	dimensionless	Minimum Stability Margin
256	BigQvt	1.099e+06	m*N	Fuselage bending moment at the vertical tail root
257	t_cone	0.0004915	m	Fuselage tail cone skin thickness
258	tau_cone	1.034e+08	dimensionless	Fuselage tail cone thickness to chord ratio

REFERENCES

REFERENCES

- [1] Price, M., Raghunathan, S., and Curran, R., “An Integrated Systems Engineering Approach to Aircraft Design,” *Progress in Aerospace Sciences*, Vol. 42, No. 4, June 2006, pp. 331–376.
- [2] Karcher, C. J., *An Optimization Centered Approach to Multifidelity Aircraft Design*, Dissertation, Massachusetts Institute of Technology, 2022.
- [3] Kirschen, P. G., York, M. A., Ozturk, B., and Hoburg, W. W., “Application of Signomial Programming to Aircraft Design,” *Journal of Aircraft*, Vol. 56, No. 1, 2019, pp. 389–403.
- [4] Drela, M., *TASOPT: Technical Reference and User’s Manual*, Massachusetts Institute of Technology (MIT), 2011, <http://web.mit.edu/drela/Public/web/tasopt/>.
- [5] Boyd, S., Kim, S.-J., Vandenberghe, L., and Hassibi, A., “A Tutorial on Geometric Programming,” *Optimization and Engineering*, Vol. 8, No. 1, 2007, pp. 67–127.
- [6] Hoburg, W. W. and Abbeel, P., “Geometric Programming for Aircraft Design Optimization,” *52nd Aerospace Sciences Meeting*, AIAA, National Harbor, MD, 2014.
- [7] Martins, J. R. R. A. and Lambe, A. B., “Multidisciplinary Design Optimization: A Survey of Architectures,” *AIAA Journal*, Vol. 51, No. 9, 2013, pp. 2049–2075.
- [8] Hart, W. E., *Pyomo: Python Optimization Modeling Objects*, 2022, <https://pyomo.readthedocs.io/>.
- [9] Hart, W. E., Laird, C. D., Watson, J.-P., Woodruff, D. L., Hackebeil, G. A., Nicholson, B. L., and Sirola, J. D., *Pyomo – Optimization Modeling in Python*, Vol. 67 of *Springer Optimization and Its Applications*, Springer, 2nd ed., 2017.
- [10] Raymer, D. P., *Aircraft Design: A Conceptual Approach*, AIAA, Washington, D.C., 1992.
- [11] Sadraey, M. H., “Tail Design,” Lecture notes, Daniel Webster College, 2012.
- [12] The Boeing Company, *Airplane Characteristics for Airport Planning: 737 Airplanes*, The Boeing Company, Seattle, WA, revision d ed., June 2020, https://www.boeing.com/commercial/airports/plan_manuals.page.
- [13] The Boeing Company, *Airplane Characteristics for Airport Planning: Next-Generation 737*, The Boeing Company, Seattle, WA, revision g ed., March 2023, https://www.boeing.com/commercial/airports/plan_manuals.page.
- [14] The B737 Technical Site, “Boeing 737 Detailed Technical Specifications,” <http://www.b737.org.uk/techspecs/techspecs.htm>, Accessed: 16 October 2025.
- [15] Kroo, I., Altus, S., Braun, R., Gage, P., and Sobieski, I., “Multidisciplinary Optimization Methods for Aircraft Preliminary Design,” Tech. rep., Stanford University, Stanford, California, 1994, Presented at the 5th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization.

- [16] Karcher, C. J., “Data fitting with signomial programming compatible difference of convex functions,” *Optimization and Engineering*, Vol. 24, 2022, pp. 973–987.
- [17] Karcher, C. and Haimes, R., “A Method of Sequential Log-Convex Programming for Engineering Design,” *Optimization and Engineering*, July 2022.
- [18] Karcher, C. J., “Logspace Sequential Quadratic Programming for Design Optimization,” *AIAA Journal*, Vol. 60, No. 3, March 2022, pp. 1471–1481.
- [19] Anderson, J. D., *Aircraft Performance and Design*, McGraw-Hill, Boston, Massachusetts, 1999.
- [20] Anderson, J. D., *Introduction to Flight*, Vol. 199, McGraw-Hill, Boston, 3rd ed., 1989.
- [21] Torenbeek, E., *Advanced Aircraft Design: Conceptual Design, Analysis and Optimization of Subsonic Civil Airplanes*, John Wiley Sons, Ltd, 2013.
- [22] Torenbeek, E., *Synthesis of Subsonic Aircraft Design*, Springer, Dordrecht, The Netherlands, 1982.
- [23] Anderson, J. D., *Fundamentals of Aerodynamics*, McGraw-Hill, Boston, MA, 2001.
- [24] Sobieski, J. S., “Multidisciplinary Design Optimization (MDO): Its Roots and Engineering Design Process Context,” Seminar at MIT, based on NASA Langley Research Center work, April 2010, Distinguished Research Associate, NASA Langley Research Center.
- [25] Henderson, R. P., Martins, J., and Perez, R. E., “Aircraft Conceptual Design for Optimal Environmental Performance,” *Aeronautical Journal*, Vol. 116, No. 1175, 2012, pp. 1–22.
- [26] Drela, M., “N3 Aircraft Concept Designs and Trade Studies—Appendix,” Cr-2010-216794, NASA, 2010.
- [27] Duffin, R. J., Peterson, E. L., and Zener, C., *Geometric Programming: Theory and Application*, Wiley, New York, 1967.
- [28] Hoburg, W., Kirschen, P., and Abbeel, P., “Data Fitting with Geometric-Programming-Compatible Softmax Functions,” *Optimization and Engineering*, Vol. 17, No. 4, 2016, pp. 897–918.
- [29] Burnell, E. and Hoburg, W., “GPkit Software for Geometric Programming,” Software Package, Version 0.5.3, 2017.
- [30] Kirschen, P. and Hoburg, W., “GPfit,” Software Package, Version 0.1, 2015.
- [31] Drela, M., “XFOIL: An Analysis and Design System for Low Reynolds Number Airfoils,” *Low Reynolds Number Aerodynamics*, Springer, Berlin, 1989, pp. 1–12.

ProQuest Number: 32279920

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2026).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA